ENGINEERING BOOTCAMP - WEEK 5 CASE STUDY

Deploying InfinityCastle on AWS

S3, EC2, RDS, Lambda, CloudWatch, Docker, ECR and ECS Fargate

Final edition: AWS from zero in pictures, every change that makes the laptop app run in the cloud, and the step-by-step record of both milestones as they were really deployed

What is inside	The class notes connected (Part 2); AWS from zero with diagrams - cloud, regions, IAM, networking, EC2, S3 and CORS, RDS, Docker and ECR, ECS and Fargate, load balancing, service discovery, configuration, CloudWatch, Lambda, scaling, cost (Part 3); design decisions (Part 4); every edit and every script explained (Part 5); the real Milestone 1 and 2 records with pause and demo-day runbooks (Parts 6-7); GitHub, troubleshooting, interview questions and appendices.
Application	InfinityCastle - Issue Tracker: eureka-server, api-gateway, user-service, project-service, issue-service (Spring Boot 4.1, Java 21, Spring Cloud) + React 19 frontend + MySQL.
Decisions taken	One EC2 per application; Lambda = profile-photo processor (S3 upload -> RDS); JARs built in Spring Tools, Docker images built on an EC2 "builder" as in class; region us-east-1.
Status	Milestone 1 (EC2) and Milestone 2 (ECS Fargate) are deployed and working. The Lambda and the CloudWatch dashboards and alarms are designed and explained as theory. Milestone 3 (GitHub) remains. Never put passwords, .pem keys or access keys in Git.
Prepared	20 October 2026 (final edition, after both deployments), for Nishant (Bruno)

Contents

How to use this guide, and the big picture 6

1.1 The case study in one page 6
1.2 What you will build - Milestone 1 6
1.3 What you will build - Milestone 2 7
1.4 Names used in this guide 8
1.5 Where things stand, and how to read this guide 8

The class notes - what was taught and how it all connects 9

2.1 The map: class demo -> case-study step 9
2.2 Cloud fundamentals (notes page 1) 9
2.3 Global infrastructure (page 2) 10
2.4 Virtualization and hypervisors (pages 2-4) 10
2.5 Connecting and hosting (pages 4-11) 10
2.6 Networking (pages 12-18) 11
2.7 Docker, ECR and ECS (pages 19-30) 11
2.8 RDS and S3 (pages 30-32) 12

AWS from zero - every idea behind the two milestones 13

3.0 The whole map on one page 13
3.1 What "the cloud" really is 14
3.2 Regions and Availability Zones 15
3.3 IAM - who may do what 16
3.4 Networking from zero 19
3.5 EC2 - renting a computer 25
3.6 S3 - storage and the React website 28
3.7 RDS - a database someone else looks after 31
3.8 Containers, Docker and ECR 34
3.9 ECS and Fargate - running containers for you 36
3.10 The Application Load Balancer 41
3.11 Service discovery - Eureka, Cloud Map and Service Connect 42
3.12 Configuration and secrets 45
3.13 CloudWatch - seeing what happens 46
3.14 Lambda - code without servers (designed, not deployed) 47
3.15 Scaling, high availability and disaster recovery 49
3.16 Other services worth knowing 50
3.17 What it costs (approximate, us-east-1) 51
3.18 The Well-Architected Framework 51

3.19 Glossary . . . 52

Design decisions, security plan and environments . . . 54

4.1 Decisions and their reasons . . . 54
4.2 How a request travels . . . 55
4.3 Least-privilege IAM . . . 55
4.4 Security-group matrix . . . 55
4.5 Secrets . . . 56
4.6 Environments: dev, test, prod (case-study guideline) . . . 56
4.7 Backups . . . 56
4.8 Authentication (non-functional requirement) . . . 56

Every change that makes the laptop app work in the cloud . . . 57

5.0 The principle - one codebase, three environments . . . 57
5.1 All edits at a glance . . . 58
5.2 Edit 1 - the aws profile of the three services . . . 59
5.3 Edits 2, 3 and 4 - the gateway and CORS . . . 60
5.4 Edit 5 - db.properties becomes optional . . . 61
5.5 Edit 6 - five Dockerfiles . . . 61
5.6 Edits 7, 8 and 9 - the aws/ folder . . . 62
5.7 Edits 11 and 12 - Git hygiene and documentation . . . 64
5.8 Cloud-side settings C1-C5 - the values that feed the placeholders . . . 65
5.9 Why scripts at all . . . 66
5.10 Every script, line by line . . . 67
5.11 Build the JARs and check that the laptop still works . . . 72

Milestone 1 - manual deployment, step by step . . . 73

6.0 What is different from the first plan . . . 73
6.1 Your notepad worksheet - what to save, what not to save . . . 74
6.2 Prepare the laptop (once) . . . 76
6.3 Key pair . . . 76
6.4 Network: the VPC (wizard) . . . 76
6.5 IAM role for the EC2 servers . . . 77
6.6 Security groups (final, working rules) . . . 77
6.7 RDS MySQL . . . 77
6.8 The five EC2 application servers . . . 79
6.9 Create the three schemas and the three database users . . . 80
6.10 Deploy user, project, issue and gateway . . . 82
6.11 Verify the backend . . . 83
6.12 The React frontend on S3 . . . 83
6.13 Milestone 1 status and evidence . . . 85

6.14 Lambda: the profile-photo processor (designed - not deployed) . . . 85
6.15 CloudWatch dashboards and alarms (designed) . . . 88
6.16 Test plan (Milestone 1, task F) . . . 88
6.17 What persists and what changes . . . 89
6.18 Pausing between now and the interview . . . 90
6.19 Demo-day runbook: bring Milestone 1 back and show it . . . 90
6.20 Problems met during the real deployment . . . 91

Milestone 2 - containers on ECR and ECS Fargate, step by step . . . 93

7.0 What changes from Milestone 1 . . . 93
7.1 Worksheet additions for Milestone 2 . . . 95
7.2 Step 1 - the builder EC2 . . . 96
7.3 Step 2 - ECR repositories, copy, build and push . . . 96
7.4 Step 3 - test the five containers on the builder . . . 97
7.5 Step 4 - security groups and the Cloud Map namespace . . . 98
7.6 Step 5 - cluster, load balancer and the IAM fix . . . 98
7.7 Step 6 - the five task definitions (final values) . . . 99
7.8 Step 7 - the five services . . . 100
7.9 Step 8 - changing a running service: new revision, then deploy it . . . 101
7.10 Step 9 - verify, then the React site for Milestone 2 . . . 101
7.11 Milestone 2 status . . . 102
7.12 Proving orchestration in the demo . . . 102
7.13 CloudWatch for Milestone 2 - what exists, what is theory . . . 103
7.14 What persists, pausing and the demo day . . . 103
7.15 Problems met during Milestone 2 . . . 104

Milestone 3 - GitHub . . . 105

8.1 What goes into Git . . . 105
8.2 Steps . . . 105

Troubleshooting, cost and clean-up . . . 106

9.1 Troubleshooting . . . 106
9.2 What costs money (approximate, us-east-1) . . . 107
9.3 Clean-up order . . . 107

Interview questions and answers . . . 108

Architecture and design . . . 108
Networking and security . . . 108
Services . . . 109
Problems we solved (tell them as short stories) . . . 110

Remaining files, copy-paste ready 111

Command cheat sheet 113

PART 1

How to use this guide, and the big picture

What the case study asks, what you will build, the names used everywhere, and the order of work.

1.1 The case study in one page

Milestone	What must be delivered (from the case study)
1 - Manual deployment	React build on **S3** static website hosting; each Spring Boot microservice on its own **EC2** (Java installed, JAR deployed, security groups); **RDS MySQL** (schemas, users, services connected); **AWS Lambda** for an event-driven task triggered by **S3 uploads**, with permissions for S3 and RDS; **CloudWatch** logs and metrics for EC2 and Lambda (dashboards and alarms optional); end-to-end test.
2 - Containers	A **Dockerfile** per service, images built and tested; one **ECR** repository per service, Docker CLI authenticated, images pushed; an **ECS cluster on Fargate**, a task definition per service, service discovery, networking and security groups, ECS services; RDS through task environment variables; CloudWatch for tasks; end-to-end test.
3 - GitHub	Push the build to the GitHub repository (Deloitte GitHub Enterprise, per the guidelines).
Technical interview	Discussion of assumptions, functionalities and validations - Parts 3, 4 and 10 prepare you for it.

> **NOTE**
> Guidelines from the case study that shape every step: best coding practices; assumptions allowed; a simple working prototype beats a good-looking broken one; **comment the deployment steps and configuration**; **separate dev, test and prod environments**; **least-privilege IAM roles and security groups**; **regular database backups**; **Deloitte GitHub Enterprise only**; Git with pull requests. Best practices: clear resource naming, containerization, Eureka or AWS-native service discovery, Spring Cloud Gateway, CloudWatch, horizontal scaling, Thunder Client tests, Swagger docs.

1.2 What you will build - Milestone 1

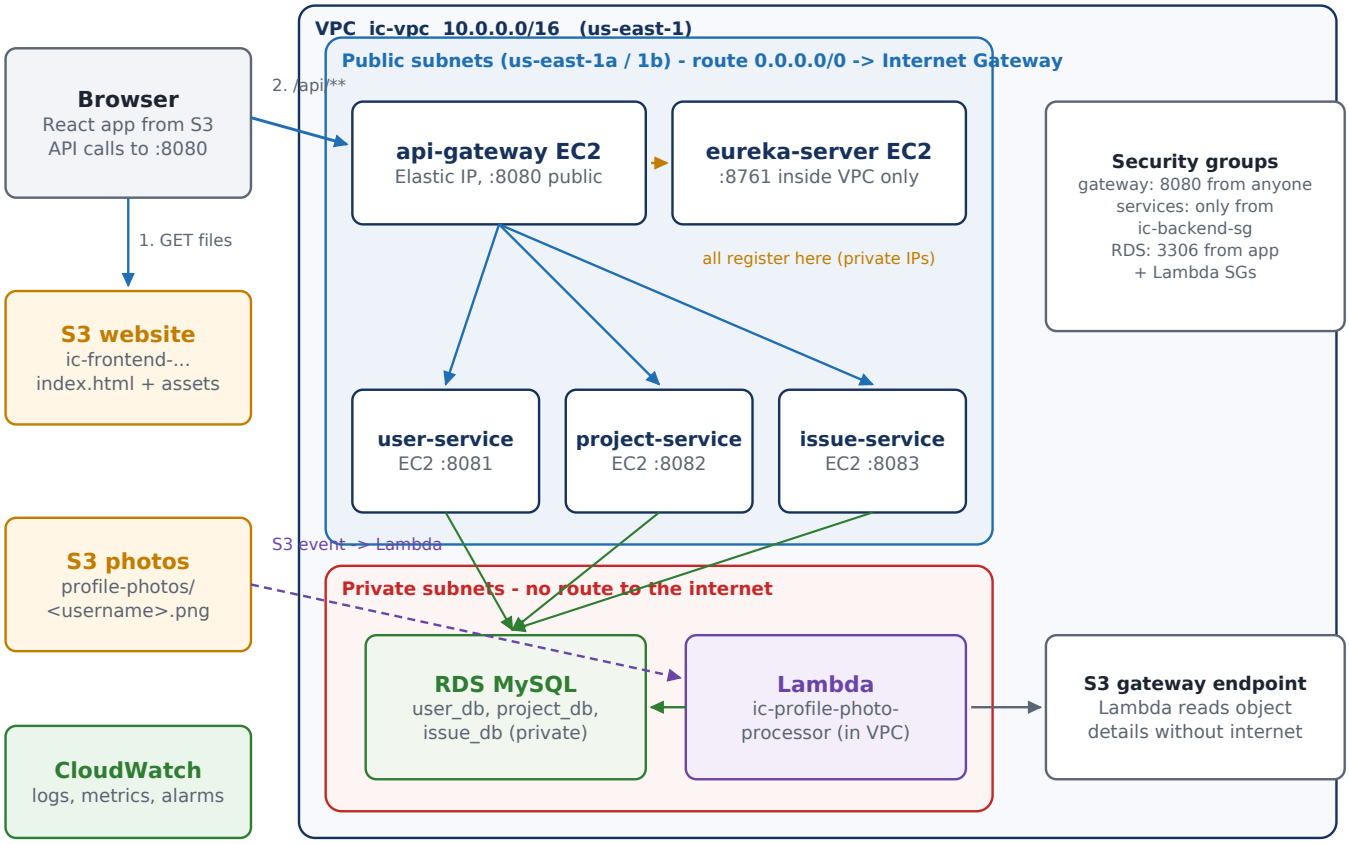


Figure 1.1 - Milestone 1. The browser loads the React files from S3 and calls the gateway on its Elastic IP. Everything else stays inside the VPC. The photos bucket and the Lambda are the design (not deployed).

1.3 What you will build - Milestone 2

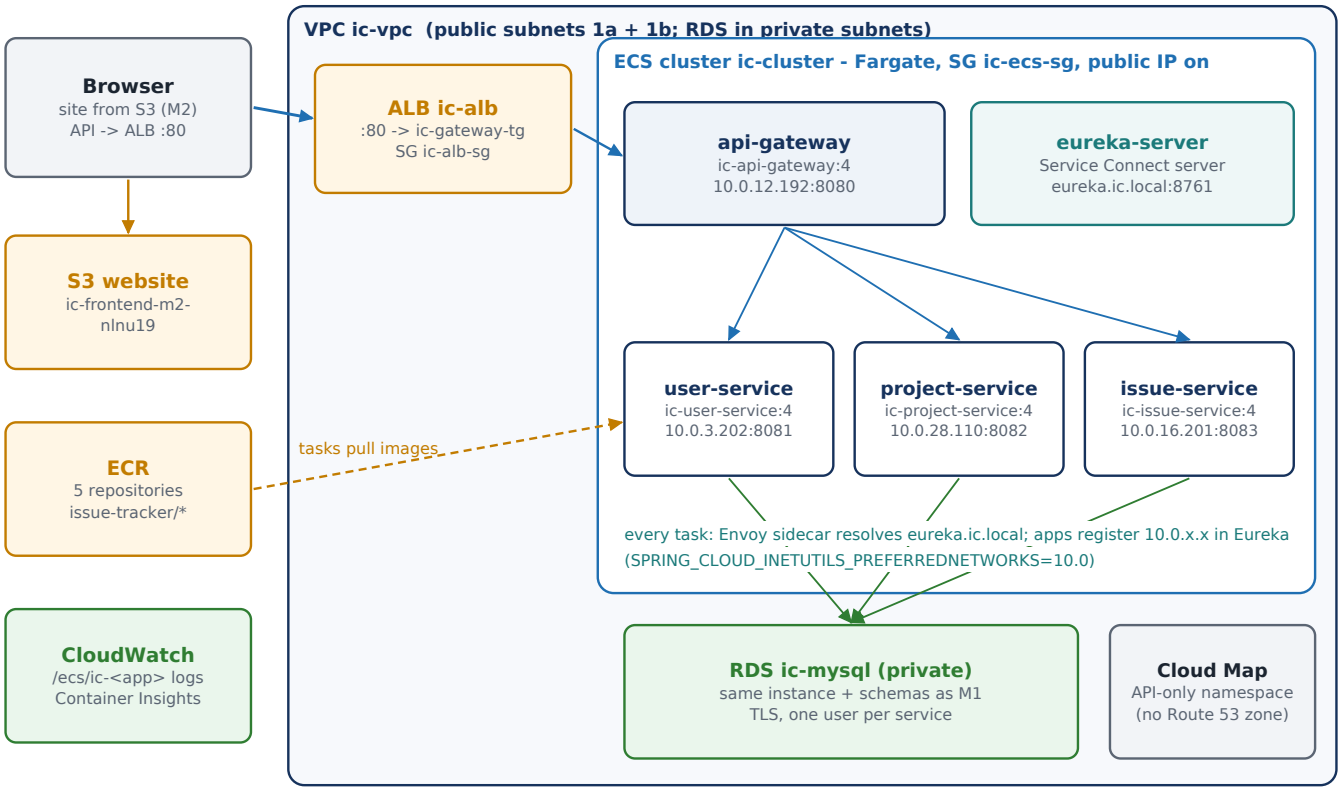


Figure 1.2 - Milestone 2 as deployed. The same five JARs run as containers on Fargate; an Application Load Balancer is the public entry point; Eureka is found as eureka.ic.local through ECS Service Connect.

1.4 Names used in this guide

Clear, consistent naming is a case-study best practice. Every resource starts with **ic** (InfinityCastle). Replace `<you>` with something unique such as your initials and a number - S3 bucket names are global across all AWS accounts.

Resource	Name
Region / AZs	`us-east-1` (N. Virginia), `us-east-1a`, `us-east-1b`
Key pair	`ic-key` (file `ic-key.pem`, kept outside the project folder)
VPC and subnets	`ic-vpc` (10.0.0.0/16), created by the "VPC and more" wizard with project name `ic`
Security groups	`ic-backend-sg`, `ic-gateway-public-sg`, `ic-rds-sg`, `ic-lambda-sg`, `ic-builder-sg`, `ic-alb-sg`, `ic-ecs-sg`
EC2 instances (M1)	`ic-eureka`, `ic-gateway`, `ic-user`, `ic-project`, `ic-issue`; builder (M2): `ic-builder`
IAM roles	`ic-ec2-cloudwatch-role`, `ic-builder-role`, `ic-photo-lambda-role`, `ecsTaskExecutionRole`
RDS	instance `ic-mysql`, schemas `user_db`, `project_db`, `issue_db`
S3 buckets	`ic-frontend-<you>` (website, M1), `ic-frontend-m2-<you>` (website, M2), `ic-photos-<you>` (profile photos); yours use `<you>` = `nlnu19`
Lambda	`ic-profile-photo-processor`
ECR repositories	`issue-tracker/eureka-server`, `.../api-gateway`, `.../user-service`, `.../project-service`, `.../issue-service`
ECS	cluster `ic-cluster`, task families `ic-<app>`, services `ic-<app>-svc`, namespace `ic.local`
Load balancer	`ic-alb`, target group `ic-gateway-tg`
CloudWatch	log groups `/issue-tracker/m1/<app>`, `/ecs/ic-<app>`, `/aws/lambda/...`; dashboard `ic-dashboard`

1.5 Where things stand, and how to read this guide

Item	Status (20 Oct 2026)	Read
Project changes for AWS	applied; JARs built	Part 5 - every edit and script
Milestone 1 - EC2, RDS, S3	**done** 19 Oct	Part 6 (record, pause, demo day)
Milestone 2 - Docker, ECR, ECS Fargate, ALB	**done** 19-20 Oct	Part 7 (record, pause, demo day)
Lambda (S3 -> Lambda -> RDS)	designed and coded, **not deployed** - theory	3.14, 5.6, 6.14
CloudWatch	logs + automatic metrics + Container Insights; dashboards and alarms as theory	3.13, 6.15, 7.13
Full test table	basic checks done; table kept as the test plan	6.16, 7.12
Milestone 3 - GitHub	to do	Part 8

1. **To understand:** Part 3 from the start - each chapter has a picture, the idea, what we did and an interview sentence. Then Part 5 (why the code looks the way it does).
2. **To rebuild or repeat:** Part 6 (Milestone 1) and Part 7 (Milestone 2) in order - they are written as the sequence that worked, with every mistake we met and its fix.
3. **Between sessions:** what survives a stop/start - 6.17 and 7.14; how to pause - 6.18 and 7.14; demo-day start-up - 6.19 and 7.14.
4. **Before the interview:** Part 10 (questions and answers) and the "Interview angle" boxes of Part 3.
5. **Clean up (Part 9)** when the case study is over, so nothing keeps costing money.

PART 2

The class notes - what was taught and how it all connects

Every topic from the 32 pages of notes, explained more fully, with the corrections the handwriting needs, and where each one is used in this case study.

2.1 The map: class demo -> case-study step

Class topic / demo	What you practised	Used in this case study
Cloud basics, regions, AZs	availability, scalability, elasticity, deployment and service models	choosing us-east-1 and two AZs; the design discussion (Part 4)
Virtualization, hypervisors, EC2 (Demo 1)	VMs, AMIs, key pairs, instance types, connecting	the five application servers and the builder
SSH, PuTTY, Instance Connect	`ssh -i key.pem ec2-user@host`, .ppk keys	every server task; SSH tunnels to private ports
httpd / Nginx (Demo 2)	install a server, open port 80 in the security group	the same pattern for port 8080
Spring Boot on EC2 (Demo 3)	Corretto 21, `mvn clean package`, `scp`, `java -jar`	Milestone 1 backend
VPC, subnets, IGW, routes, NAT, bastion (Demo 4)	networking from scratch	ic-vpc, public/private subnets, private RDS
Docker (Demos 6-7)	images, containers, `docker run -p`, Dockerfile	Milestone 2 images
ECR (Demo 8)	repository, `get-login-password`, tag, push, pull	Milestone 2 registry
ECS	cluster, task definition, task, service	Milestone 2 on Fargate
RDS + DBeaver	create MySQL, connect with endpoint, update application.properties	shared database (both milestones)
S3 static website	`npm run build`, bucket, policy, website hosting	React frontend

Not covered in the notes but required by the case study: **IAM roles** (needed by EC2, Lambda and ECS), **AWS Lambda**, **CloudWatch**, **Application Load Balancer** and **Cloud Map**. They are explained from scratch in Part 3.

2.2 Cloud fundamentals (notes page 1)

Term	Meaning	Example in our deployment
Availability	The system keeps answering even when a part fails	RDS backups; ECS restarts a crashed task automatically
Scalability	Can grow for long-term demand (bigger or more machines)	add a second user-service task (Part 7.12)
Elasticity	Grows and shrinks automatically with short-term demand	ECS service auto scaling on CPU (optional)
Load balancing	Spreads requests over several targets	ALB in front of the gateway; Spring Cloud LoadBalancer between services
Managed services	AWS runs the servers; you use the service	RDS, S3, Lambda, Fargate, ECR, CloudWatch

The load-balancer list in the notes reads "Application: ALB, Network: NLB, Database: DLB". AWS has no "DLB": the four Elastic Load Balancing types are **ALB** (Application, layer 7, HTTP routing), **NLB** (Network,

layer 4, TCP/UDP, very high throughput), **GWLB** (Gateway, for third-party firewall appliances) and the old **CLB** (Classic). The third item was most likely GWLB.

- **Deployment models:** public cloud (AWS), private cloud (your own data centre run like a cloud), hybrid (both, connected, e.g. with Direct Connect), multi-cloud (several providers).
- **Service models:** IaaS = you get machines and networks (EC2, VPC) and manage the OS; PaaS = you bring code or containers and the platform runs them (Elastic Beanstalk, Fargate, RDS for databases); SaaS = finished software (Gmail). FaaS/serverless (Lambda) is PaaS taken further: you bring a function.
- Milestone 1 is mostly **IaaS** (you patch and run the EC2s); Milestone 2 moves to **PaaS/CaaS** (Fargate runs the containers, you never see a server). Being able to say this is a good interview answer.

2.3 Global infrastructure (page 2)

A **Region** is a geographic area (us-east-1 = N. Virginia) with its own copy of most services. Each region has several **Availability Zones** - one or more separate data centres with their own power and network, close enough for fast links. **Edge locations** are small sites used by CloudFront (the CDN) to cache content near users. The notes quote 36 regions, 6 AZs in N. Virginia and 140+ edge locations; these numbers grow every year, so check the AWS global infrastructure page before quoting them. The unclear "240+" was most likely "245+ countries and territories served".

> **WHY TWO AZS**
> We use **two AZs** because RDS needs a subnet group covering at least two AZs and an ALB needs subnets in at least two AZs. Spreading across AZs is the basic high-availability pattern.

2.4 Virtualization and hypervisors (pages 2-4)

- A **virtual machine** has its own guest OS, binaries/libraries and application, and shares the physical server with other VMs through a **hypervisor**.
- **Type 1 (bare metal):** the hypervisor runs directly on the hardware - VMware ESXi, Microsoft Hyper-V, Xen, KVM, Oracle VM Server. AWS runs EC2 on its **Nitro** hypervisor (KVM-based, with networking and storage offloaded to dedicated cards).
- **Type 2 (hosted):** the hypervisor is an application on a normal OS - VirtualBox, VMware Workstation. Good for laptops, slower for servers.
- **EC2** gives you a VM ("instance") from an **AMI** (a disk image such as Amazon Linux 2023), an **instance type** (CPU/RAM size, e.g. t3.micro = 2 vCPU, 1 GiB), an **EBS volume** (the disk; 8 GB default in the notes) and a **key pair** for SSH.

2.5 Connecting and hosting (pages 4-11)

- `ssh -i "ic-key.pem" ec2-user@<public-DNS-or-IP>` - `ec2-user` is the default user of Amazon Linux. Windows 10/11 has OpenSSH built in, so this works in PowerShell; PuTTY needs the key converted to .ppk with PuTTYgen.
- **EC2 Instance Connect** opens a browser terminal without your key (it pushes a temporary key); it needs port 22 open to the Instance Connect service range, so "My IP" rules block it.
- **Private vs public IP:** the private IP (10.0.x.x) is used inside the VPC and never changes for the life of the instance; the public IP changes on every stop/start unless you attach an **Elastic IP**. That is why the gateway gets an Elastic IP: its address is compiled into the React build.
- The httpd and Nginx demos show the universal recipe: install, `systemctl enable` + `start`, open the port in the **security group**, browse to it. The notes mention changing httpd's `Listen 80` to another port (the handwriting looks like 5680): any port works if it is opened in the security group too.
- Demo 3 (Spring Boot): `sudo yum install -y java-21-amazon-corretto-devel`, build with `mvn clean package`, copy with `scp`, run with `java -jar`. Milestone 1 uses exactly this, plus systemd so the service

restarts by itself and survives a reboot. (`-headless` is enough on a server: it is the runtime without the desktop libraries; `-devel` adds the compiler.)

2.6 Networking (pages 12-18)

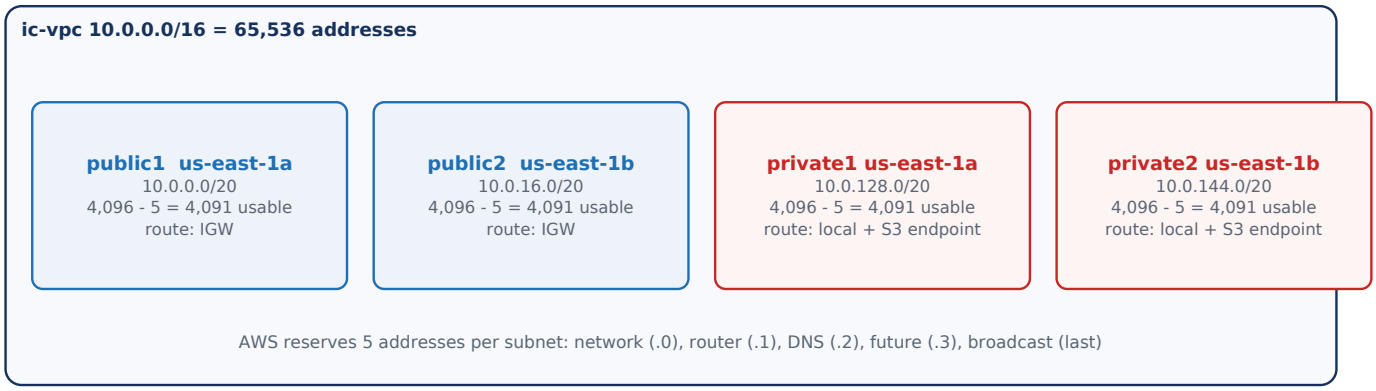


Figure 2.1 - The VPC the wizard builds for us: two public and two private /20 subnets in two AZs.

- **VPC** = your own isolated network in a region. **CIDR** (Classless Inter-Domain Routing) gives its address range: `10.0.0.0/16` means the first 16 bits are fixed, leaving 2^16 = 65,536 addresses.
- **Subnets** split the VPC and each lives in one AZ. A /20 has 2^12 = 4,096 addresses; AWS keeps 5, so 4,091 are usable. A subnet is **public** only because its **route table** sends `0.0.0.0/0` to an **Internet Gateway**; otherwise it is **private**.
- **NAT Gateway** (in a public subnet, with an Elastic IP) lets private resources reach the internet outbound only. It costs money per hour, so this guide avoids it: private resources here (RDS, Lambda) do not need the internet, and Lambda reaches S3 through a free **S3 gateway endpoint**.
- **VPC peering** joins two VPCs; **Transit Gateway** is a hub for many VPCs; **Direct Connect** is a private line from an office to AWS. Not needed here, but know the differences.
- **Bastion host (jump box):** a public instance used to reach private ones. The class copied the .pem onto the bastion; never do that outside a lab. Use `ssh -J ec2-user@<bastion> ec2-user@<private-ip>` (ProxyJump) - the key never leaves your laptop. In this guide `ic-gateway` doubles as the bastion for reaching RDS.

CORRECTIONS TO THE NOTES

Two values in the notes need care. (1) The VPC CIDR reads `12.35.0.0/16`, but the subnets are `10.35.16.0/20` and `10.35.32.0/20`. A subnet must lie inside its VPC, so the VPC must have been `10.35.0.0/16`. Always use private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) - 12.x.x.x belongs to real internet hosts. (2) The private IP `10.35.16.5` is plausible: the first usable address in 10.35.16.0/20 is .4, because AWS reserves .0 to .3.

2.7 Docker, ECR and ECS (pages 19-30)

- A **container** shares the host's kernel (no guest OS), so it starts in seconds and is small. An **image** is the read-only template built from a **Dockerfile**; a container is a running instance of it.
- `docker run -d -p 5566:80 nginx:latest` maps host port 5566 to container port 80. On **Fargate** there is no host port mapping: each task gets its own network card and IP (`awsvpc` mode), so the host port always equals the container port. The class example `http://<ip>:8989/api/hello` (8989 -> 8080) only works with `docker run` on an EC2.
- `docker ps` (running), `docker ps -a` (all), `docker images`, `docker stop`, `docker logs`. Without `sudo`, add `ec2-user` to the docker group (`sudo usermod -aG docker ec2-user`, then log in again).
- **ECR** is AWS's private Docker registry. `aws ecr get-login-password ... | docker login ...` gives Docker a token valid for 12 hours; then `docker tag` + `docker push`.

- **ECS** concepts from the notes: a **task definition** (blueprint: image, CPU, memory, ports, environment, logs), a **task** (a running copy), a **service** (keeps N tasks running, replaces failed ones, joins them to a load balancer) and a **cluster** (logical group). **Fargate** is the serverless launch type: no EC2 to manage.

2.8 RDS and S3 (pages 30-32)

- RDS: "Standard create" (the notes' "full configuration"), MySQL, Single-AZ, db.t3.micro, master user and password, VPC and security group, then connect with the **endpoint** (a DNS name such as `ic-mysql.xxxx.us-east-1.rds.amazonaws.com`) - in DBeaver and in `application.properties`.
- In class the database was made publicly accessible so DBeaver could reach it directly. Here RDS stays **private** (least privilege), and DBeaver connects through an **SSH tunnel** via `ic-gateway` (Part 6.9).
- S3 static website: `npm run build`, create a bucket, turn off "Block all public access", add a bucket policy allowing `s3:GetObject`, enable static website hosting with `index.html`. For a React app with React Router the **error document must also be** `index.html` (not error.html), otherwise refreshing `/owner/dashboard` returns a 404 from S3.
- The last page mentions uploading a WAR/JAR: in this case study the JARs go to EC2 (Milestone 1) or into Docker images (Milestone 2); S3 only holds the React files and the profile photos.

PART 3

AWS from zero - every idea behind the two milestones

Written to be understood without any previous cloud knowledge. Each chapter goes: the idea in plain words, a picture, how it really works underneath, how it appears in our project, the mistakes we actually made, and the sentence to say in the interview. Read it once slowly; afterwards the diagrams alone are enough to revise.

3.0 The whole map on one page

AWS has more than 200 services, but this project uses about twenty. They fall into eight families. Every box below appears somewhere in Milestone 1 or 2 (Lambda only as a design). If you can explain each box in two sentences and say where it sits in our system, you know enough AWS for this case study.

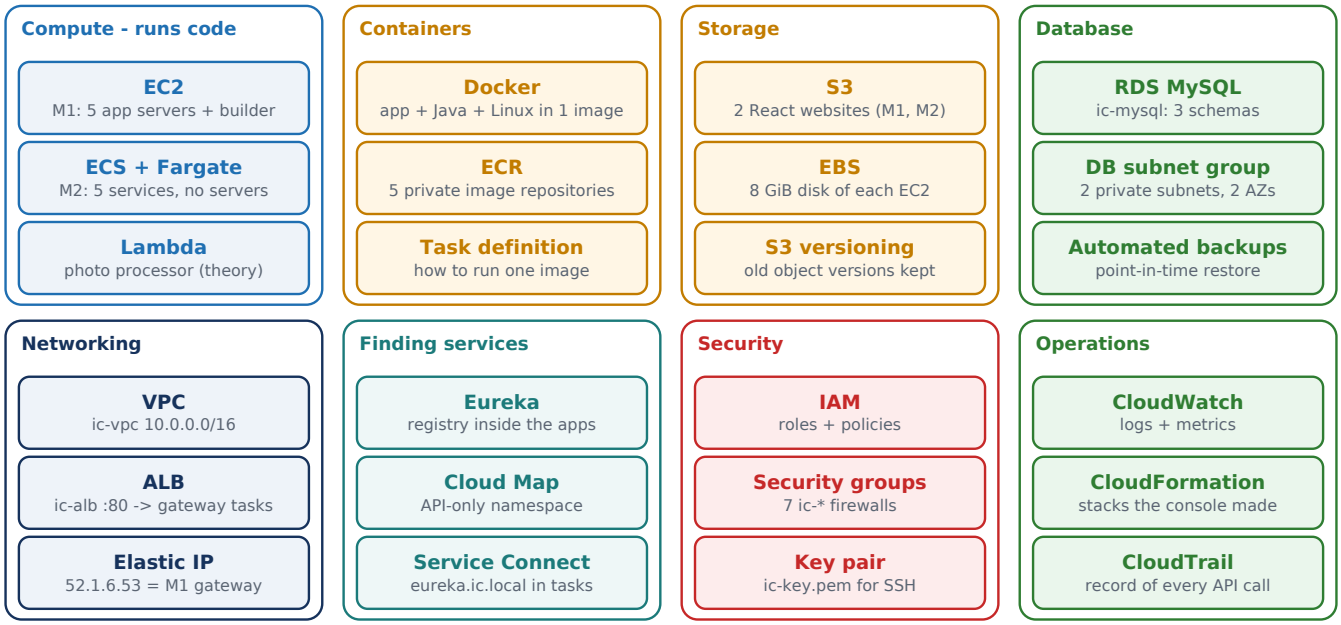


Figure 3.1 - The AWS services of this project, grouped by what they do.

The fastest way to make these services feel familiar is to map them to things you already know. Keep this table in mind; the rest of the part fills in the details.

AWS thing	Everyday picture	What it really is
Region (us-east-1)	a city	a geographic area with several separate data-centre groups
Availability Zone	a district of the city with its own power station	one or more data centres, isolated from the other AZs
VPC	your gated campus inside the city	your private network, address range 10.0.0.0/16
Subnet	one building on the campus	a slice of the VPC addresses that lives in exactly one AZ
Route table	the signposts at the building exit	rules "to reach X, go through Y"
Internet Gateway	the campus main gate to the public road	the VPC's door to the internet
Security group	a guard at each office door with a guest list	a firewall on each network card
EC2 instance	a rented computer in a server room	a virtual machine
EBS volume	the hard disk plugged into that computer	a network disk that survives stop/start
S3 bucket	an endless warehouse of labelled boxes	object storage; also hosts static websites

AWS thing	Everyday picture	What it really is
RDS	a database with a full-time caretaker	MySQL that AWS installs, patches and backs up
IAM role	a visitor badge handed out at the door, valid for an hour	permissions without a password, given as temporary keys
Docker image	a sealed shipping container with the goods inside	your app + Java + a small Linux, frozen
ECR	the depot where the containers are stored	a private registry for images
ECS service	a harbour master who always keeps N containers unloaded	keeps N copies of a task running
Fargate	cranes you rent by the minute and never see	serverless machines that run your tasks
ALB	a receptionist sending visitors to a free desk	a load balancer with one fixed DNS name
Eureka	the staff phone directory	a registry: service name -> current IP:port
CloudWatch	CCTV and the building logbook	metrics, logs, alarms
Lambda	an on-call contractor paid per job	a function AWS runs only when an event arrives
CloudFormation	a construction plan executed by robots	infrastructure created from a template

ONE FACT THAT EXPLAINS MANY ERRORS
Every click in the console is an **API call** such as `rds:CreateDBInstance` or `ecs:CreateService`. That is why the error messages you saw name actions: the console just calls the same API that the AWS CLI and SDKs call. Permissions are checked per API call.

3.1 What "the cloud" really is

"The cloud" means renting computing from someone else's data centre, by the second, through an API, instead of buying servers. Five properties make it different from a rented server in a basement:

Property	Meaning	Seen in this project
On demand, self-service	create a server, database or network in minutes with no human involved	five EC2s, an RDS database and a VPC created in one afternoon
Pay as you go	billed per second / hour / GB; stop paying when you delete it	stopping EC2s and setting ECS tasks to 0 to save credits
Elasticity	capacity grows and shrinks with the load, automatically	ECS service auto scaling (denied in our sandbox)
Managed services	AWS runs the boring parts (patching, backups, failover)	RDS, Fargate, Lambda, S3
Global	the same services in many regions	we use us-east-1 only

Service models - how much AWS does for you

Model	You manage	AWS manages	Our example
IaaS (infrastructure)	OS, Java, app, data	hardware, virtualisation	EC2 in Milestone 1
CaaS (containers)	image (app + Java), config	servers, OS of the hosts, scheduling	ECS Fargate in Milestone 2
PaaS (platform)	data, schema, users	engine, OS, patches, backups	RDS MySQL
FaaS / serverless	function code	everything else, scaling to zero	Lambda (designed)
SaaS (software)	only your data	the whole application	Gmail, GitHub

The shared responsibility model

Security is split. AWS secures the cloud itself (buildings, hardware, the hypervisor, the inside of managed services). You secure what you put in it (who can log in, which ports are open, your code, your data, the passwords). The more managed the service, the more rows belong to AWS:

	Your laptop	EC2 (M1)	ECS Fargate (M2)	RDS	Lambda
Your data, users, IAM, security groups	You	You	You	You	You
Your application code + its config	You	You	You	You**	You
Runtime (JVM, Python, MySQL engine)	You	You	You*	AWS	AWS
Operating system + security patches	You	You	AWS	AWS	AWS
Virtualisation / host machines	You	AWS	AWS	AWS	AWS
Hardware, network, buildings, power	You	AWS	AWS	AWS	AWS

* Fargate: Java is inside YOUR image (amazoncorretto:21-alpine), so you choose and update it. ** RDS: your schemas, users and queries.
The more managed the service, the more rows AWS owns - and the less you can (and must) change.

Figure 3.2 - Who is responsible for what, per service.

Word	Simple definition	Example
Scalability	the system CAN handle more load when you add resources	run 2 user-service tasks instead of 1
Elasticity	resources are added AND removed automatically with the load	target tracking at 60% CPU
Availability	the share of time it works (99.9% = 8.8 h down per year)	two AZs, health checks
Fault tolerance	keeps working while a part has failed	ECS replaces a crashed task
Durability	data is not lost	S3: 99.999999999% (11 nines) per object per year

THINK OF IT LIKE THIS
Owning a car (on premises) - you buy, insure and repair it. Renting a car (EC2) - no purchase, but you drive and refuel. A taxi (Fargate, RDS) - you only say where to go. A bus ticket (Lambda) - you pay per ride and someone else worries about everything.

3.2 Regions and Availability Zones

A **region** is a geographic area (us-east-1 = Northern Virginia, ap-south-1 = Mumbai). Each region contains at least three **Availability Zones (AZs)**: groups of data centres a few kilometres apart, each with its own power, cooling and network, linked by fast private fibre. A fire or power cut in one AZ does not touch the others. This is the foundation of high availability on AWS: put copies of important things in two or more AZs.

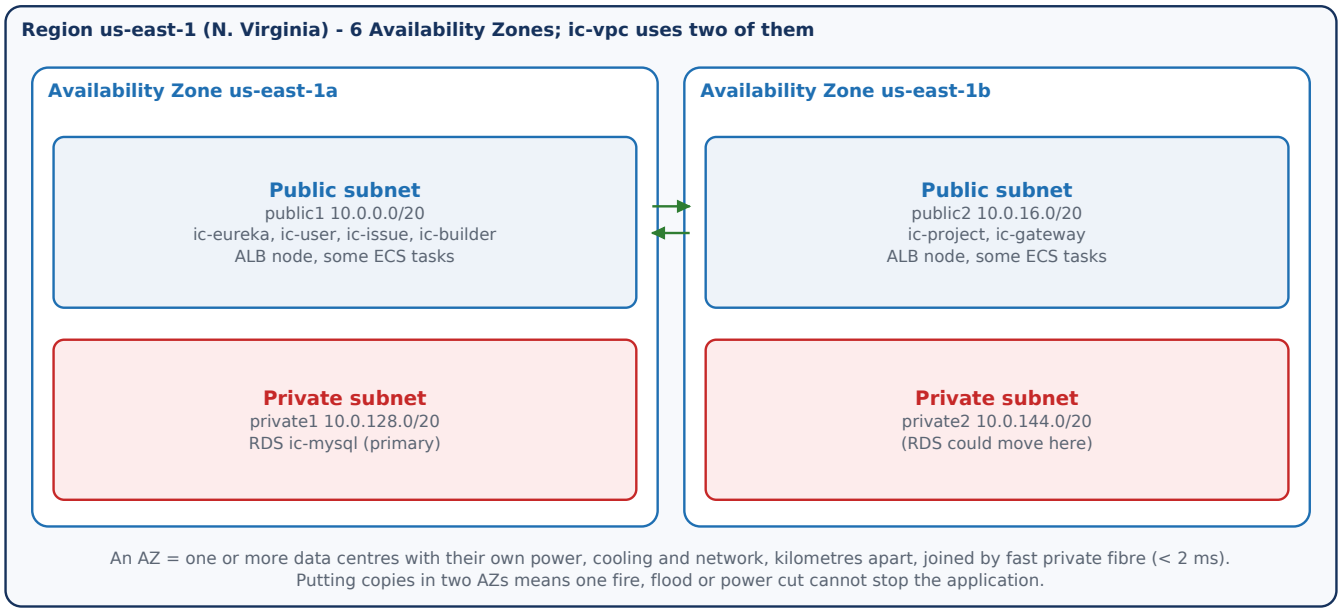


Figure 3.3 - Where our resources live inside us-east-1.

Scope	Meaning	Our resources
Global	one copy for the whole account	IAM users, roles and policies; Route 53; CloudFront
Regional	lives in one region, can span its AZs	VPC, S3 buckets (data), ECR, ECS cluster, ALB, Lambda, CloudWatch, Cloud Map, security groups
Zonal	lives in exactly one AZ	subnets, EC2 instances, EBS volumes, a Single-AZ RDS, each Fargate task

- **How to choose a region:** latency to users, price (us-east-1 is usually cheapest), which services exist there, and data-residency laws. We use us-east-1 because the bootcamp and the class demos did.
- **The region selector (top right) matters:** a resource created in Mumbai is invisible while the console shows N. Virginia. EC2 Global View and Tag Editor (All regions) show everything at once - that is how the account was cleaned before Milestone 2.
- **S3 bucket names are global** (no two accounts may share one) although the data stays in one region - hence names like `ic-frontend-nlnu19`.
- **Edge locations** (hundreds of them) are small sites used by CloudFront and Route 53 to be close to users.

3.3 IAM - who may do what

Every request to AWS - a console click, a CLI command, a call from the CloudWatch agent - carries an identity and asks for one **action** on one **resource**. IAM (Identity and Access Management) decides yes or no. It is global, free, and the most important security service: almost every problem in this project that was not a network problem was an IAM decision.

Identities - who is asking

Identity	What it is	In this project
Root user	the e-mail that opened the account; can do everything; never use it day to day	owned by the bootcamp
IAM user	a person or program with long-term credentials (password, access keys)	-
IAM role	permissions with NO password; something "assumes" it and receives temporary keys	EC2, builder, ECS and Lambda roles

Identity	What it is	In this project
Federated user	a temporary session created by another system (an SSO portal)	you: `federated-user/edfawsoctuser95` - the bootcamp portal logs you in for a limited time with limited rights
Service principal	an AWS service acting on its own	`ecs-tasks.amazonaws.com`, `s3.amazonaws.com`

Policies - the rule documents

A policy is a JSON document of **statements**. Each statement says: **Effect** (Allow or Deny), **Action** (which API calls, e.g. `logs:CreateLogGroup`), **Resource** (which things, written as an ARN), and optionally **Condition** (only from this IP, only with this tag...). This is the exact policy you added to `ecsTaskExecutionRole` during Milestone 2:

Listing 3.1 - inline policy ic-ecs-create-log-groups

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": "logs:CreateLogGroup",
      "Resource": "arn:aws:logs:us-east-1:169748358864:log-group:/ecs/*"
    }
  ]
}
```

Part	Meaning
`"Version": "2012-10-17"`	the policy language version - always this value (it is not a date you change)
`"Effect": "Allow"`	this statement grants something
`"Action": "logs:CreateLogGroup"`	service prefix `logs` (CloudWatch Logs) + API name. Wildcards exist (`logs:*`) but give more than needed
`"Resource": "arn:aws:logs:...:/ecs/*"`	ARN = Amazon Resource Name: `arn:partition:service:region:account:resource`. Here: any log group whose name starts with `/ecs/` in our account and region

- **Identity-based policies** are attached to a user, group or role ("this role may...").
- **Resource-based policies** are attached to the resource ("this bucket allows anyone to GetObject"). The S3 bucket policies of our websites are resource-based, with `"Principal": "*"` = everyone.
- **AWS managed policies** are ready-made and maintained by AWS (`AmazonECSTaskExecutionRolePolicy`). **Inline policies** belong to one role only (ours above). **Customer managed** policies are reusable ones you write.

How AWS decides - the evaluation logic

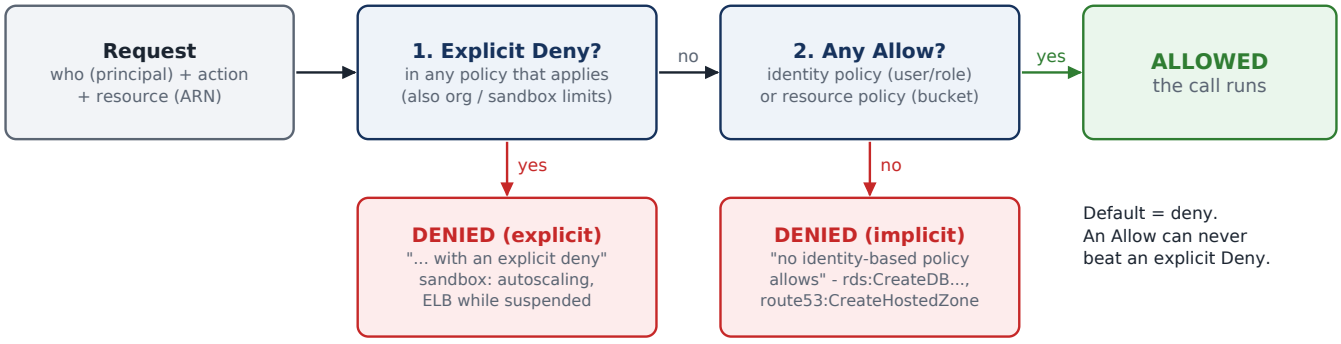


Figure 3.4 - Every request goes through these two questions.

The default answer is **no**. A request succeeds only if some policy allows it AND no policy explicitly denies it. Organisations (like the bootcamp) can add guard-rail policies above your own that deny certain actions for everyone; those show up as "explicit deny". The five refusals you met are a perfect summary of the logic:

Message you saw	Kind	Meaning	What we did
not authorized to perform `rds:CreateDBInstance` ... because no identity-based policy allows	implicit deny	the bootcamp policy allows creating RDS only with small settings; Dev/Test + 200 GiB matched no Allow	Sandbox template, db.t3.micro
not authorized to perform `route53:CreateHostedZone`	implicit deny	a DNS namespace in Cloud Map needs a Route 53 private zone; not allowed	Service Connect with an API-only namespace
`ecsTaskExecutionRole` ... not authorized to perform `logs:CreateLogGroup`	implicit deny (for a ROLE)	the managed policy may write to log groups but not create them	inline policy (Listing 3.1)
`elasticloadbalancing:DescribeTargetGroups` / `ec2:DescribeVpcs` ... with an explicit deny	explicit deny	the account was temporarily suspended (credits / time limit)	waited for reactivation
`application-autoscaling:DescribeScalableTargets` ... with an explicit deny	explicit deny	the sandbox forbids auto scaling	scaled by hand (desired count)
`access-analyzer:ValidatePolicy` not authorized	implicit deny	only the console's optional policy checker; harmless	ignored

Roles - permissions without passwords

A role has two policies. The **trust policy** says WHO may assume it (for example `ec2.amazonaws.com` or `ecs-tasks.amazonaws.com`). The **permission policies** say WHAT it may do. When an EC2 instance, an ECS task or a Lambda function starts, AWS's Security Token Service (STS) hands it **temporary keys** for the role, valid about an hour and renewed automatically. Software such as the AWS CLI, the CloudWatch agent or the Docker credential helper finds these keys by itself.

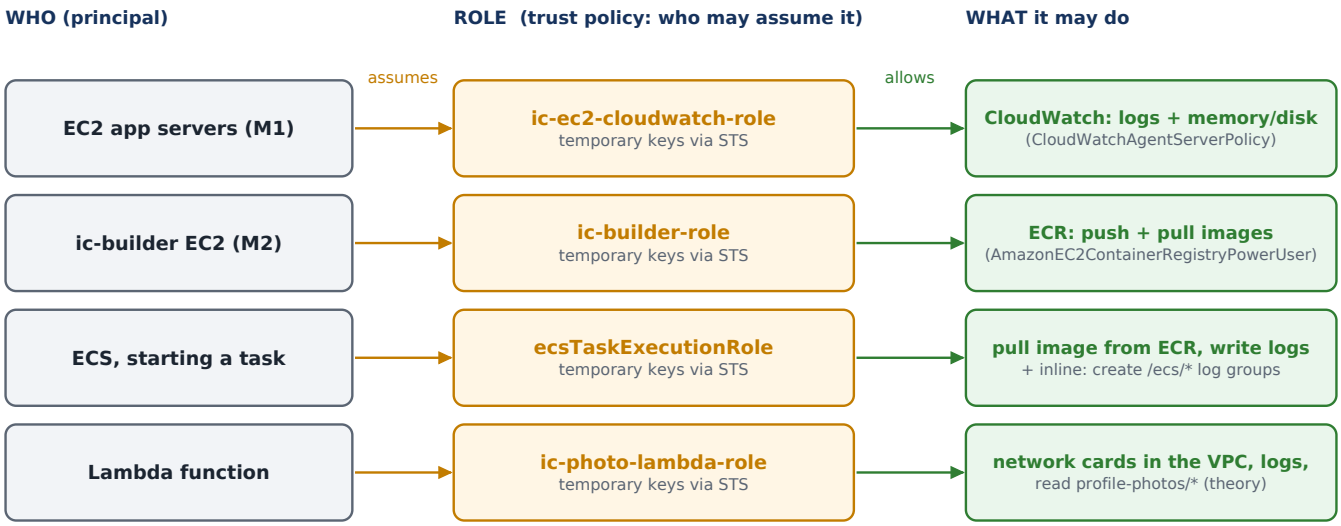


Figure 3.5 - The four roles of this project: who uses them and what they allow.

- **On EC2** the role is attached through an **instance profile**. The software asks the **instance metadata service** at `http://169.254.169.254` (a special local address inside every EC2 instance) for the keys. IMDSv2 (the default you kept) requires a session token first, which blocks a class of attacks.
- **On ECS** there are two different roles: the **task execution role** is used by ECS itself to pull the image and write logs; the **task role** would be used by YOUR code to call AWS (S3, SQS...). Our services only call each other and MySQL, so they need no task role.
- **On Lambda** the **execution role** is what the function code runs as; a separate resource-based policy lets S3 invoke the function.

- **Why this matters:** the class used `aws configure` with access keys on a VM. Keys on a disk can leak and never expire. Roles give the same power with keys that expire and rotate by themselves.

REMEMBER
Least privilege = give exactly the actions and resources needed, nothing more. Our examples: the execution role may create log groups only under `/ecs/`; each MySQL user sees only its own schema; RDS accepts connections only from our security groups.

INTERVIEW ANGLE
"Every AWS call is principal + action + resource. IAM denies by default, an explicit deny always wins, and an allow is needed for everything else. We hit both kinds in the sandbox and fixed the implicit one for ECS logs with a narrow inline policy on the task execution role."

3.4 Networking from zero

Most of the hours in this deployment were spent on networking: SSH that timed out, `mysql` that hung, a gateway that could not reach a service. All of them become obvious once you know six ideas: IP addresses, ports, subnets, routes, firewalls and DNS.

3.4.1 IP addresses and ports

An **IPv4 address** is four numbers from 0 to 255 (`10.0.5.157`). It identifies one network card. A **port** (0-65535) identifies one program on that card - think of the address as a building and the port as the office door. A connection always targets `address:port`.

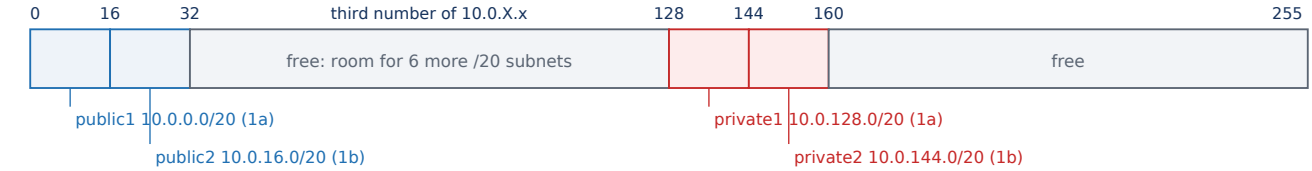
Range	Kind	Meaning	Where you met it
`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`	private (RFC 1918)	usable inside private networks, never routed on the internet; every company reuses them	ic-vpc 10.0.0.0/16, every task and server
all other ranges	public	globally unique, assigned by AWS from its pools	52.1.6.53, 13.220.9.243, task public IPs
`127.0.0.0/8`	loopback	"this same machine"; localhost = 127.0.0.1	the laptop setup; Service Connect uses 127.255.0.x
`169.254.0.0/16`	link-local	only valid on one local link; never routed; every machine can have the same one	EC2 metadata 169.254.169.254; the Fargate problem 169.254.172.2

Port	Program	Port	Program
22	SSH (remote shell, scp)	8080	api-gateway
80 / 443	HTTP / HTTPS (the ALB uses 80)	8081 / 8082 / 8083	user / project / issue service
3306	MySQL (RDS)	8761	eureka-server

3.4.2 CIDR - writing a range of addresses

`10.0.0.0/16` means "the addresses whose first 16 bits are the same as 10.0.0.0". An IPv4 address has 32 bits; 32 - 16 = 16 free bits = 2^16 = 65,536 addresses. The bigger the number after the slash, the smaller the range: /16 = 65,536, /20 = 4,096, /24 = 256, /32 = exactly one address. Subnets are cut out of the VPC range and must not overlap.

10.0.0.0/16 -> the first 16 bits (the numbers "10.0") are fixed, the last 16 bits are free -> 2^16 = 65,536 addresses: 10.0.0.0 ... 10.0.255.255
10.0.0.0/20 -> 20 bits fixed, 12 free -> 2^12 = 4,096 addresses = 16 values of the third number: 10.0.0.0 ... 10.0.15.255



Each subnet loses 5 addresses to AWS (.0 network, .1 router, .2 DNS, .3 reserved, last = broadcast): 4,091 usable.

Figure 3.6 - How the wizard cut ic-vpc into four /20 subnets.

3.4.3 VPC and subnets

A **VPC** (Virtual Private Cloud) is your own isolated network in a region: nothing gets in or out unless you build a door. A **subnet** is a slice of its addresses in **one** AZ; every EC2, RDS network card, Lambda network card and Fargate task is placed in exactly one subnet. The "VPC and more" wizard created ic-vpc with two public and two private subnets across two AZs:

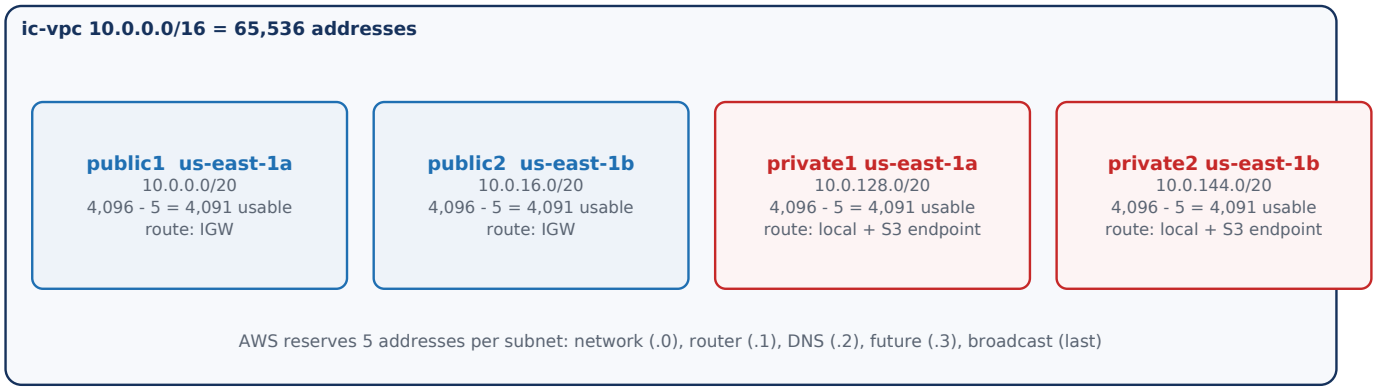


Figure 3.7 - The four subnets of ic-vpc.

Nothing in the subnet itself is "public" or "private". The words describe only **its route table**: a subnet is public if its route table sends internet traffic to an Internet Gateway.

3.4.4 Route tables and the Internet Gateway

A **route table** is a list of "destination -> target" rules that every packet leaving a subnet consults; the most specific matching rule wins. Every route table has the rule `10.0.0.0/16 -> local`, so everything inside the VPC can always reach everything else (security groups then decide who may). The public route table adds `0.0.0.0/0 -> igw-...` ("everything else goes to the Internet Gateway"); the private ones do not.

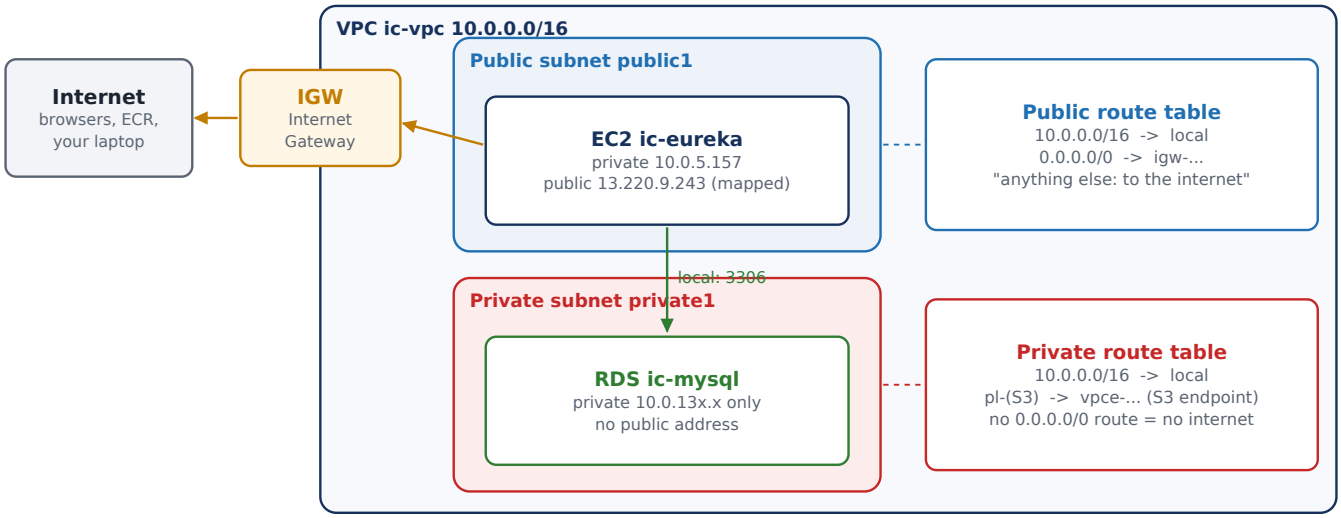


Figure 3.8 - A public and a private subnet, their route tables, and the Internet Gateway.

- The **Internet Gateway** translates one-to-one between an instance's private IP and its public IP. The instance itself never sees its public IP: on the server `ip addr` shows only 10.0.5.157 - that is why the shell prompt was `ip-10-0-5-157`.
- An instance in a public subnet still needs a **public IP** (auto-assign, or an Elastic IP) to be reached from the internet. RDS in a private subnet has neither a route nor a public IP - two independent locks.
- The private route tables also contain a route to the **S3 gateway endpoint** (`pl-... -> vpce-...`) that the wizard created - a private path to S3 for things without internet (the designed Lambda).

3.4.5 Getting out from a private subnet - three options

Fargate tasks must download their image from ECR, which lives on the internet side of AWS. A task with no route to the internet fails with `CannotPullContainerError`. There are three ways to give it a way out:

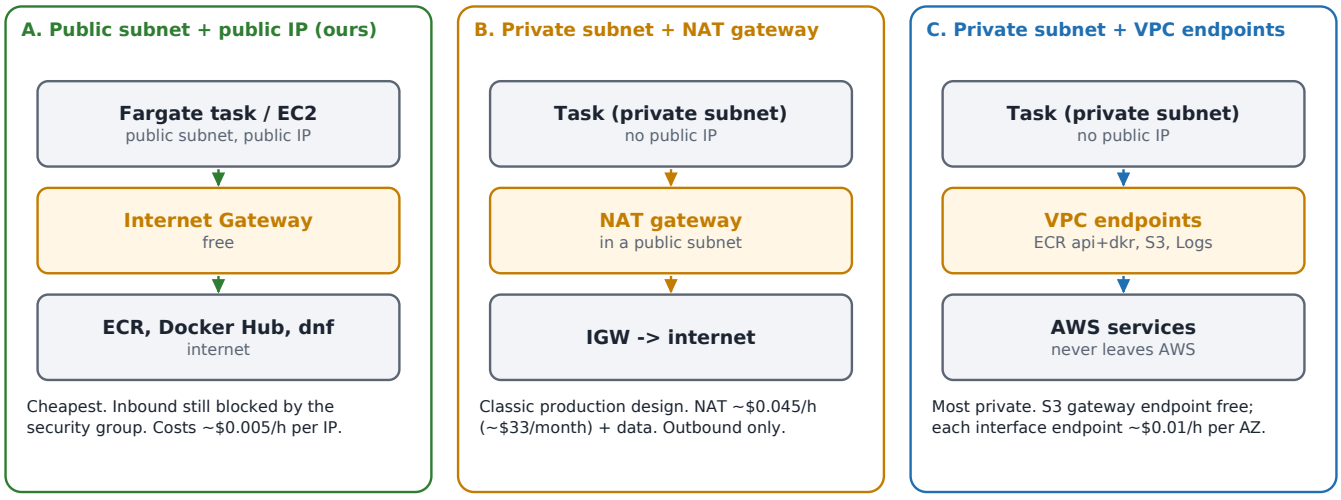


Figure 3.9 - Option A is what we built; B and C are the production patterns.

WHY OPTION A

We chose A: tasks in the public subnets with "Public IP: turned on". Nobody can connect IN because `ic-ecs-sg` only accepts traffic from the ALB and from other tasks; the public IP is only used to go OUT (ECR, Docker images). It avoids ~$33/month for a NAT gateway on a credit-limited account. In the interview, name B or C as the production design.

3.4.6 Security groups and network ACLs - the firewalls

A **security group (SG)** is a firewall attached to a network card. It has only **allow** rules (type, port, source); everything not allowed is dropped silently - which is why a missing rule never gives an error, only a **hang** or "connection timed out". SGs are **stateful**: if a request was allowed in, its reply is allowed out automatically. **Network ACLs** are a second, optional firewall on the whole subnet; they are stateless and we left them at the default "allow all".

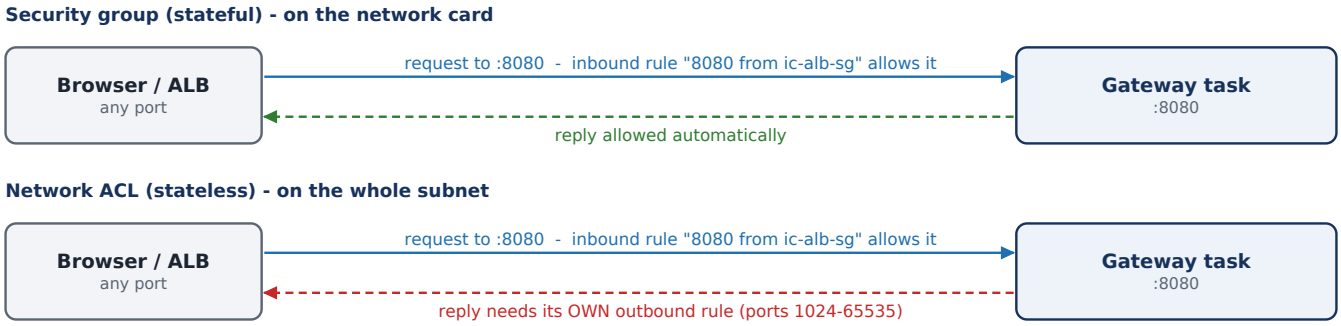
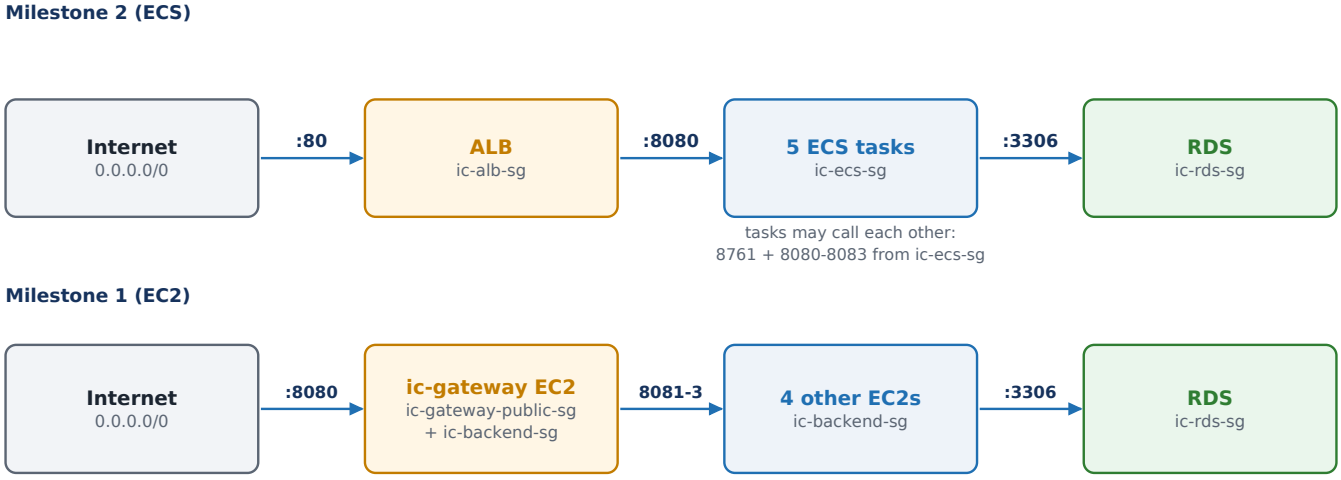


Figure 3.10 - Stateful (security group) vs stateless (network ACL).

The most powerful feature: the **source of a rule can be another security group**. "RDS accepts 3306 from members of ic-ecs-sg" means: any network card that carries ic-ecs-sg may connect - today's tasks, tomorrow's replacements with new IPs, a second copy after scaling. No IP address is written anywhere, so nothing breaks when IPs change.



SSH 22 from 0.0.0.0/0 on ic-backend-sg (key-only login). The rule source is a GROUP, not an IP: new servers or new task IPs are covered automatically.

Figure 3.11 - The security-group chains of both milestones.

Symptom	Firewall meaning
`ssh` / `Test-NetConnection` "timed out"	packets dropped: no rule for your source (or a network on your side blocks the port - the office blocks 8080)
`mysql` waits forever after the password	ic-rds-sg had no rule for the caller's group
"Connection refused" (immediately)	the packet arrived, but no program listens on that port (service stopped, wrong port)
HTTP 500 from the gateway	the network is fine; the gateway reached something that failed (our 169.254 case)

3.4.7 DNS - names instead of numbers

DNS turns a name into IP addresses, like a phone book. In the cloud you almost never type an IP for AWS services, because the IPs behind a name change: the ALB adds and replaces nodes, RDS moves to its standby during a failover. The name stays.

- **Inside a VPC** a DNS resolver answers at the VPC base address + 2 (`10.0.0.2`). It resolves public names and private ones such as the RDS endpoint `ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com`, which returns a private 10.0.x.x address inside the VPC.
- **Route 53** is AWS's DNS service. A **private hosted zone** holds names visible only inside one VPC. A Cloud Map *DNS* namespace creates such a zone - that is the `route53:CreateHostedZone` call the sandbox refused.
- **TTL** (time to live) = how long an answer may be cached (we chose 15 s when we still planned DNS discovery).
- **/etc/hosts** is a local override file read before DNS. Service Connect writes `eureka.ic.local` into every task's /etc/hosts (3.11).

3.4.8 One request from the browser to the database

Put together, this is what happens when the Milestone 2 site loads the user list. Each numbered step is also a place where a check can fail - a useful order for debugging:

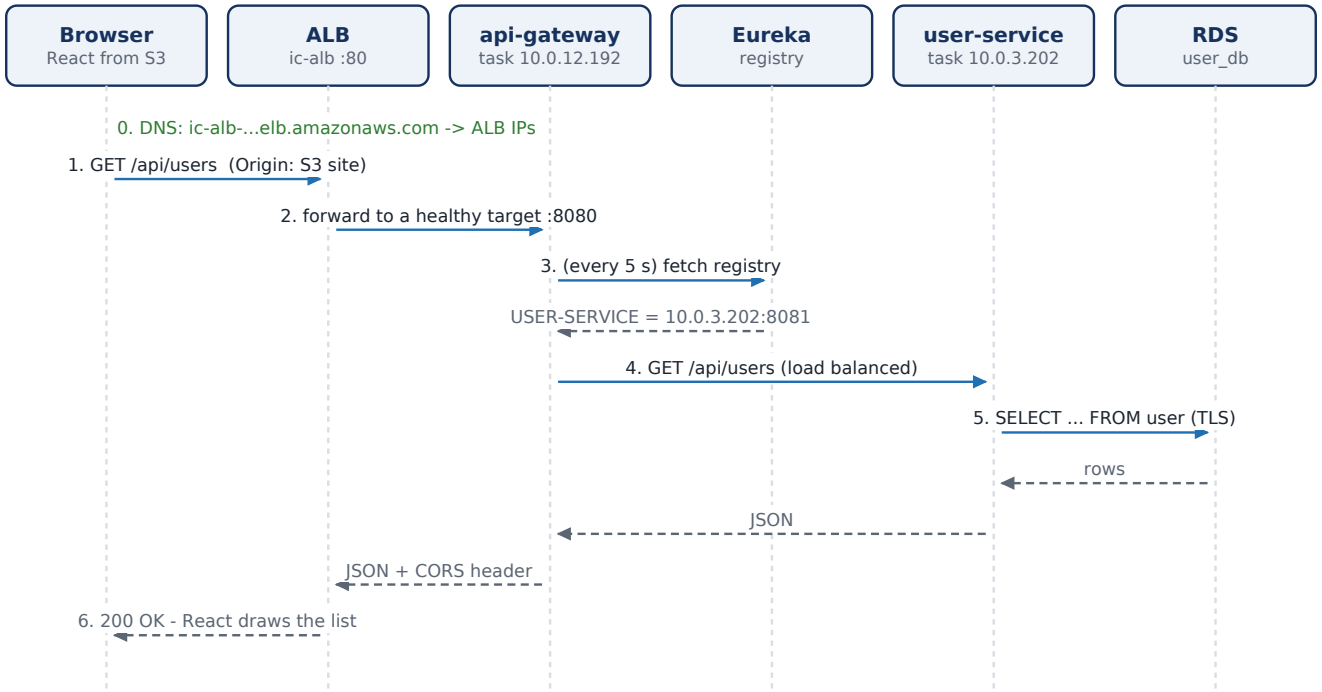


Figure 3.12 - GET /api/users through the Milestone 2 deployment.

1. **DNS** turns the ALB name into its current public IPs.
2. The browser connects to port 80; **ic-alb-sg** allows 80 from anywhere; the **listener** HTTP:80 forwards to target group **ic-gateway-tg**.
3. The ALB picks a **healthy** target (10.0.12.192:8080); **ic-ecs-sg** allows 8080 from ic-alb-sg.
4. The gateway matches the route `/api/users/**` to USER-SERVICE and looks it up in its cached copy of the **Eureka** registry (re-fetched every 5 s; our services use fast laptop timings).
5. It calls 10.0.3.202:8081 directly; **ic-ecs-sg** allows 8080-8083 from itself.
6. user-service queries **RDS** over TLS; **ic-rds-sg** allows 3306 from ic-ecs-sg. The JSON travels back the same way and the gateway adds the **CORS** header the browser needs (3.6).

3.4.9 SSH, key pairs and tunnels

A **key pair** is two linked keys. AWS put the **public** key on every instance at launch (`~/.ssh/authorized_keys`); you hold the **private** key (`ic-key.pem`). SSH proves you own the private key without ever sending it:

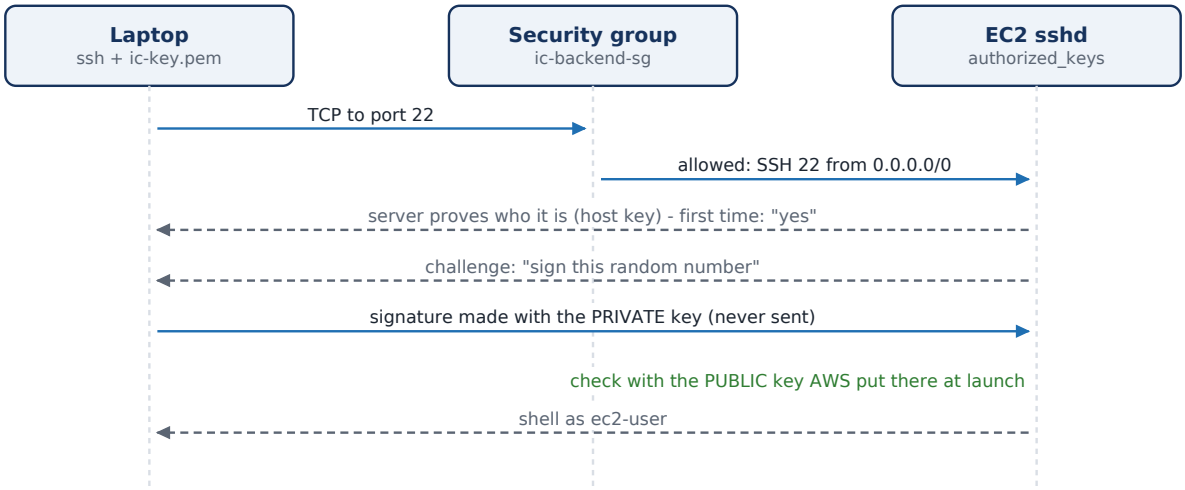


Figure 3.13 - What happens when you type ssh -i ic-key.pem ec2-user@...

- Windows OpenSSH refuses a key that other users can read - hence the two `icacls` commands.
- `scp` copies files over the same connection. The destination must contain a colon (`host:~`); without it, scp copies locally and creates a file called "ec2-user@..." - exactly what happened once.
- An **SSH tunnel** (`-L local_port:target:target_port`) lets the laptop reach a private address through a server that can see it:

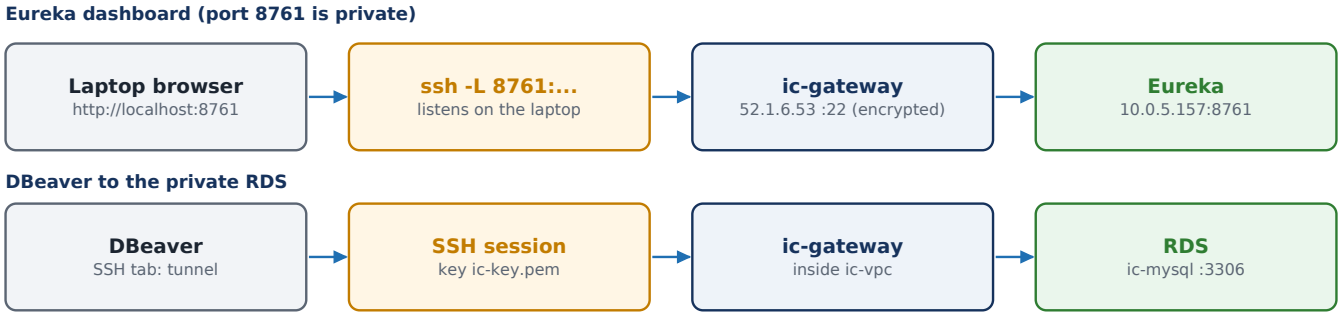


Figure 3.14 - Tunnels to private Eureka and private RDS through ic-gateway.

> **INTERVIEW ANGLE**
> "Public vs private is decided by the route table, not the subnet name. Security groups are stateful allow-lists on each network card and can reference other groups, so our database only accepts traffic from our application tier, whatever its IPs are."

3.5 EC2 - renting a computer

EC2 (Elastic Compute Cloud) gives you virtual machines. Physically, your "instance" is a slice of a big server in an AWS data centre: the **Nitro hypervisor** divides that server's CPUs, memory and network between many customers and keeps them isolated. To you it looks exactly like a Linux PC you reach over SSH. In Milestone 1 every microservice got its own instance - five small Linux computers talking to each other over the VPC.

> **THINK OF IT LIKE THIS**
> EC2 is a furnished flat you rent by the second. You choose the size (instance type), the furniture that comes pre-installed (the AMI), the front-door key (the key pair), the guest list at the door (security groups) and a note for the cleaner who prepares it on the first day (user data). You still live in it and clean it yourself (patch Linux, install Java, run the app).

The launch wizard, box by box

Wizard box	What it really decides	What we chose and why
Name and tags	a tag `Name=ic-eureka`; tags are key=value labels on any resource	`ic-eureka`, `ic-gateway`, `ic-user`, `ic-project`, `ic-issue`, `ic-builder`. The `ic-` prefix made the clean-up and the Tag Editor search easy
AMI (Amazon Machine Image)	the disk image copied onto the new disk: operating system + preinstalled tools	Amazon Linux 2023: free, uses `dnf`, AWS CLI v2 included, login user `ec2-user`
Instance type	how many vCPUs and how much memory	`t3.micro` (2 vCPU, 1 GiB) per app; `t3.medium` (2 vCPU, 4 GiB) for the Docker builder
Key pair	which public key is placed on the machine for SSH	`ic-key` (you hold `ic-key.pem`)
Network settings	VPC, subnet (= AZ), public IP yes/no, security groups	ic-vpc, a public subnet, auto-assign public IP on, `ic-backend-sg` (+ `ic-gateway-public-sg` on the gateway only)
Storage	the root EBS volume (size and type)	8 GiB gp3 (20 GiB on the builder: images are big)
Advanced - IAM instance profile	the role the software on the machine runs as	`ic-ec2-cloudwatch-role` (app servers), `ic-builder-role` (builder)
Advanced - User data	a script cloud-init runs once, as root, at the first boot	`user-data.sh` with only `SERVICE=` changed

Instance types - reading the name

`t3.micro` = family **t** (burstable, general purpose) + generation **3** + size **micro**. Other families: **m** general purpose (steady CPU), **c** compute-heavy, **r** memory-heavy, **g/p** GPU. A letter after the generation names the processor: `t4g` = AWS Graviton (ARM), `m6a` = AMD. Every size step doubles CPU and/or memory and roughly doubles the price.

- **Burstable** means the instance has a **baseline** (10% of each vCPU for t3.micro) and earns **CPU credits** while it is below it. Spring Boot's start-up spends credits for half a minute, then the app idles and earns them back - perfect for a demo workload. t3 runs in "unlimited" mode by default: if the CPU stays above the baseline for a long time you pay a small surcharge instead of being slowed down.
- **Memory is the real limit** on 1 GiB: Linux, the CloudWatch agent and the JVM must all fit. That is why the service file starts Java with `-Xms128m -Xmx384m` - the heap can never grow big enough to push the machine into swapping or the out-of-memory killer.
- **Architecture** follows the type: t3 is x86_64, t4g is ARM64. Software (and Docker images) built for one does not run on the other - important for Milestone 2 (3.8).

From an empty disk to a running service

An instance boots twice in its life in two very different ways. The **first boot** runs the user data script; **every boot** (including the first) starts the services that systemd knows about:

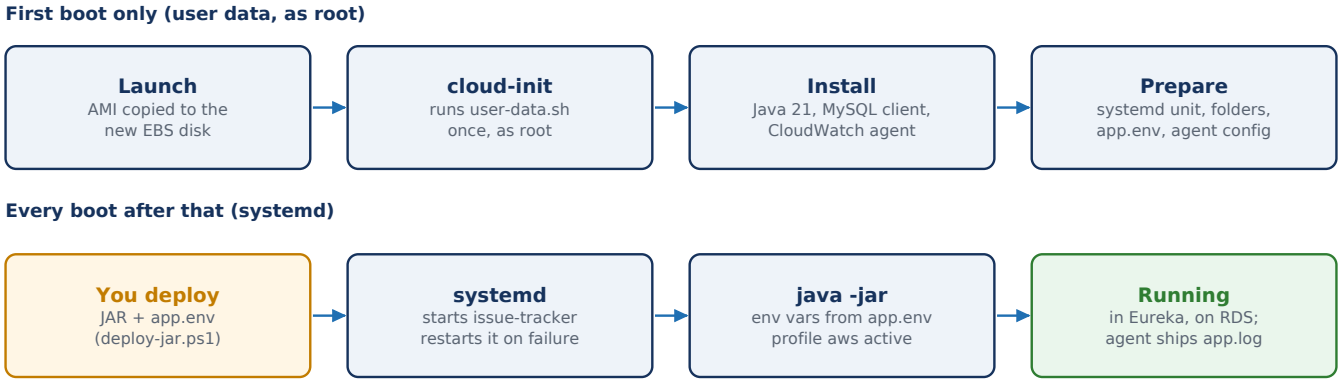


Figure 3.15 - user data prepares the server once; systemd runs the app on every boot.

- **cloud-init** is the program inside the AMI that reads the user data from the metadata service at first boot and runs it as root. Its output goes to `/var/log/cloud-init-output.log` - the first file to read if Java or the agent is missing (`sudo tail -50 /var/log/cloud-init-output.log`).
- User data runs **only once**. Editing it later (instance stopped -> Actions -> Edit user data) does not re-run it on the next start. To change a configured server you SSH in and change it, or launch a new one.
- **systemd** is Linux's service manager (process number 1). A **unit file** tells it how to run a program as a background service. `java -jar` typed in an SSH window would die the moment you closed the window; a systemd service survives log-outs, reboots and crashes.

Listing 3.2 - /etc/systemd/system/issue-tracker.service as written by user-data.sh

```
[Unit]
Description=Issue Tracker - user-service
After=network-online.target
Wants=network-online.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/app
EnvironmentFile=/etc/issue-tracker/app.env
ExecStart=/usr/bin/java -Xms128m -Xmx384m -jar /home/ec2-user/app/user-service-1.0.0.jar
SuccessExitStatus=143
Restart=on-failure
RestartSec=10
StandardOutput=append:/var/log/issue-tracker/app.log
StandardError=append:/var/log/issue-tracker/app.log

[Install]
WantedBy=multi-user.target
```

Line	What it does	Why we need it
`After=` / `Wants=network-online.target`	start only after the network is up	the app immediately contacts Eureka and RDS
`User=ec2-user`	run Java as a normal user	a hacked app cannot change the system
`EnvironmentFile=/etc/issue-tracker/app.env`	every `KEY=value` line becomes an environment variable of the Java process	this is how `SPRING_PROFILES_ACTIVE`, `EUREKA_URL`, `DB_*` reach Spring
`ExecStart=... -Xms128m -Xmx384m`	the command; heap between 128 and 384 MB	fits a 1 GiB machine
`SuccessExitStatus=143`	143 = 128 + 15 (SIGTERM): Java's normal reply to "please stop"	`systemctl stop` is not logged as a failure
`Restart=on-failure`, `RestartSec=10`	crash -> wait 10 s -> start again	self-healing, e.g. when RDS was briefly unreachable

Line	What it does	Why we need it
`StandardOutput=append:...app.log`	all console output goes to one file	the CloudWatch agent ships that file
`WantedBy=multi-user.target`	`systemctl enable` links the unit into the normal boot	the app starts by itself after a stop/start

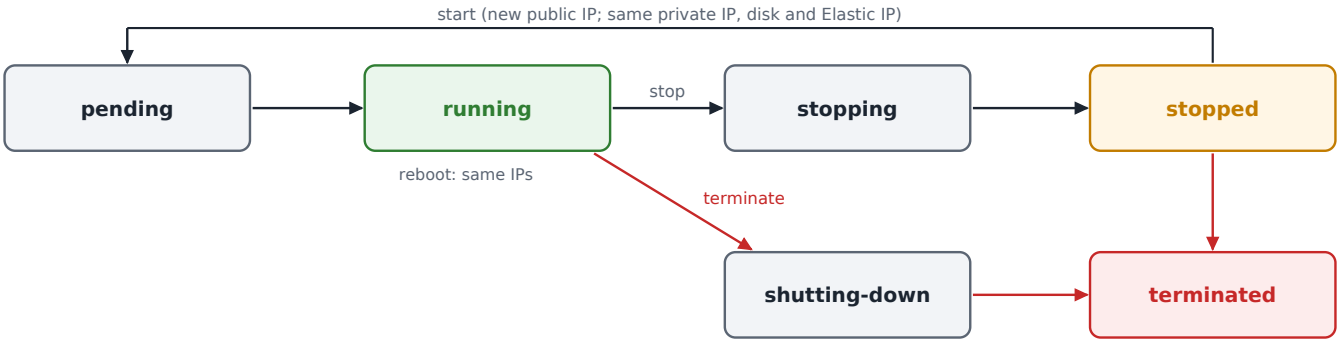
```
sudo systemctl status issue-tracker        # running? last log lines
sudo systemctl restart issue-tracker       # after a new JAR or a changed app.env
sudo systemctl stop issue-tracker          # stop (systemd will not restart it)
tail -f /var/log/issue-tracker/app.log     # follow the application log (Ctrl+C ends)
sudo journalctl -u issue-tracker -n 50     # systemd's own messages about the unit
sudo tail -50 /var/log/cloud-init-output.log   # what user data printed on first boot
```

The instance metadata service (IMDS)

Every instance can ask "who am I?" at the special address `http://169.254.169.254`. The answer includes the instance ID, region, AZ, private and public IP, the user data and - most importantly - the **temporary keys of the instance role**. The CloudWatch agent uses it for `${aws:InstanceId}` and for its credentials; the AWS CLI on the builder uses it to push to ECR without any keys. IMDSv2 first asks for a session token with a PUT request, which stops attackers who can only make the app send GET requests. Try it on any server:

```
TOKEN=$(curl -s -X PUT http://169.254.169.254/latest/api/token \
        -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/public-ipv4
```

Lifecycle - what stop, start, reboot and terminate do



running: compute billed per second | stopped: no compute bill, disk (EBS) + Elastic IP still billed | terminated: everything gone, IPs released
What survives a stop/start: instance ID, private IP, disk (JAR, app.env, logs), security groups, role, Elastic IP. What changes: the public IP + DNS.

Figure 3.16 - Instance states and what survives each transition.

Thing	Reboot	Stop -> start	Terminate
Instance ID, private IP	same	same	gone
Public IP (auto-assigned)	same	**new one**	gone
Elastic IP	same	same (stays attached)	detached, still billed until released
EBS disk: JAR, app.env, logs	kept	kept	deleted (default "delete on termination")
Memory, running processes	lost	lost	lost
Physical host	same	usually a different one	-

Because the private IP survives a stop/start, `EUREKA_URL=http://10.0.5.157:8761/eureka/` in every app.env stays valid; because the public IP does not, everything you type from the laptop (SSH, deploy-jar.ps1) needs the new public IP after a restart - except the gateway, which has an **Elastic IP** precisely because the React build of Milestone 1 contains its address.

Ways to get a shell

Method	Needs	Notes
SSH with a key (ours)	port 22 open in the SG, a public IP, the .pem file	works from any terminal; also gives scp and tunnels
EC2 Instance Connect	port 22 open to AWS's Instance Connect range	a terminal in the browser; pushes a one-minute key
Session Manager (SSM)	role with `AmazonSSMManagedInstanceCore`, the SSM agent (in the AMI)	no open port at all, no key, every session logged - the production choice
EC2 serial console	a password user on the machine	emergency access when networking is broken

Paying for EC2

Model	How it works	When
On-Demand (ours)	per second, no commitment; t3.micro about $0.0104/hour	learning, unpredictable use
Savings Plans / Reserved	1 or 3 year commitment, up to ~70% cheaper	steady production servers
Spot	spare capacity, up to ~90% cheaper, AWS can take it back with 2 minutes' notice	batch jobs, CI builders
Always extra	EBS (~$0.08/GB-month gp3), every public IPv4 address ($0.005/hour, ~$3.60/month, in use or not), data leaving AWS	even when the instance is stopped, the disk and an Elastic IP are billed

INTERVIEW ANGLE
"In Milestone 1 each microservice runs on its own t3.micro. User data installs Java and the CloudWatch agent and writes a systemd unit, so the JAR runs as a supervised service that restarts on failure and starts at boot. Configuration comes from an environment file, so the same JAR runs everywhere. The gateway has an Elastic IP because the frontend build depends on its address."

3.6 S3 - storage and the React website

S3 (Simple Storage Service) stores files - called **objects** - in containers called **buckets**. There are no folders and no disks: an object is just a **key** (its full name, such as `assets/index-D8x2.js`) + the bytes + some metadata (Content-Type, size, date). The "folders" the console shows are only the parts of keys before a `/`, called **prefixes**. S3 is regional, practically unlimited (one object up to 5 TB), and designed for 99.999999999% durability: each object is stored in several AZs.

THINK OF IT LIKE THIS
S3 is a huge cloakroom. You hand over a coat (the object), get a ticket with a unique number (the key), and later present the ticket to get exactly that coat back. You never see the racks, and there is no limit on how many coats the cloakroom takes.

Who may read a bucket

Every bucket starts **private**: only principals of your account with IAM permission can read it. A website must be readable by any browser without an AWS login, so two locks have to be opened:

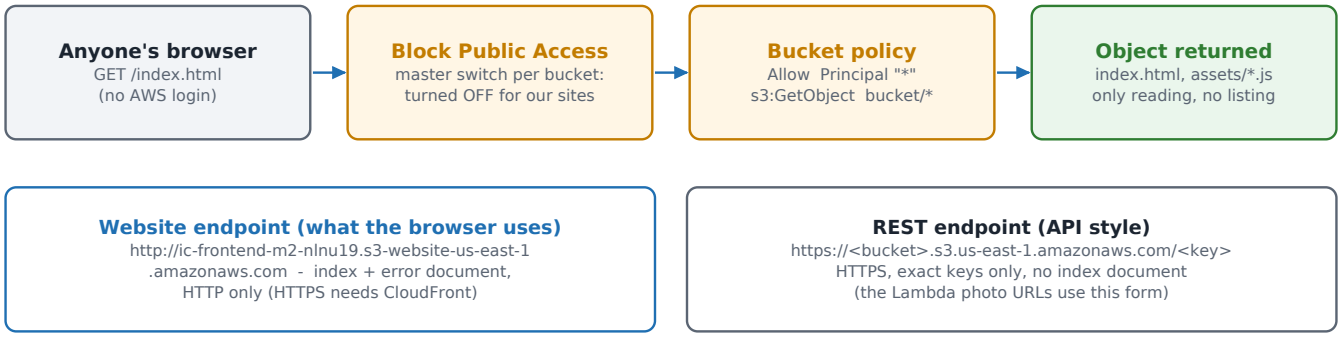


Figure 3.17 - The path of an anonymous request, and the two kinds of S3 address.

- **Block Public Access** - four switches on the bucket (and account) that override any policy making data public. It is the safety catch against accidental leaks; for a website bucket you switch it off and confirm.
- **The bucket policy** - a resource-based IAM policy (3.3) that actually grants the access:

Listing 3.3 - bucket policy of ic-frontend-m2-nlnu19 (the M1 bucket has the same with its name)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadWebsite",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::ic-frontend-m2-nlnu19/*"
  }]
}
```

`"Principal": "*"` = anyone; `s3:GetObject` = read one object by its exact key; the resource `bucket/*` = every object in it. Nobody can list the bucket, upload or delete - those would need `s3:ListBucket`, `s3:PutObject`, `s3:DeleteObject`. The `/*` matters: without it the ARN means the bucket itself, and the console rejects the policy as not matching the action.

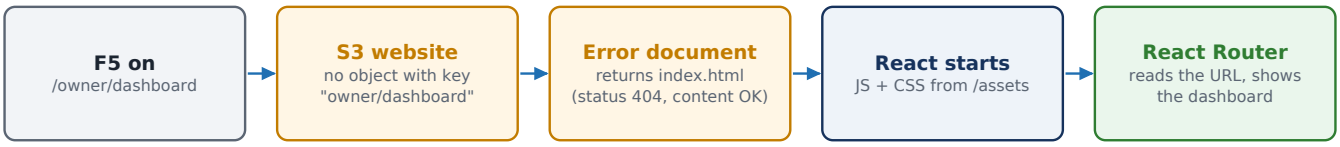
Static website hosting

A React production build (`npm run build`) is only static files: `index.html`, one JavaScript bundle, one CSS file and images in `assets/`. No server code runs - the browser executes everything. S3 can serve such files as a website once **Static website hosting** is enabled (Properties tab): you name an **index document** (`index.html`, returned for `/`) and an **error document** (returned when a key does not exist). The bucket then gets a **website endpoint** of the form `http://<bucket>.s3-website-us-east-1.amazonaws.com`.

- **Upload the contents of `dist`, not the folder**: `index.html` must be at the top of the bucket. A bucket with `dist/index.html` gives a 404 at `/`.
- The website endpoint is **HTTP only**. HTTPS needs **CloudFront** (AWS's CDN) in front of the bucket, with a certificate from ACM - the standard production setup, which also caches the files at edge locations close to users.
- Browsers cache the old JavaScript after you re-upload a build. Vite names bundles with a content hash (`index-D8x2.js`), so a new build gets new file names; a hard refresh (Ctrl+F5) reloads `index.html`.

Why a refresh on /owner/dashboard still works - SPA routing

React Router changes the address bar without asking the server for a new page. When you press F5 on such an address, however, the browser asks S3 for an object called `owner/dashboard`, which does not exist:



Without "error document = index.html" the refresh would show an S3 "404 Not Found" page.

Figure 3.18 - Setting the error document to index.html makes deep links work.

Two buckets, two builds

The API address is **compiled into** the JavaScript (`VITE_API_BASE_URL`, Part 5). The Milestone 1 build points at `http://52.1.6.53:8080/api`, the Milestone 2 build at the ALB. One bucket can host only one build, so Milestone 2 got its own bucket `ic-frontend-m2-nlnu19` and both sites stay available for the demo. Each gateway's `FRONTEND_ORIGIN` must name the bucket that talks to it - which leads directly to CORS.

Feature	What it is	Use here
Versioning	keeps every older version when an object is overwritten or deleted	undo a bad upload; on for the photos design
Storage classes	Standard; Intelligent-Tiering; Standard-IA (rarely read); Glacier (archive, minutes to hours to restore)	Standard (tiny site)
Lifecycle rules	move or delete objects after N days	e.g. delete old photo versions after 30 days
Encryption at rest	SSE-S3 is automatic for every new object since 2023; SSE-KMS uses your own key	default
Event notifications	S3 calls Lambda / SQS / SNS / EventBridge when objects are created or deleted, filtered by prefix and suffix	the designed photo Lambda (3.14)
Gateway endpoint	a route from private subnets to S3 without internet	created by the VPC wizard

CORS - why the browser blocks calls the server would answer

An **origin** is scheme + host + port: `http://ic-frontend-m2-nlnu19.s3-website-us-east-1.amazonaws.com` and `http://ic-alb-1965966534.us-east-1.elb.amazonaws.com` are two different origins. Browsers apply the **same-origin policy**: JavaScript loaded from one origin may send a request to another origin, but may only **read the answer** if that other server says it is allowed, with the header `Access-Control-Allow-Origin`. That permission system is **CORS** (Cross-Origin Resource Sharing). It protects users: a random website you visit cannot use your browser to read data from your company intranet.

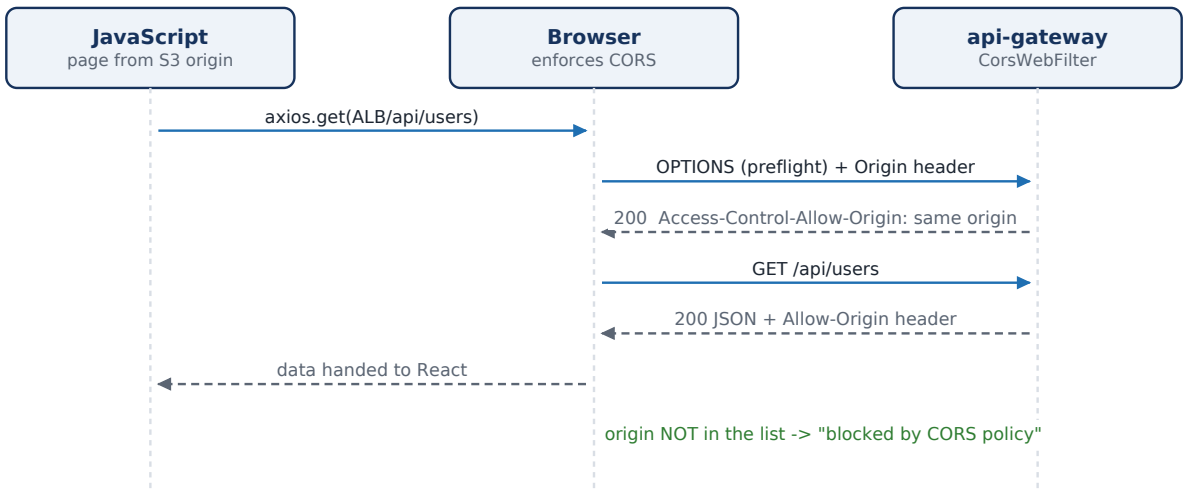


Figure 3.19 - A cross-origin call from the S3 site to the API, with its preflight.

- **Preflight:** for requests that could change data or that send JSON (`Content-Type: application/json`, PUT, DELETE), the browser first sends an `OPTIONS` request asking "may this origin send this method with these headers?". Only if the answer allows it does the real request go out. `setMaxAge(3600)` lets the browser remember the answer for an hour.
- **Who answers:** the gateway's `CorsWebFilter` (GatewayConfig). It compares the request's `Origin` header with `app.cors.allowed-origins` - on the laptop the localhost list, on AWS `${FRONTEND_ORIGIN}`.
- **CORS is enforced by the browser only.** curl, Postman, Swagger on the same origin, and typing `/api/users` into the address bar are not affected - which is why "the API works in the browser tab but the React page says Network Error" is the typical CORS symptom.
- **Exact match:** `http` vs `https`, a trailing `/`, or the M1 bucket name on the M2 gateway all fail. Copy the origin from the address bar of the open site, without the path.

REMEMBER
The error text to recognise in the browser console (F12): *has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource*. Fix: the gateway's FRONTEND_ORIGIN, then restart the gateway (EC2) or deploy a new task-definition revision (ECS).

INTERVIEW ANGLE
"The frontend is a static build on S3 website hosting, made public with a bucket policy that allows only GetObject. The error document is index.html so client-side routes survive a refresh. Because the site and the API are different origins, the gateway answers CORS with an allowed-origin list that comes from configuration. In production I would put CloudFront with HTTPS in front."

3.7 RDS - a database someone else looks after

Amazon RDS (Relational Database Service) runs MySQL (or PostgreSQL, MariaDB, Oracle, SQL Server) for you. Underneath it is an EC2-like machine with an EBS disk, but you never log into it: you get a **DNS endpoint** and a **master user**, and AWS takes care of the operating system and the engine.

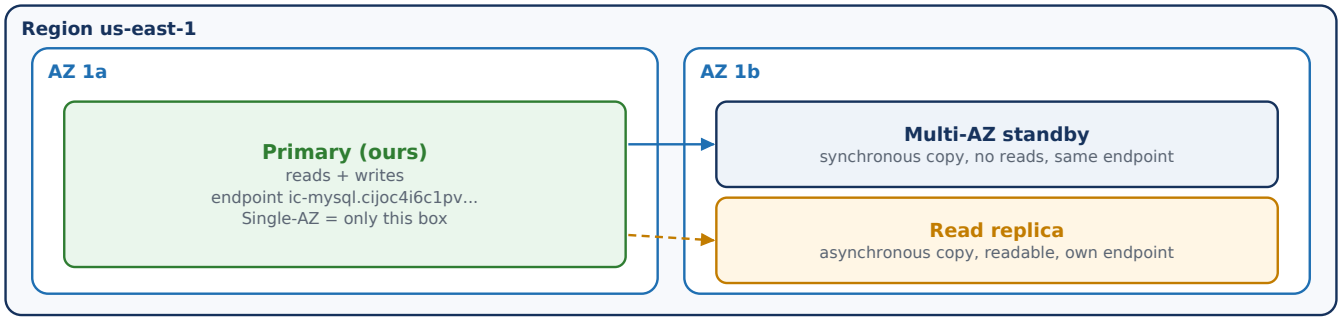
Job	Laptop MySQL	MySQL on EC2	RDS
Install / upgrade MySQL	you	you	AWS (you pick the version and a maintenance window)
OS patches	you	you	AWS
Backups + point-in-time restore	you	you (scripts)	AWS, automatic
Standby in another AZ + failover	-	very hard	one setting (Multi-AZ)
Monitoring	you	you	CloudWatch metrics built in

Job	Laptop MySQL	MySQL on EC2	RDS
Schemas, users, grants, queries	you	you	**still you**

The creation choices and what they mean

Setting	Meaning	Ours
Template	presets for the form. The bootcamp policy allows only the **Sandbox** preset	Sandbox (Dev/Test with 200 GiB was refused: implicit deny)
Engine version	MySQL major/minor	8.4.x (default)
Availability	Single-AZ, Multi-AZ instance (one standby), Multi-AZ cluster (two readable standbys)	Single-AZ
Instance class	CPU and memory; burstable like EC2 t3	db.t3.micro (2 vCPU, 1 GiB)
Storage	gp2/gp3 SSD; can auto-grow	20 GiB, autoscaling off
Master user	the admin account of the engine (not an IAM user)	`admin` - used only to run the set-up script
VPC + DB subnet group	which subnets RDS may place its network card in; must cover 2 AZs even for Single-AZ, so it can move later	ic-vpc, `ic-db-subnet-group` = private1 + private2
Public access	whether the endpoint gets a public IP	**No**
Security group	the firewall in front of 3306	`ic-rds-sg`
Initial database name	creates one schema at launch	empty - the script creates three
Backups	automatic daily snapshot + transaction logs	Sandbox default

Endpoints, availability and replicas



Multi-AZ = availability: if 1a fails, AWS points the SAME DNS name at the standby (about 1-2 minutes). Read replica = performance: spreads reads, not failover. That is why applications must use the endpoint name, never an IP address.

Figure 3.20 - Single-AZ (ours), a Multi-AZ standby and a read replica.

- The **endpoint** `ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com` resolves to a private 10.0.x.x address inside the VPC. It never changes for the life of the instance, even when the address behind it does - always configure the name (`DB_HOST`).
- **Multi-AZ** = availability (a synchronous copy takes over in 1-2 minutes; costs double). **Read replicas** = read performance (asynchronous, own endpoint, the application must send reads there). They solve different problems.
- **Aurora** is AWS's own MySQL/PostgreSQL-compatible engine: storage is shared and kept as six copies over three AZs, failover is faster, and Aurora Serverless v2 scales capacity automatically. It costs more than a db.t3.micro.

Backups and restore

Kind	How	Restore
Automated backups	a daily snapshot + transaction logs every 5 minutes, kept 1-35 days	to any second in that window (point-in-time restore)

Kind	How	Restore
Manual snapshot	you click "Take snapshot"; kept until you delete it, also after the DB is deleted	exactly that moment

A restore **always creates a new DB instance with a new endpoint** - it never overwrites the old one. The apps would need the new `DB_HOST`. Take a manual snapshot before risky changes and before the demo.

Users, privileges and TLS

- **Database per service** is a microservices rule: each service owns its data and nobody else touches it. We keep it at the logical level: one RDS instance, three schemas (`user_db`, `project_db`, `issue_db`), and one MySQL user per service that can only use its own schema - cheaper than three instances, with the same isolation of data access.
- `'user_svc'@'10.0.%'` - the part after `@` limits where the user may connect FROM: only addresses inside ic-vpc. A leaked password is useless from the internet (and the network blocks it anyway).
- Grants: `SELECT, INSERT, UPDATE, DELETE` for the application, plus `CREATE, ALTER, INDEX, REFERENCES` because Hibernate (`ddl-auto=update`) creates the tables on the first start. No `DROP`, no other schema.
- **TLS:** `sslMode=REQUIRED` in the JDBC URL makes Connector/J encrypt the connection (it does not check the certificate - `VERIFY_CA` with the RDS certificate bundle would). The MariaDB client on Amazon Linux uses `--ssl`; the Lambda design verifies the certificate with `global-bundle.pem`.
- **Both milestones share this one database.** Users, projects and issues created through the Milestone 1 site are visible in the Milestone 2 site - a nice point to show in the demo. Only `ic-rds-sg` had to be extended with `ic-ecs-sg` (and `ic-builder-sg` for the container test).

Reaching a private database

RDS has no public IP and no route from the internet, so the laptop cannot connect directly. Three ways in: run `mysql` on a server inside the VPC (we used ic-gateway); an **SSH tunnel** through that server (DBeaver's SSH tab, Figure 3.14); or AWS Systems Manager port forwarding. The typical symptom of a missing `ic-rds-sg` rule is `mysql` waiting forever after the password - the packets are dropped, nothing answers.

- **Stopping RDS:** possible for at most **7 days**; then AWS starts it again automatically (to apply maintenance). Storage and snapshots are billed while it is stopped.
- **Parameter group** = the server's settings file (`max_connections`, `require_secure_transport`...). **Encryption at rest** (KMS) must be chosen at creation. **Performance Insights** shows which queries load the database.

> **INTERVIEW ANGLE**
> "RDS is private: no public access, private subnets, and a security group that only accepts the application tier's security groups. One instance hosts three schemas with one least-privilege MySQL user each, so the services stay isolated. Connections use TLS. For production: Multi-AZ, automated backups with a longer retention, and passwords in Secrets Manager."

3.8 Containers, Docker and ECR

In Milestone 1 we prepared each server by hand: install Java, write a service file, copy the JAR, write app.env. A **container** packages the application **together with** its runtime (Java 21) and a minimal Linux into one sealed unit, the **image**. Wherever the image runs - the builder EC2, Fargate, a colleague's laptop - it behaves the same, because everything it needs is inside.

	Virtual machine (EC2)	Container (Docker / Fargate)
Contains	a whole operating system with its own kernel	the app + libraries; shares the host's kernel
Size	gigabytes	megabytes (ours: a few hundred MB each)
Start time	a minute or more	seconds (then Spring Boot's own start-up)
Isolation	very strong (hypervisor)	strong (Linux namespaces + cgroups); Fargate gives every task its own isolated kernel
Configured by	user data + SSH	the Dockerfile (build time) + environment variables (run time)

THINK OF IT LIKE THIS

A shipping container: the crane, the ship and the truck do not care what is inside; they only handle the standard box. ECS is the port that moves boxes around; the Dockerfile is the packing list; the image is the sealed box; a running container is the box opened at its destination.

Image, layers and container

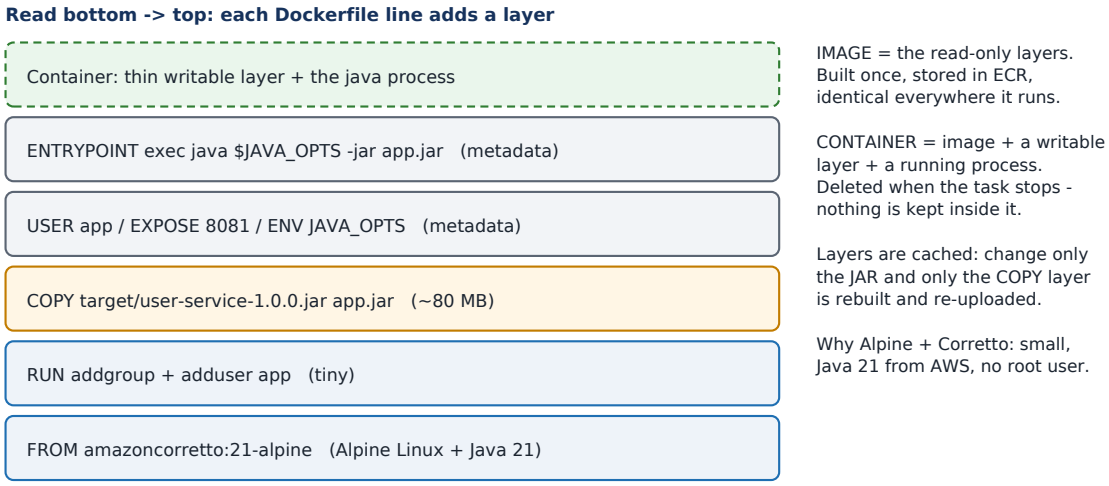


Figure 3.21 - Our image as a stack of layers.

Listing 3.4 - user-service/Dockerfile (comments removed; the other four differ only in name and port)

```
FROM amazoncorretto:21-alpine
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/user-service-1.0.0.jar app.jar
USER app
EXPOSE 8081
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

Instruction	What it does	Why this way
`FROM amazoncorretto:21-alpine`	start from an image that already has Java 21 (Amazon Corretto) on Alpine Linux	small (~200 MB with Java), maintained by AWS, the same Java version as in STS
`RUN addgroup ... adduser ...`	runs a command at build time, here: create user `app`	a container should never run as root
`WORKDIR /app`	the folder later commands work in	tidy, predictable path

Instruction	What it does	Why this way
`COPY target/...jar app.jar`	copies the JAR from the build context into the image	the fixed name `app.jar` keeps ENTRYPOINT the same for all five
`USER app`	everything after, and the container, run as `app`	least privilege
`EXPOSE 8081`	documentation of the port (opens nothing!)	the real port mapping is in the task definition
`ENV JAVA_OPTS=-XX:MaxRAMPercentage=75.0`	the JVM sizes its heap as 75% of the container memory	1 GB task -> ~768 MB heap, no hand-tuned -Xmx
`ENTRYPOINT ["sh","-c","exec java ..."]`	the command run when the container starts	`sh` expands `$JAVA_OPTS`; `exec` replaces the shell with Java

PID 1 AND THE EXEC IN ENTRYPOINT

Inside a container the first process has **PID 1**. When ECS stops a task it sends **SIGTERM** to PID 1, waits 30 seconds, then sends SIGKILL. Without `exec`, PID 1 would be the shell, which does not pass SIGTERM on: Java would be killed hard after 30 s, without closing database connections or telling Eureka "I am leaving". With `exec`, Java is PID 1, shuts down gracefully and de-registers at once.

The Docker commands we used on ic-builder

Command	Meaning
`docker build -t issue-tracker/user-service:1.0.0 ./user-service`	build an image from the Dockerfile in that folder (the **build context**) and name it
`docker images`	list local images with their sizes
`docker network create ic-net`	a private network where containers find each other by name
`docker run -d --name user-service --network ic-net -e KEY=value ... <image>`	start a container in the background with environment variables
`docker ps` / `docker ps -a`	running containers / also the stopped ones (a crashed app shows "Exited (1)")
`docker logs -f user-service`	the container's console output - this is how the issue_svc password typo showed up
`docker exec -it user-service sh`	a shell inside a running container
`docker rm -f $(docker ps -aq)`	remove all containers after the test
`docker tag` + `docker push`	give the image its registry name, upload it

- **Permission denied on /var/run/docker.sock**: Docker runs as root; `sudo usermod -aG docker ec2-user` adds you to the `docker` group, but group membership is read at log-in - exit and SSH in again.
- `/bin/bash^M: bad interpreter`: a shell script saved with Windows line endings (CRLF). The project's `.gitattributes` forces LF for `*.sh` and Dockerfiles so this cannot happen after a Git checkout.
- `exec format error` at start: the image was built for another CPU architecture (ARM on an Apple laptop, x86 on Fargate). Our builder is a t3 (x86_64) and the task definitions say X86_64 - they match.

ECR - the private image registry

Amazon ECR (Elastic Container Registry) stores images, like Docker Hub but private to your account. A **registry** is per account and region; inside it you create one **repository** per application (`issue-tracker/user-service`...). An image is addressed by repository + **tag** or by **digest**:

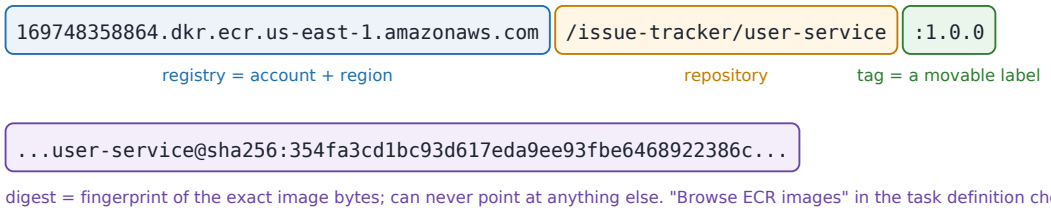


Figure 3.22 - The parts of an image URI, and the digest form ECS chose.

- **Logging in:** `aws ecr get-login-password` asks ECR (with the instance role's permissions) for a password that is valid for **12 hours**, and `docker login --password-stdin` stores it. The user name is always `AWS`. If a push says "no basic auth credentials" or "authorization token has expired", log in again.
- **Who may push and pull:** the builder's `ic-builder-role` has `AmazonEC2ContainerRegistryPowerUser` (push + pull); ECS pulls with the **task execution role** (`AmazonECSTaskExecutionRolePolicy`).
- **Tags are labels that can move** - pushing a new `1.0.0` re-points the tag; **a digest can never change**. Task definitions that store the digest (what "Browse ECR images" did) always start exactly the image that was tested. To deploy new code: push a new tag (`bash build-and-push.sh 1.0.1`) and create a new task-definition revision with it.
- **Avoid** `latest` - you cannot tell which code is running or roll back to it.
- **Scan on push** checks images for known vulnerabilities; **lifecycle policies** delete old untagged images. Storage costs about $0.10 per GB-month.

> **INTERVIEW ANGLE**
> "The JARs are packaged into images on an EC2 builder: Corretto 21 on Alpine, a non-root user, the heap sized from the container limit, and exec so Java receives SIGTERM. The builder pushes to ECR with an instance role - no access keys - and the task definitions reference the image digest, so exactly the tested image runs."

3.9 ECS and Fargate - running containers for you

Docker on one machine starts containers when you type `docker run`. In production you need much more: start the right number of copies, spread them over AZs, restart crashed ones, replace them with a new version without downtime, connect them to the load balancer and to service discovery, and collect their logs. A program that does all this is an **orchestrator**. **Amazon ECS (Elastic Container Service)** is AWS's orchestrator; **Fargate** is the option where AWS also provides the machines, so there is no server for you at all.

> **THINK OF IT LIKE THIS**
> The **task definition** is a recipe. A **task** is one dish cooked from it. A **service** is the restaurant manager who promises "there are always N dishes of this on the counter" and cooks a new one the moment one is dropped. The **cluster** is the kitchen. **Fargate** means you rent the kitchen's stoves by the minute and never see them.

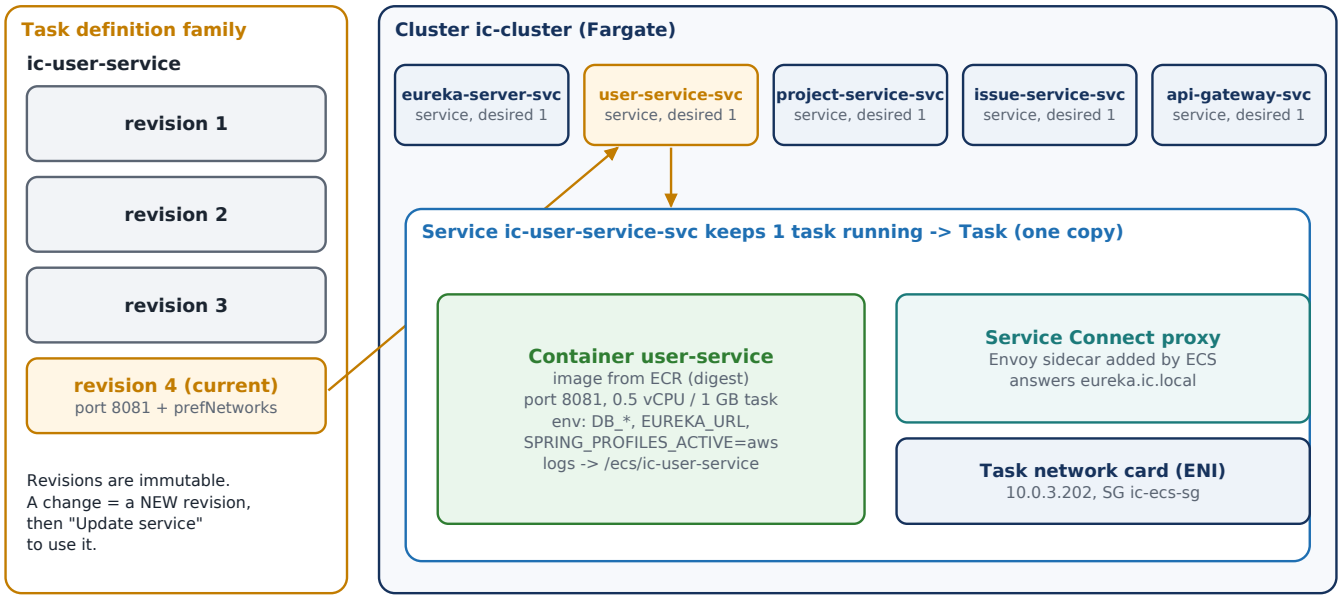


Figure 3.23 - Cluster, services, task-definition revisions and one running task.

Word	Meaning	Ours
Cluster	a named group of services and tasks (plus settings such as Container Insights)	`ic-cluster`
Task definition	a versioned JSON blueprint: image, CPU/memory, ports, environment, logs, roles	`ic-<app>` families
Revision	every edit creates a new, immutable number (`:1`, `:2`...)	user/project/issue/gateway at `:4`, Eureka at `:1`
Container definition	one container inside a task definition (a task can hold several)	one app container; ECS adds the Envoy proxy
Task	one running copy of a task definition	one per service
Service	keeps the **desired count** of tasks running; deploys new revisions; registers tasks in the target group and Service Connect	`ic-<app>-svc`, desired 1
Launch type / capacity provider	where tasks run: FARGATE, FARGATE_SPOT, or your EC2 instances	FARGATE
Deployment	one roll-out of a revision to a service	visible in the service's Deployments tab

Way to run containers	You manage	Fits
ECS on **Fargate** (ours)	only task definitions and services	small teams, no servers to patch; billed per task-second
ECS on **EC2** launch type	the EC2 instances in the cluster (AMI, scaling, patching)	large steady loads (cheaper per container), GPUs
EKS (Kubernetes)	Kubernetes objects + nodes (or EKS on Fargate)	teams already using Kubernetes; portable to other clouds
App Runner / Elastic Beanstalk	almost nothing - give it an image or code	one simple web app

The task definition, field by field

The console form produces a JSON document (task definition -> JSON tab). This simplified version of `ic-user-service:4` shows every field that mattered:

Listing 3.5 - ic-user-service:4, simplified (the password is never printed in this guide)

```
{
  "family": "ic-user-service",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "512",
  "memory": "1024",
  "runtimePlatform": { "cpuArchitecture": "X86_64", "operatingSystemFamily": "LINUX" },
  "executionRoleArn": "arn:aws:iam::169748358864:role/ecsTaskExecutionRole",
  "containerDefinitions": [{
    "name": "user-service",
    "image": "169748358864.dkr.ecr.us-east-1.amazonaws.com/issue-tracker/user-service@sha256:...",
    "essential": true,
    "portMappings": [{ "name": "user-service-8081-tcp", "containerPort": 8081,
                       "protocol": "tcp", "appProtocol": "http" }],
    "environment": [
      { "name": "SPRING_PROFILES_ACTIVE", "value": "aws" },
      { "name": "EUREKA_URL", "value": "http://eureka.ic.local:8761/eureka/" },
      { "name": "DB_HOST", "value": "ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com" },
      { "name": "DB_USERNAME", "value": "user_svc" },
      { "name": "DB_PASSWORD", "value": "(not shown here)" },
      { "name": "SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS", "value": "10.0" }
    ],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/ic-user-service",
        "awslogs-create-group": "true",
        "awslogs-region": "us-east-1",
        "awslogs-stream-prefix": "ecs"
      }
    }
  }]
}
```

Field	Meaning	Why our value
`requiresCompatibilities`, `networkMode: awsvpc`	run on Fargate; every task gets its own network card (ENI) and IP in our subnet	Fargate supports only awsvpc
`cpu: 512`, `memory: 1024`	0.5 vCPU and 1 GB for the whole task. Fargate allows fixed pairs (0.25 vCPU with 0.5-2 GB, 0.5 with 1-4 GB, 1 with 2-8 GB...)	Spring Boot starts in ~60 s on 0.5 vCPU; 1 GB fits heap + Envoy
`runtimePlatform`	CPU architecture and OS	X86_64 - the builder is a t3
`executionRoleArn`	the role ECS uses to pull the image and write logs	`ecsTaskExecutionRole` + our inline policy
`taskRoleArn` (absent)	the role the app code would use for AWS APIs	our apps call no AWS API
`image` with `@sha256:`	the exact image (3.8)	chosen with "Browse ECR images"
`essential: true`	if this container stops, the whole task stops	ECS then replaces the task
`portMappings`	the port the container listens on + a name (the Service Connect port alias)	8761 / 8080 / 8081 / 8082 / 8083
`environment`	environment variables for the container - the ECS version of app.env	see 3.12 and Part 5
`logConfiguration` (awslogs)	send stdout/stderr to CloudWatch Logs; `awslogs-create-group` makes ECS create the group	needed `logs:CreateLogGroup` (3.3)

PORTS MUST MATCH

The port in the form is easy to shift by one row. Ours once came out as user 8080, project 8081, issue 8082, gateway 8083 - each container listened on its real port from `application.properties`, but the task definitions, the ALB target group and the security group expected other ones. Rule: the container port must equal `server.port` of that app (eureka 8761, gateway 8080, user 8081, project 8082, issue 8083).

Fargate networking - and the 169.254 surprise

With `awsvpc` each task is like a tiny EC2 instance: its own ENI in a subnet, a private 10.0.x.x address, its own security group (`ic-ecs-sg`) and, because we enabled it, a public IP for pulling images. But inside the task Linux also shows a second, internal interface that Fargate uses to reach its agent and the task metadata endpoint. That interface has a **link-local** address (`169.254.172.2`) - the same in every task and unreachable from anywhere else.

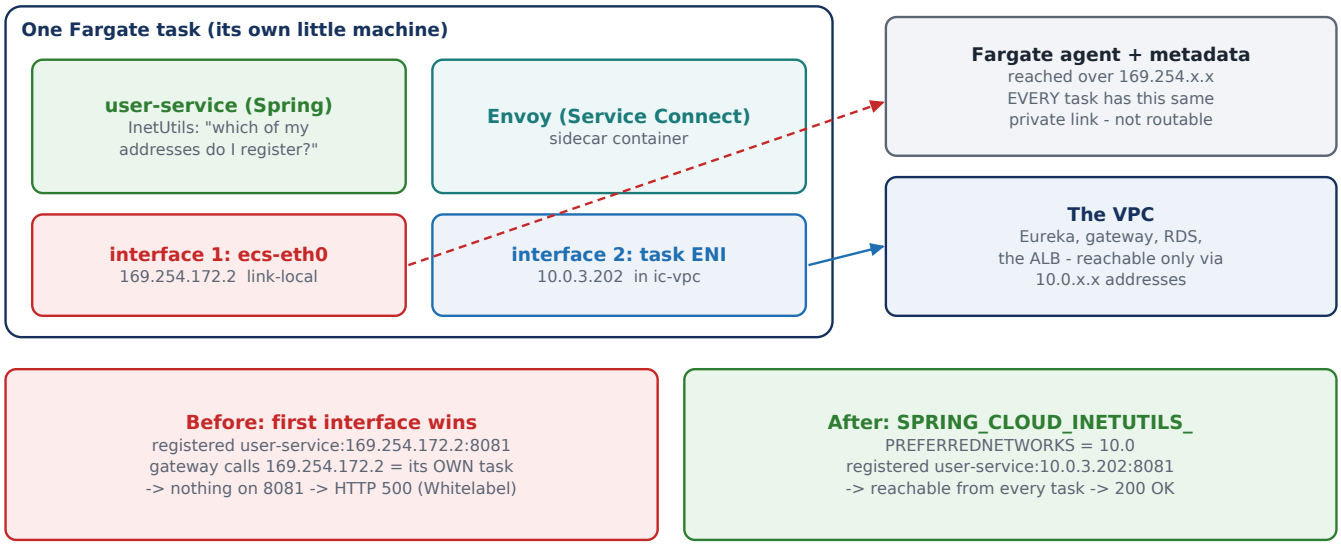


Figure 3.24 - Why every service registered 169.254.172.2, and the one-line fix.

1. At start-up Spring Cloud's **InetUtils** chooses "the" IP of the machine for `prefer-ip-address`: it takes the first non-loopback IPv4 address of the interface with the lowest index - inside Fargate that was `ecs-eth0` = 169.254.172.2.
2. Every service registered in Eureka as `169.254.172.2:<port>`. The Eureka dashboard looked fine.
3. The gateway asked Eureka for USER-SERVICE, got 169.254.172.2:8081 and connected - to **its own task**, where nothing listens on 8081. Result: HTTP 500 and the Whitelabel error page.
4. Fix, with no code change: the environment variable `SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0` (relaxed binding of `spring.cloud.inetutils.preferred-networks`, 3.12) tells InetUtils to pick only addresses starting with 10.0. New revisions -> the logs showed 10.0.x.x registrations -> 200 OK.

> **NOTE**
> Equivalent alternatives: `SPRING_CLOUD_INETUTILS_IGNOREDINTERFACES=ecs-eth0`, or `SPRING_CLOUD_INETUTILS_USEONLYSITELOCALINTERFACES=true` (10.x is a "site-local" range, 169.254.x is not). On EC2 the problem never appeared because an instance has only one interface.

The service scheduler - self-healing and rolling deployments

A service constantly compares **desired** with **running**. If a task stops for any reason (the app crashed, failed its ALB health check, you pressed Stop, the AZ had a problem), ECS starts a new one - with a **new IP**, which is exactly why Eureka, Service Connect and the ALB exist. Replacing a version uses the same mechanism:

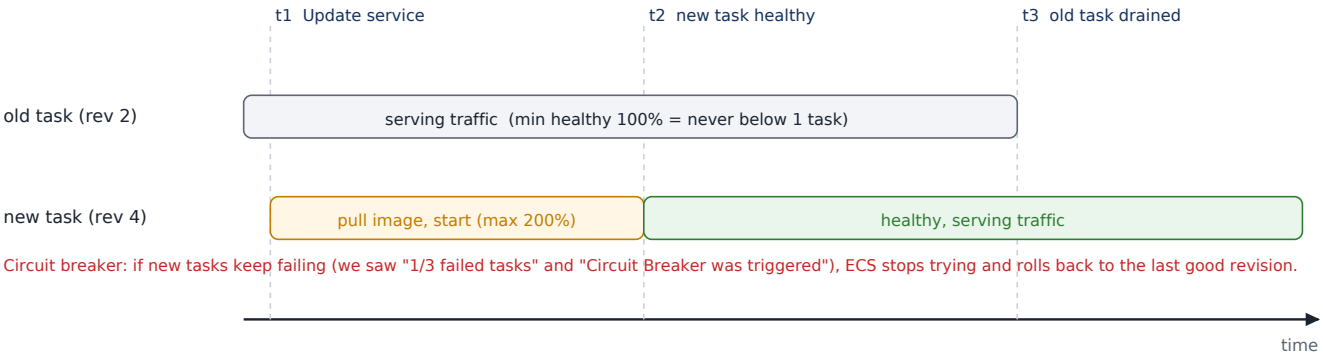


Figure 3.25 - A rolling deployment with the default 100% / 200% settings.

- **Minimum healthy 100% / maximum 200%** (defaults): with desired 1, ECS may run 2 tasks during the switch and never fewer than 1 healthy one - zero downtime when health checks are meaningful.
- **Deployment circuit breaker:** if new tasks keep failing to start (threshold: half the desired count, at least 3 failures), ECS marks the deployment failed and, with rollback enabled, returns to the last revision that worked. That is the "Circuit Breaker was triggered" and "1/3 failed tasks" you saw while the log-group permission was missing.
- **New revision != new deployment.** Creating revision 4 changes nothing that is running: a service points at one specific revision. You must **Update service** and choose the revision (or task definition -> Deploy -> Update service). **Force new deployment** restarts tasks with the SAME revision (useful after pushing a new image under the same tag).
- **The console uses CloudFormation:** creating a service from the console creates a stack named `ECS-Console-V2-Service-...`. A failed creation shows up there as `CREATE_FAILED` / rollback. Delete services from ECS, not by deleting the stack by hand.

Why a task stops - reading the "Stopped reason"

Stopped reason (task -> Details)	Meaning	Our case
`ResourceInitializationError: failed to validate logger args ... AccessDeniedException ... logs:CreateLogGroup`	the execution role may not create the log group	inline policy (Listing 3.1)
`CannotPullContainerError`	no route to ECR (no public IP / NAT), wrong image URI, or no pull permission	avoided with public IP on
`Essential container in task exited` (exit code 1)	the app crashed - read its log stream	issue-service with the mistyped DB password on the builder test
`Task failed ELB health checks`	the ALB never got 200 from /actuator/health in time	grace period 120 s
`Scaling activity initiated by deployment`	normal: an old task replaced during a deployment	-
`Task stopped by user`	you pressed Stop (or set desired 0)	the self-healing demo

INTERVIEW ANGLE

"Milestone 2 runs each microservice as an ECS service on Fargate: a task definition per app with the image digest, 0.5 vCPU and 1 GB, environment variables and awslogs. The service keeps the desired count, replaces failed tasks and rolls out new revisions with the circuit breaker. The interesting bug was Fargate's link-local interface: Spring registered 169.254.172.2 in Eureka until we set preferred-networks to 10.0."

3.10 The Application Load Balancer

Gateway tasks get a new IP every time they are replaced, but the React build needs **one fixed address**. A **load balancer** provides it: one DNS name in front of any number of targets. It sends traffic only to healthy targets and spreads it between them.

Type	Works on	Use
ALB - Application (ours)	layer 7: HTTP/HTTPS; can route by path, host, header	web apps and APIs
NLB - Network	layer 4: TCP/UDP; extremely fast, static IPs	non-HTTP protocols, fixed IPs
Gateway LB	layer 3: sends traffic through firewall appliances	network security vendors
Classic	the old generation	do not use for new work

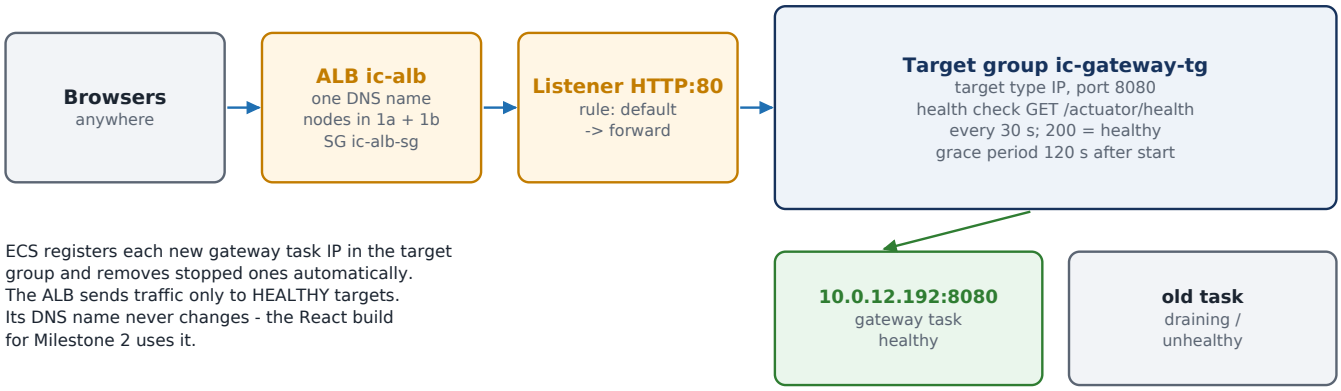


Figure 3.26 - Listener, target group and targets of ic-alb.

Part	What it is	Ours
Load balancer nodes	AWS-run proxies, one in each chosen AZ, in public subnets (internet-facing)	`ic-alb` in public1 (1a) + public2 (1b), SG `ic-alb-sg`
DNS name	resolves to the current node IPs; never use the IPs	`ic-alb-1965966534.us-east-1.elb.amazonaws.com`
Listener	a port + protocol it accepts, with rules	HTTP:80, default rule -> forward
Rules	conditions (path `/api/*`, host) -> action (forward, redirect, fixed response)	only the default rule
Target group	a set of targets + health-check settings	`ic-gateway-tg`, type **IP**, port 8080
Targets	instances, IPs or Lambdas; ECS registers and removes task IPs by itself	the gateway task

Health checks, grace period and draining

Setting	Default	Meaning
Path / port	`/` on the traffic port	ours: `/actuator/health` (Spring Actuator answers 200 with `{"status":"UP"}`)
Interval / timeout	30 s / 5 s	one check every 30 s; no answer within 5 s = failure
Healthy / unhealthy threshold	5 / 2	consecutive successes to become healthy / failures to become unhealthy
Success codes	200	anything else counts as failure
ECS health check grace period	0 s	ours **120 s**: ECS ignores ALB failures while Spring Boot starts, otherwise it would kill the task before it is ready
Deregistration delay	300 s	a target being removed keeps finishing open requests ("draining") for up to 5 minutes - why the old gateway task lingered after each update

ALB answer	Meaning	Look at
502 Bad Gateway	the target closed the connection or sent garbage	gateway log

ALB answer	Meaning	Look at
503 Service Unavailable	no healthy target registered	target group -> Targets; task status
504 Gateway Timeout	the target did not answer within the idle timeout (60 s)	slow service / RDS
500 from the app	the ALB is fine; the gateway itself failed	gateway log (our 169.254 case)

- **Two levels of load balancing:** the ALB balances browsers over gateway tasks; inside, Spring Cloud LoadBalancer balances the gateway's `lb://USER-SERVICE` calls (and Feign calls) over the instances Eureka lists, round robin. With 2 user-service tasks both receive traffic without the ALB knowing about them.
- **Why not an ALB in Milestone 1:** the task was manual deployment on EC2; an Elastic IP on the gateway was enough. In Milestone 2 an ALB is the only sensible way to give changing tasks a fixed address.
- **HTTPS** would add a listener on 443 with a free ACM certificate - it needs a domain name, which the bootcamp account does not have. Then listener 80 would only redirect to 443.
- **Cost:** about $0.0225 per hour (~$16/month) plus a small usage charge - the largest fixed cost of Milestone 2, and it is billed even when all tasks are at 0. Delete it if the project pauses for weeks.

INTERVIEW ANGLE
"The ALB gives the frontend one stable DNS name. It listens on 80, forwards to an IP-type target group that ECS keeps up to date, and health-checks /actuator/health. A 120-second grace period protects Spring Boot's start-up. Behind it, Eureka and Spring Cloud LoadBalancer spread calls between service instances."

3.11 Service discovery - Eureka, Cloud Map and Service Connect

Microservices call each other over the network, and in the cloud addresses change all the time: a restarted task, a scaled-out copy, a replacement in another AZ. **Service discovery** answers the question "where is user-service right now?" There are two families:

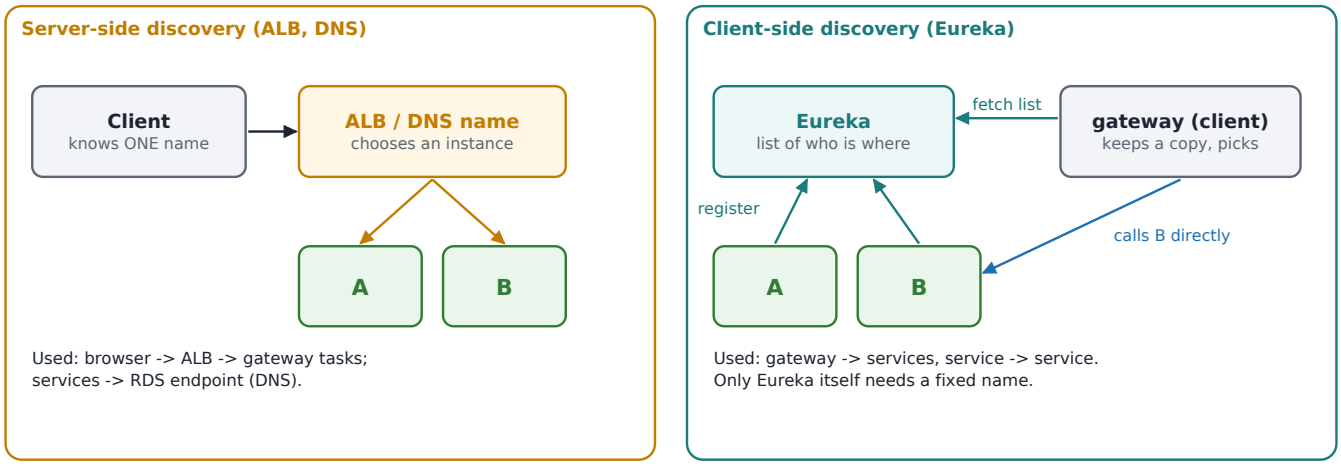


Figure 3.27 - Server-side vs client-side discovery; we use both.

Eureka - the registry inside our application

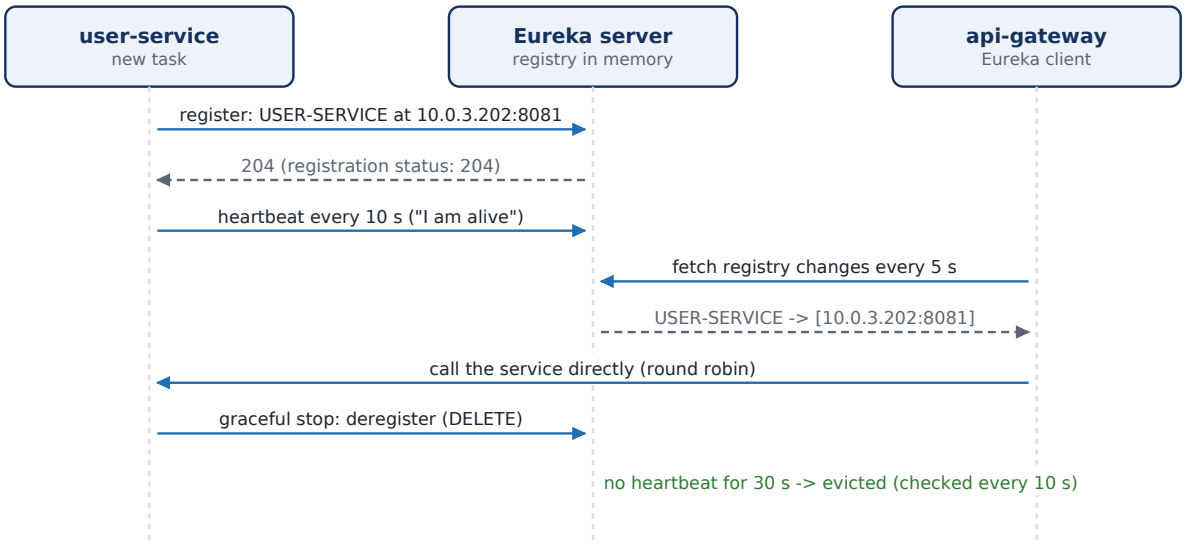


Figure 3.28 - Register, heartbeat, fetch, call, de-register, evict.

Setting (from the project)	Value	Default	Effect
`eureka.instance.lease-renewal-interval-in-seconds`	10	30	heartbeat frequency
`eureka.instance.lease-expiration-duration-in-seconds`	30	90	silence after which an instance may be removed
`eureka.client.registry-fetch-interval-seconds`	5	30	how fresh the clients' copy is
`spring.cloud.loadbalancer.cache.ttl`	5s	35s	the load balancer's own cache
`eureka.server.eviction-interval-timer-in-ms`	10000	60000	how often the server removes expired instances
`eureka.server.enable-self-preservation`	false	true	self-preservation stops evictions when many heartbeats are missing at once (protects against network splits)

These fast "laptop" timings were kept on AWS on purpose: a replaced task is noticed within seconds. Production systems keep the defaults to reduce traffic and avoid flapping.

- **What a registration contains:** application name (`USER-SERVICE`), instance id, host/IP, port, status URL. On the laptop `eureka.instance.hostname=localhost` registered "localhost" - correct on one machine, fatal on five: the gateway would call itself. The aws profile sets `prefer-ip-address=true`, so the IP is registered instead, and puts the IP in the **instance-id** so two copies of one service never overwrite each other.
- **Routing:** the gateway's routes use `lb://USER-SERVICE`. Spring Cloud LoadBalancer resolves that name from the Eureka copy and picks an instance (round robin). Feign clients between services work the same way.
- **Eureka itself needs a fixed address.** It is the only thing found WITHOUT Eureka. Milestone 1: the private IP 10.0.5.157 (it survives stop/start). Milestone 2: the name `eureka.ic.local`, provided by Service Connect.
- **Weak spot:** a single Eureka is a single point of failure for NEW lookups; clients keep their cached copy, so existing routes keep working for a while. Production runs two or three Eureka peers.

AWS Cloud Map - and the namespace saga

Cloud Map is AWS's registry. A **namespace** (like `ic.local`) contains **services** (`eureka`), which contain **instances** (IP:port). ECS registers and removes task IPs automatically. The namespace type decides how clients can look names up:

Namespace type	Lookup	Needs	For us
API calls only (HTTP)	`DiscoverInstances` API (or Service Connect)	nothing else	**works** - what we use
API calls and DNS in VPCs (private DNS)	API + normal DNS inside the VPC	a Route 53 private hosted zone	denied: `route53:CreateHostedZone`
API calls and public DNS	API + public DNS	a public domain	not applicable

The first attempt failed for a different reason: the name was typed as " ic.local" with a leading space - namespace names must be valid DNS names. The second (private DNS) was refused by the sandbox. With an API-only namespace there are no DNS records, so Spring could not simply resolve `eureka.ic.local`. **ECS Service Connect** closes that gap.

ECS Service Connect - a name without DNS

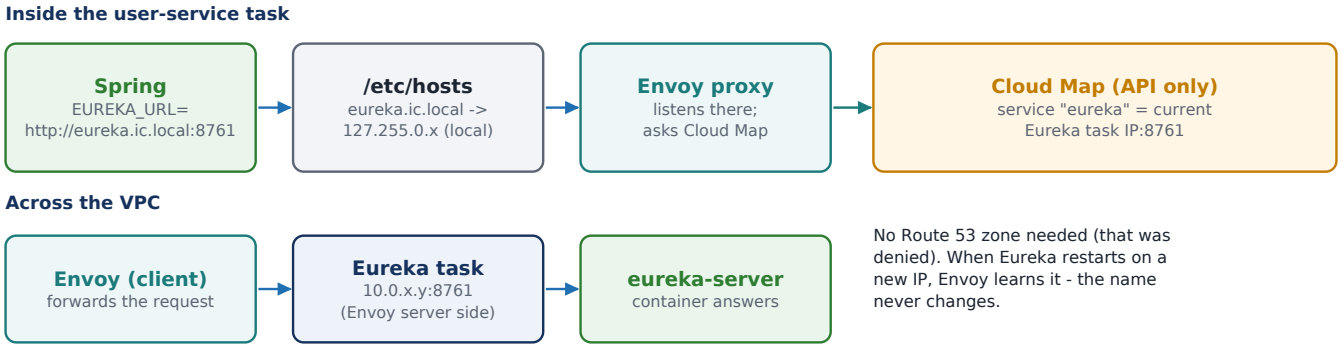


Figure 3.29 - How "eureka.ic.local" reaches the Eureka task.

Service Connect setting	Meaning	Ours
Mode "Client and server"	this service is reachable under a name AND may call others	`ic-eureka-server-svc`
Mode "Client side only"	may call names in the namespace, offers none	the four other services
Port alias	the port-mapping name in the task definition to expose	Eureka's 8761 mapping
Discovery name	the service name inside Cloud Map	`eureka`
Client alias: DNS + port	the name and port clients use	`eureka.ic.local` : `8761`
Namespace	the Cloud Map namespace of the cluster	API-only `ic.local`

- ECS adds a proxy container (**Envoy**) to every task of a Service Connect service and writes `eureka.ic.local` into the task's `/etc/hosts` with a local address from 127.255.0.0/16. Spring resolves the name from /etc/hosts (no DNS needed), connects to the local Envoy, and Envoy forwards to the current Eureka task, wherever it runs.
- The Eureka service was created first so the name existed before any client started.
- Envoy also gives retries, outlier detection and per-service traffic metrics in CloudWatch. The cost is a little CPU and memory in every task - one more reason for 1 GB tasks.
- **Why not Service Connect for everything (and drop Eureka)?** It would work, but the gateway's `lb://` routes and the Feign clients are written for Eureka. Keeping Eureka means the same code and the same discovery model as on the laptop and in Milestone 1 - a deliberate "no code change" decision.

INTERVIEW ANGLE
"Inside the app we use client-side discovery with Eureka; at the edge, server-side discovery with the ALB. On Fargate only Eureka needs a stable address, which we give it through ECS Service Connect on an API-only Cloud Map namespace because the sandbox blocks Route 53 hosted zones."

3.12 Configuration and secrets

The **twelve-factor** rule "store config in the environment" is the backbone of this deployment: the code and the JAR never change between the laptop, EC2 and Fargate; only the environment does. Spring Boot reads properties from many places and the higher place wins:

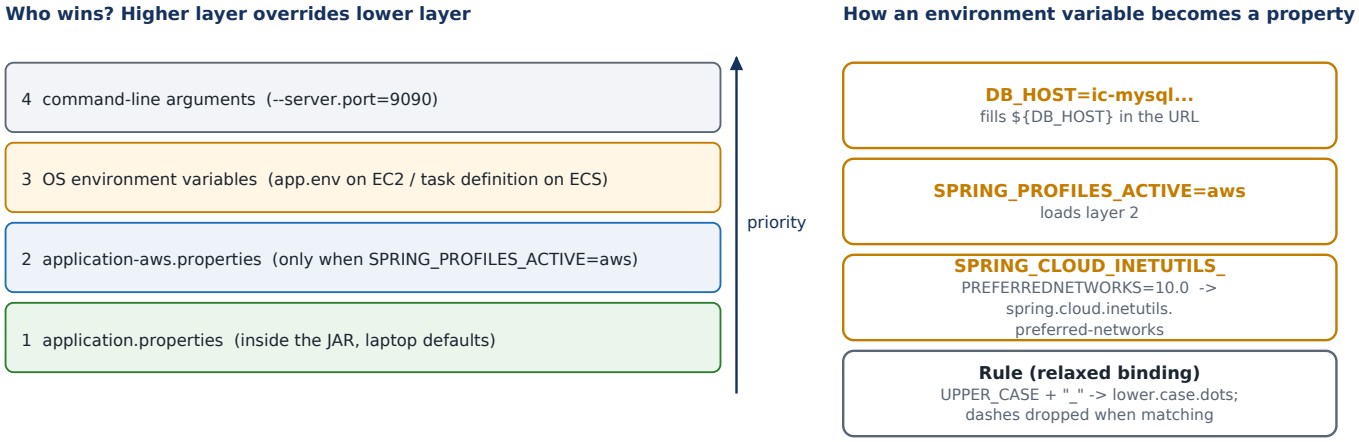


Figure 3.30 - Spring's property layers and relaxed binding of environment variables.

- **Profiles:** `SPRING_PROFILES_ACTIVE=aws` makes Spring load `application-aws.properties` on top of `application.properties`. The profile file contains only the lines that differ, written as placeholders `${EUREKA_URL}`, `${DB_HOST}`... that Spring fills from environment variables.
- **Relaxed binding:** an environment variable can also set any property directly. Upper case, `_` for `.`, dashes dropped: `SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS` sets `spring.cloud.inetutils.preferred-networks`. That is how we fixed Fargate with no new JAR.
- **Missing variable = loud failure:** an unresolved `${DB_HOST}` stops the app at start-up with "Could not resolve placeholder" - much better than silently connecting to localhost.

Variable	Milestone 1 (app.env on each EC2)	Milestone 2 (task definition)	Used for
`SPRING_PROFILES_ACTIVE`	`aws`	`aws`	load the aws profile
`EUREKA_URL`	`http://10.0.5.157:8761/eureka/`	`http://eureka.ic.local:8761/eureka/`	where to register
`DB_HOST`	RDS endpoint	RDS endpoint	JDBC URL
`DB_USERNAME` / `DB_PASSWORD`	`user_svc` / ... per service	same	MySQL login
`FRONTEND_ORIGIN` (gateway)	M1 bucket website URL	M2 bucket website URL	CORS
`SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS`	not needed	`10.0`	register the VPC IP

Where secrets should live

Plain environment variables are fine for addresses but weak for passwords: anyone allowed to read the task definition sees them, and every revision keeps a copy. AWS has two stores whose values ECS injects at start-up (task definition -> environment -> **ValueFrom**):

	Secrets Manager	SSM Parameter Store (SecureString)
Price	$0.40 per secret per month + API calls	standard parameters free

	Secrets Manager	SSM Parameter Store (SecureString)
Rotation	built in (can change the RDS password automatically)	no
RDS integration	RDS can create and manage the master password there	-
ECS usage	`secrets: [{name: DB_PASSWORD, valueFrom: <secret ARN>}]`	same with the parameter ARN
Permission	execution role: `secretsmanager:GetSecretValue` (+ `kms:Decrypt` for a custom key)	execution role: `ssm:GetParameters`

IN OUR PROJECT
We kept passwords as plain values in app.env (root-only, chmod 600) and in the task definitions, which is acceptable for a bootcamp account. Saying "in production I would move DB_PASSWORD to Secrets Manager and reference it with valueFrom" is the expected answer.

3.13 CloudWatch - seeing what happens

CloudWatch collects **metrics** (numbers over time), **logs** (text lines), and turns them into **alarms** and **dashboards**. Most AWS services send metrics automatically; logs need something to send them (the CloudWatch agent on EC2, the awslogs driver in ECS, built-in for Lambda).

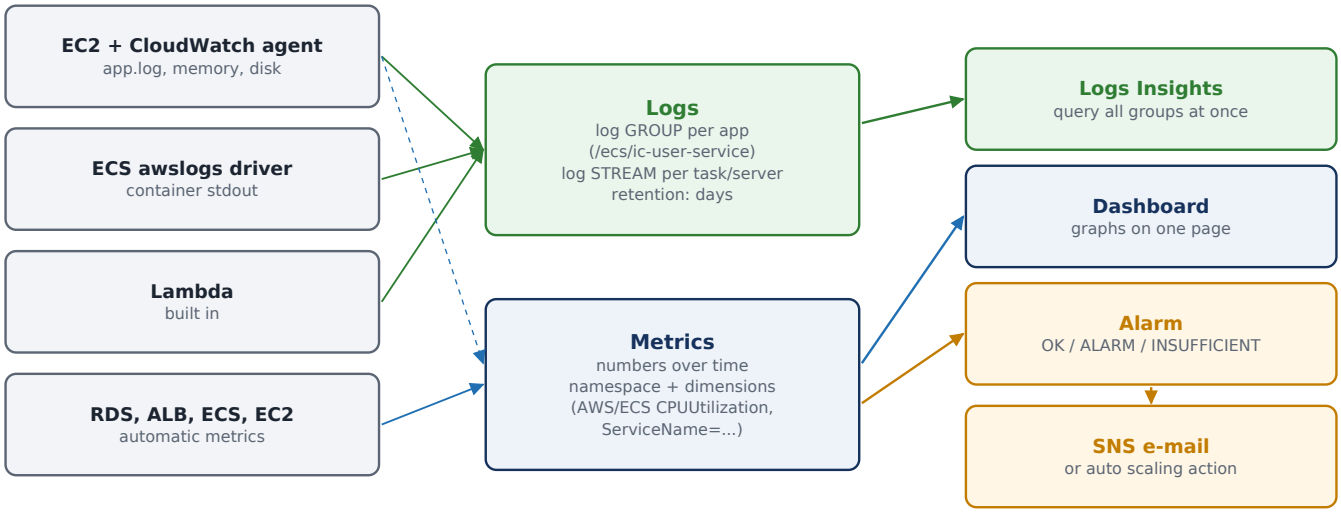


Figure 3.31 - Where data comes from and what you build on top of it.

Concept	Meaning	Example
Namespace	a folder of metrics	`AWS/EC2`, `AWS/ECS`, `AWS/RDS`, `AWS/ApplicationELB`, `IssueTracker/EC2` (ours, from the agent)
Metric + dimensions	one measured value, identified by name and key=value pairs	`CPUUtilization` with `ServiceName=ic-user-service-svc`
Period + statistic	the time bucket and how values are combined (Average, Maximum, Sum, p99)	Average over 5 minutes
Resolution	EC2 basic monitoring every 5 minutes (free), detailed every minute (paid)	basic
Log group / stream	a group per application, a stream per source (instance or task)	`/ecs/ic-user-service`, stream `ecs/user-service/<task-id>`
Retention	how long logs are kept; the default is **never expire** (cost grows forever)	set 7 or 14 days

- **EC2 does not report memory or disk usage by itself** - the hypervisor cannot see inside the operating system. The CloudWatch agent (installed and configured by user-data.sh) sends `mem_used_percent` and disk `used_percent`, and ships `/var/log/issue-tracker/app.log` to `/issue-tracker/m1/<app>`.

- **ECS** sends `CPUUtilization` and `MemoryUtilization` per service. **Container Insights** (enabled on ic-cluster, "enhanced observability") adds per-task and per-container metrics and a ready-made dashboard - it is charged extra, so it can be switched off in the cluster settings when not needed.
- **Logs Insights** queries several log groups at once - the fastest way to find which service failed:

```
fields @timestamp, @logStream, @message
| filter @message like /ERROR|Exception/
| sort @timestamp desc
| limit 50
```

Alarm part	Meaning
Metric + threshold	e.g. ECS `CPUUtilization` of ic-user-service-svc > 70
Evaluation	e.g. 3 of 3 periods of 1 minute must breach (avoids alarms on single spikes)
States	`OK`, `ALARM`, `INSUFFICIENT_DATA` (no data yet - normal for a new alarm or a stopped service)
Actions	notify an SNS topic (e-mail), trigger auto scaling, stop/reboot an EC2

IN OUR DEPLOYMENT
What exists: the `/ecs/ic-<app>` log groups (we read every Milestone 2 error there), Container Insights on ic-cluster, the automatic metrics of EC2, RDS, ECS and the ALB, and - on the Milestone 1 servers - the agent configured by user data. Dashboards, alarms and SNS e-mails were left as theory: Part 6.15 describes how they would be built.

INTERVIEW ANGLE
"Logs go to CloudWatch Logs: the agent on EC2, awslogs on ECS. Metrics come from the services plus the agent for memory and disk. I would put CPU, memory, ALB 5xx and RDS connections on a dashboard and alarm on them through SNS, with retention set so logs do not grow forever."

3.14 Lambda - code without servers (designed, not deployed)

AWS Lambda runs a function when an **event** happens - a file uploaded to S3, an HTTP request through API Gateway, a message in SQS, a schedule. There is no server, no idle cost and no capacity to plan: AWS starts as many copies as there are simultaneous events and bills per request and per millisecond. The case study asks for an S3-triggered Lambda; ours is fully designed and coded (`aws/lambda/profile-photo-processor`, Part 6.14) but was not deployed.

THINK OF IT LIKE THIS
EC2 is a full-time employee paid every hour, busy or not. Lambda is a freelancer who is called only when a job arrives, paid by the minute of work, and of whom you can call a thousand at once.

How a function runs

INIT = download the code, start Python, run the code OUTSIDE the handler (the S3 client). Warm invokes reuse it.
Two events at the same time -> a second environment (concurrency 2).

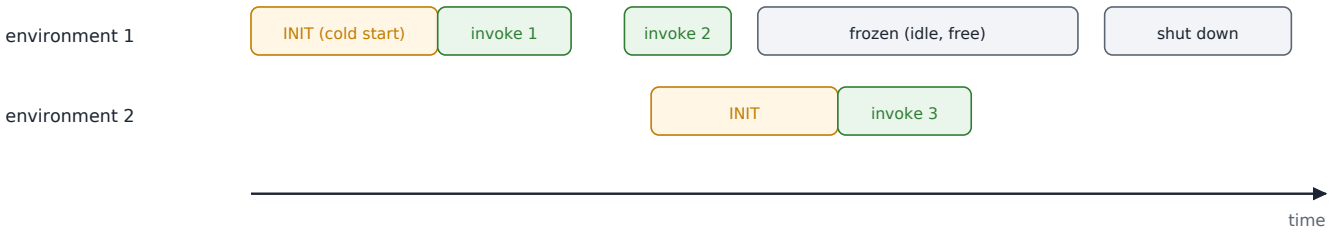


Figure 3.32 - Execution environments: cold start, warm invocations, concurrency.

- **Handler:** the function AWS calls with `(event, context)` - ours is `lambda_function.lambda_handler`. The event is JSON describing what happened (for S3: bucket, key, size).
- **Cold start:** the first call (or a call when all environments are busy) creates a new environment: download the code, start Python, run the module-level code. Later calls reuse it (**warm**), which is why the S3 client is created outside the handler.
- **Concurrency:** one environment handles one event at a time; 10 simultaneous uploads = 10 environments. The account default limit is 1,000 at once per region.
- **Stateless:** nothing in memory or `/tmp` is guaranteed to survive between calls. State goes to S3, RDS or DynamoDB.

Invocation type	Who	Behaviour on error
Synchronous	API Gateway, ALB, the console "Test" button	the caller gets the error and decides
Asynchronous (ours)	S3, SNS, EventBridge	Lambda queues the event and retries **twice**; then a failure destination / dead-letter queue
Poll-based	SQS, Kinesis, DynamoDB Streams	Lambda reads batches; failed messages return to the queue

Limit / setting	Value
Timeout	1 s to 15 minutes (ours: 15 s)
Memory	128 MB to 10,240 MB; CPU grows with memory (about 1 vCPU at 1,769 MB)
Package	50 MB zipped upload, 250 MB unzipped (with layers); container images up to 10 GB
/tmp	512 MB by default, up to 10 GB
Environment variables	4 KB in total
Price	$0.20 per million requests + about $0.0000167 per GB-second; the free tier covers 1 million requests and 400,000 GB-seconds a month

Permissions and networking

- **Execution role** (what the code may do): `AWSLambdaVPCAccessExecutionRole` (write logs, create network cards in the VPC) + an inline `s3:GetObject` on `profile-photos/*` only.
- **Resource-based policy** (who may invoke it): added automatically when the S3 trigger is created - "s3.amazonaws.com may invoke this function for bucket ic-photos-nlnu19".
- **In a VPC:** to reach the private RDS the function is attached to the private subnets with `ic-lambda-sg`. It then has no internet access; it reaches S3 through the free **S3 gateway endpoint** that the VPC wizard created. `ic-rds-sg` already allows 3306 from `ic-lambda-sg`.
- **Packaging:** libraries that are not in the runtime (PyMySQL) and the RDS certificate bundle go inside the zip - `package-lambda.sh` builds it (Part 5).

The function we designed

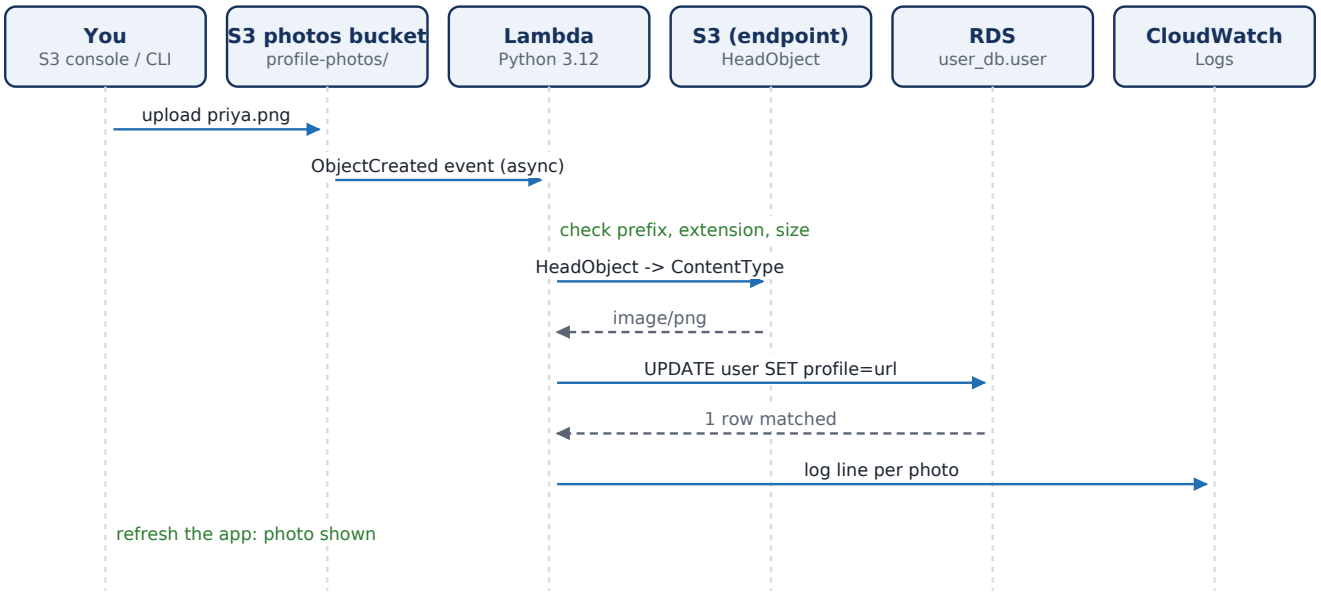


Figure 3.33 - S3 upload -> Lambda -> RDS, as designed in Part 6.14.

- **Idempotent:** S3 delivers events at least once - very rarely twice. Running our function twice sets the same URL again, so a duplicate does no harm.
- **Validation before work:** prefix, extension, size, then `HeadObject` for the real Content-Type - metadata only, no download.
- **Least privilege in MySQL too:** the `photo_lambda` user may only `SELECT (username)` and `UPDATE (profile)` on one table.
- **Never write to the prefix that triggers you** - a function that writes into `profile-photos/` would trigger itself in a loop.

> **INTERVIEW ANGLE**
> "The Lambda is an S3-triggered, asynchronous, idempotent function in the private subnets: it validates the upload, reads its metadata through the S3 gateway endpoint, and updates the user's profile URL in RDS over TLS with a column-level MySQL user. It is designed and coded; we prioritised the container milestone in the time available."

3.15 Scaling, high availability and disaster recovery

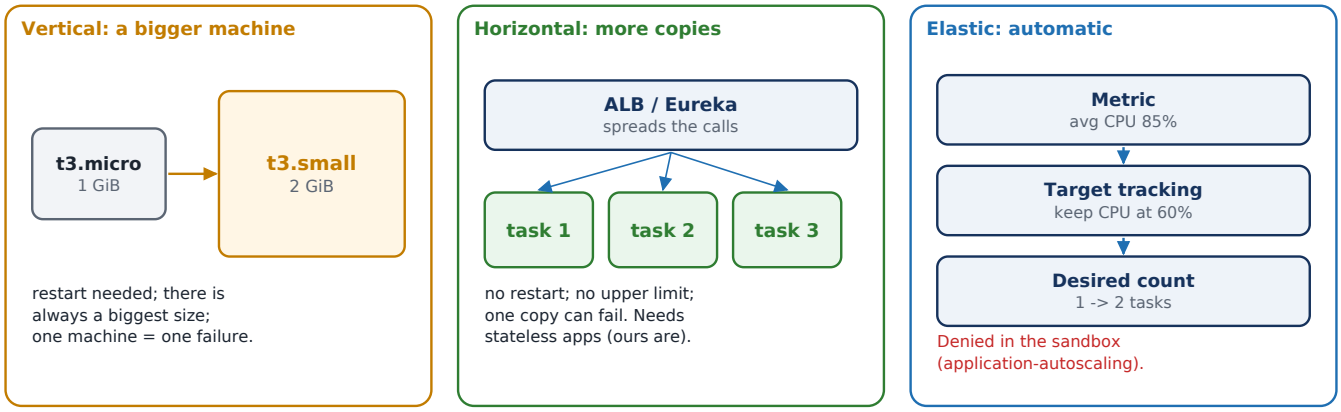


Figure 3.34 - Vertical, horizontal and automatic (elastic) scaling.

- **Our services are stateless** - no user session in memory, all data in MySQL - so any copy can answer any request. That is the precondition for horizontal scaling.

- **ECS service auto scaling** (Application Auto Scaling) can change the desired count: **target tracking** ("keep average CPU at 60%"), **step scaling** (thresholds from alarms) or **scheduled** ("2 tasks from 9 to 18"). The sandbox denies `application-autoscaling:*`, so we demonstrate scaling by setting the desired count to 2 by hand: the second task registers in Eureka and the gateway alternates between them.
- EC2 has the same idea with **Auto Scaling groups** and a launch template.

Single points of failure in our deployment

Component	Today	If it fails	Production fix
RDS	Single-AZ	all writes and reads stop until AWS recovers it	Multi-AZ
Eureka	1 task / 1 EC2	existing routes keep working from caches; new instances not found	2-3 peers
Gateway (M2)	1 task behind the ALB	ECS replaces it in ~2 minutes	desired 2, in two AZs
Gateway (M1)	1 EC2 + Elastic IP	down until restarted	Auto Scaling group + ALB
ALB	nodes in 2 AZs	already redundant	-
S3 website	regional, multi-AZ	practically never	CloudFront for speed + HTTPS

Term	Meaning
RTO - Recovery Time Objective	how long the business accepts being down
RPO - Recovery Point Objective	how much data (in time) it accepts losing; RDS point-in-time restore gives an RPO of about 5 minutes
Backup and restore	cheapest DR: restore snapshots and redeploy (hours)
Pilot light / warm standby	a minimal or scaled-down copy running in another region (minutes)
Multi-site active-active	full copies serving traffic in several regions (near zero, most expensive)

3.16 Other services worth knowing

Not used in the milestones, but they come up in interviews, and each has an obvious place in this project's next version:

Service	What it does	Where it would fit
CloudFront	CDN: caches content at edge locations; HTTPS; custom domains	in front of the S3 site (and the ALB)
Route 53	DNS: domains, records, health-checked routing	`app.example.com` -> CloudFront / ALB
ACM	free TLS certificates, renewed automatically	HTTPS on CloudFront and the ALB
WAF / Shield	web application firewall (SQL injection, rate limits) / DDoS protection	attached to the ALB or CloudFront
API Gateway	managed HTTP API front door: throttling, keys, auth	alternative to our Spring gateway
Cognito	user sign-up/sign-in, JWT tokens	real authentication for the API (Part 4.8)
SQS / SNS / EventBridge	queue / publish-subscribe / event bus	decouple services, e-mail on alarms
Step Functions	workflows of Lambdas and services with retries	multi-step photo processing
DynamoDB	serverless key-value / document database	sessions, activity feed
ElastiCache	managed Redis / Memcached	caching the board data
Systems Manager	Session Manager, Parameter Store, Run Command, patching	SSH-less admin of EC2
CloudFormation / CDK / Terraform	infrastructure as code	rebuild the whole environment from a file
CodePipeline / CodeBuild	CI/CD on AWS (or GitHub Actions)	push to Git -> build images -> deploy ECS
CloudTrail	a log of every API call in the account	who changed the security group?
AWS Config / GuardDuty	configuration history and rules / threat detection	compliance, security

Service	What it does	Where it would fit
X-Ray	distributed tracing across services	follow one request through gateway -> services
Budgets / Cost Explorer	cost alerts / cost analysis	warn before the credits run out

3.17 What it costs (approximate, us-east-1)

Prices are On-Demand list prices at the time of writing and change; the point is the order of magnitude and which items keep billing when nothing is used.

Item	Unit price	Our use	Per day
EC2 t3.micro	~$0.0104/hour	5 servers (M1)	~$1.25
EC2 t3.medium (builder)	~$0.0416/hour	only while building	~$1.00 if left on
Public IPv4 address	$0.005/hour each	5 EC2 + 5 tasks + ALB nodes	~$0.12 each
EBS gp3	~$0.08/GB-month	5 x 8 GiB + 20 GiB	~$0.16
RDS db.t3.micro	~$0.017/hour + storage	1 instance, 20 GiB	~$0.50
Fargate	~$0.0405/vCPU-hour + ~$0.0044/GB-hour	5 tasks x 0.5 vCPU / 1 GB	~$3.00
ALB	~$0.0225/hour + usage	1	~$0.55
NAT gateway (avoided)	~$0.045/hour + $0.045/GB	0	(~$1.10)
S3, ECR, CloudWatch Logs	cents per GB	small	cents

- **Stopped is not free:** stopped EC2 still pays for EBS and Elastic IPs; an ECS service at desired 0 pays nothing for tasks, but the ALB keeps billing; a stopped RDS pays for storage and restarts itself after 7 days.
- Set an **AWS Budget** with an e-mail alert - the cheapest insurance against surprises.

3.18 The Well-Architected Framework

AWS's checklist for good cloud design has six **pillars**. Use it to structure any "how would you improve it?" answer:

Pillar	What we did	Next step for production
Operational excellence	documented runbooks (Parts 6-7), scripts for every repeated step, logs in CloudWatch, versioned images	infrastructure as code, CI/CD pipeline
Security	roles instead of keys, SGs referencing SGs, private RDS, per-service DB users, TLS to MySQL, non-root containers, no secrets in Git	Secrets Manager, HTTPS, Cognito/JWT, WAF
Reliability	systemd and ECS restart failures, ALB health checks, two AZs for the ALB and tasks, RDS backups	Multi-AZ RDS, 2+ tasks per service, Eureka peers
Performance efficiency	right-sized t3.micro / 0.5 vCPU, heap tied to container memory, horizontal scaling ready	auto scaling, CloudFront caching
Cost optimisation	no NAT gateway, small sizes, stop/scale-to-zero routine, clean-up of unused resources	Savings Plans, Fargate Spot for non-critical tasks, budgets
Sustainability	managed services, nothing idle at full size	Graviton (ARM) instances and tasks

3.19 Glossary

Term	Meaning
ALB	Application Load Balancer - HTTP load balancer with one DNS name (3.10)
AMI	Amazon Machine Image - the disk image a new EC2 starts from
ARN	Amazon Resource Name - the unique id of any resource: `arn:aws:service:region:account:resource`
AZ	Availability Zone - isolated data-centre group inside a region
awslogs	the ECS log driver that sends container output to CloudWatch Logs
awsvpc	ECS network mode: each task has its own network card and IP
Bucket policy	resource-based policy on an S3 bucket
CIDR	notation for an address range, e.g. 10.0.0.0/16
Circuit breaker	ECS feature that stops and rolls back a failing deployment
cloud-init	the program that runs EC2 user data at first boot
Cloud Map	AWS service registry (namespaces, services, instances)
Cold start	creating a new Lambda execution environment before the first call
Container	a running instance of an image
CORS	browser rules for reading responses from another origin (3.6)
Desired count	the number of tasks an ECS service keeps running
Digest	sha256 fingerprint that identifies one exact image
EBS	Elastic Block Store - network disks for EC2
Elastic IP	a public IPv4 address you keep until you release it
ENI	Elastic Network Interface - a virtual network card
Envoy	the proxy that Service Connect adds to each task
Execution role	ECS: role used to pull images and write logs; Lambda: role the code runs as
Explicit / implicit deny	a Deny statement / simply no Allow - both refuse; explicit always wins
Fargate	serverless compute for ECS tasks
Health check	periodic request that decides whether a target gets traffic
IGW	Internet Gateway - the VPC's door to the internet
IMDS	instance metadata service at 169.254.169.254
Instance profile	the wrapper that attaches an IAM role to an EC2 instance
Link-local	169.254.0.0/16 - addresses valid only on one local link
Listener	the port/protocol an ALB accepts, with its rules
Namespace	Cloud Map container for service names (`ic.local`)
NAT gateway	lets private subnets reach the internet outbound only
Origin	scheme + host + port of a web page
Preflight	the OPTIONS request a browser sends before a cross-origin call
Principal	the identity that makes a request (user, role, service)
Profile (Spring)	a named set of properties loaded on demand (`aws`)
Region	geographic area with several AZs (us-east-1)
Relaxed binding	Spring maps `UPPER_CASE` env variables to `lower.case` properties
Revision	one immutable version of an ECS task definition
Role	IAM identity without password, assumed for temporary credentials
Route table	rules deciding where packets leaving a subnet go
Security group	stateful allow-list firewall on a network card

Term	Meaning
Service (ECS)	keeps N tasks of a task definition running and deploys new revisions
Service Connect	ECS feature giving services names through Cloud Map and an Envoy proxy
STS	Security Token Service - issues temporary credentials
Subnet	slice of the VPC address range in one AZ
systemd	Linux service manager; runs our JAR as a service on EC2
Target group	the set of targets an ALB forwards to, with health-check settings
Task	one running copy of a task definition
Task definition	versioned blueprint of a containerised app
Task role	role the application code inside a task uses for AWS APIs
TTL	how long a DNS answer may be cached
User data	script EC2 runs once as root at first boot
VPC	Virtual Private Cloud - your isolated network in a region

PART 4

Design decisions, security plan and environments

Every important choice with its alternatives and the reason - the answers to "why did you do it this way?".

4.1 Decisions and their reasons

Decision	Alternatives	Reason
One EC2 per application (M1)	all on one big EC2; two EC2s	matches "provision EC2 instances for each microservice", each app scales and fails independently, and Eureka/Feign are exercised over a real network
Apps in public subnets with strict SGs (M1)	private subnets + NAT + bastion	no NAT cost; only the gateway port is open to the world, everything else only from `ic-backend-sg` or your IP
RDS private, one instance, three schemas	public RDS; one RDS per service	least exposure; low cost; per-service users keep isolation
Configuration via environment variables + Spring profile `aws`	edit application.properties on each server; separate JARs per environment	one JAR for every environment (12-factor), secrets stay out of the code and Git
Elastic IP on the gateway (M1)	raw public IP; ALB already in M1	the React build contains the API address, so it must not change
Lambda = profile-photo processor	CSV import, attachment log, scheduled report	fits the case-study diagram (S3 upload -> Lambda -> RDS) and our existing photo/initials feature
Lambda in Python with PyMySQL	Java Lambda	a tiny function; quick cold start; simple packaging
S3 gateway endpoint for Lambda	NAT gateway	free, and keeps the Lambda off the internet
Column-level DB grant for Lambda	reuse user_svc	Lambda can only read `username` and change `profile` - nothing else
Docker build on a builder EC2	Docker Desktop on the laptop	exactly the class workflow; no local installs; x86 images
Keep Eureka in ECS; give Eureka the name `eureka.ic.local` with **ECS Service Connect** on an API-only Cloud Map namespace	Cloud Map DNS namespace (denied: `route53:CreateHostedZone`); replace Eureka by Service Connect everywhere	no code change; the same discovery model as the laptop and Milestone 1; works inside the sandbox restrictions
`SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0` in the task definitions	ignore the `ecs-eth0` interface; a new JAR with the property	Fargate tasks have a link-local interface Spring picked first; an environment variable fixes it without rebuilding images
Inline policy `logs:CreateLogGroup` on `/ecs/*` for `ecsTaskExecutionRole`	create the log groups by hand; broader `CloudWatchLogsFullAccess`	least privilege: exactly one action on one name prefix
A second S3 bucket for the Milestone 2 build	overwrite the Milestone 1 site	the API address is compiled into the build; two buckets keep both milestones demonstrable
Lambda and CloudWatch dashboards/alarms kept as a design	deploy them under time pressure	a working Milestone 2 had priority; the design and code are complete and explainable
ALB in front of the gateway (M2)	public IP on the gateway task	task IPs change on every deployment; ALB gives a fixed DNS name, health checks, scaling
Fargate tasks in public subnets with public IPs	private subnets + NAT or VPC endpoints	tasks must pull images from ECR; this avoids NAT/endpoint costs; inbound is still blocked by `ic-ecs-sg`
HTTP only	HTTPS with ACM certificates + CloudFront/ALB	no domain name in the bootcamp; noted as the first production improvement

4.2 How a request travels

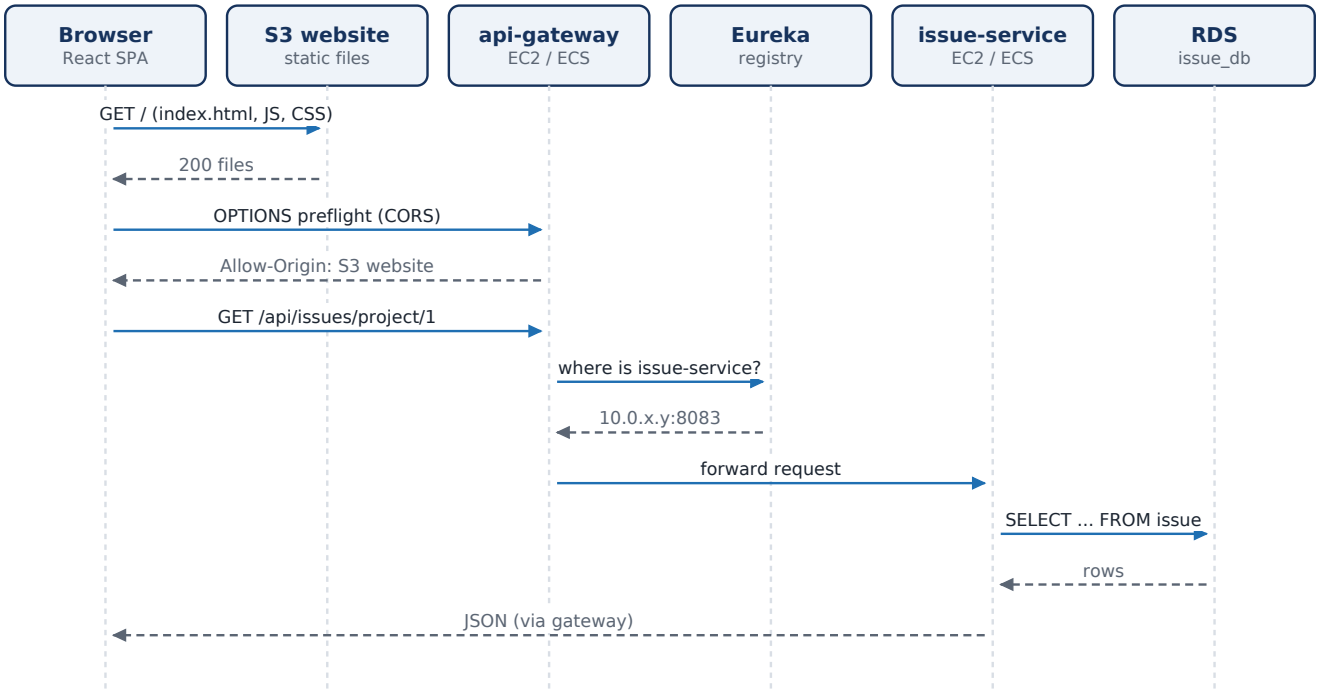


Figure 4.1 - A board load in Milestone 1. In Milestone 2 the browser calls the ALB, which forwards to a gateway task; the rest is identical.

4.3 Least-privilege IAM

Role	Trusted by	Permissions	Why
`ic-ec2-cloudwatch-role`	EC2	`CloudWatchAgentServerPolicy` (+ optional `AmazonSSMManagedInstanceCore`)	the agent sends logs and memory/disk metrics; SSM allows Session Manager
`ic-builder-role`	EC2	`AmazonEC2ContainerRegistryPowerUser`	push images to ECR without access keys
`ic-photo-lambda-role` (designed)	Lambda	`AWSLambdaVPCAccessExecutionRole` + inline `s3:GetObject` on `ic-photos-<you>/profile-photos/*`	logs, VPC network cards, read object details only
`ecsTaskExecutionRole`	ECS tasks	`AmazonECSTaskExecutionRolePolicy` + inline `ic-ecs-create-log-groups` (`logs:CreateLogGroup` on `/ecs/*`)	pull images, create and write the `/ecs/ic-<app>` log groups

4.4 Security-group matrix

Security group	Inbound rules (type, port, source)	Attached to
`ic-backend-sg`	SSH 22 from **0.0.0.0/0** (key-only login; "My IP" fails behind the company proxy); TCP 8761, 8080-8083 from **ic-backend-sg** (itself)	all five M1 EC2s
`ic-gateway-public-sg`	TCP 8080 from 0.0.0.0/0	`ic-gateway` only
`ic-rds-sg`	MySQL 3306 from **ic-backend-sg**, **ic-lambda-sg**, **ic-builder-sg**, **ic-ecs-sg**	RDS
`ic-lambda-sg`	none (outbound only)	the Lambda (designed)
`ic-builder-sg`	SSH 22 from 0.0.0.0/0	`ic-builder`
`ic-alb-sg`	HTTP 80 from 0.0.0.0/0	the ALB
`ic-ecs-sg`	TCP 8080 from **ic-alb-sg**; TCP 8761, 8080-8083 from **ic-ecs-sg** (itself)	all ECS tasks

> **NOTE**
> Outbound rules stay at the default (all traffic allowed): the apps must reach Eureka, each other, RDS, and (on EC2) the package repositories. Because security groups are stateful, answers to allowed inbound requests are always let back out.

4.5 Secrets

- Passwords live only in: your password manager; `/etc/issue-tracker/app.env` on each EC2 (owner root, mode 600); Lambda and ECS environment variables. **Never** in Git, the React build, screenshots or chat.
- Better (optional): store each password in **Secrets Manager** or **SSM Parameter Store** (SecureString) and reference it from the ECS task definition ("ValueFrom") - the password then never appears in the console. The execution role needs permission to read that one secret.
- Your local `db.properties` is packaged inside the JARs you build (it is in `src/main/resources`). It only contains your laptop's MySQL password, but do not share the JARs publicly. The `aws` profile overrides it on AWS.

4.6 Environments: dev, test, prod (case-study guideline)

Environment	Where	How it is selected
dev	your laptop: local MySQL, Eureka on localhost, Vite on 5173	default Spring profile; `.env.development` / defaults
test	same AWS design with `ic-test-*` names, its own schemas or RDS instance	profile `aws` + test variables
prod	the deployment in this guide	profile `aws` + prod variables; React built with `build-frontend.ps1 -ApiUrl <prod API>`

The code is identical in every environment; only configuration differs. That is the point of putting addresses and passwords in environment variables. Name every AWS resource with its environment so nothing is mixed up (for a second environment, `ic-test-mysql` next to `ic-mysql`).

4.7 Backups

- RDS automated backups with 7-day retention (point-in-time restore) + a manual snapshot before demos and before deleting the instance.
- S3 versioning on both buckets (overwritten photos or a bad frontend upload can be restored).
- Code and configuration templates in Git; images in ECR with version tags (roll back by deploying an older task-definition revision).

4.8 Authentication (non-functional requirement)

The application checks email and password at login, but the API does not yet verify who sends each request (there is no token). The deployment keeps internal services private, so only the gateway is reachable. The next step would be JWT: the user-service issues a signed token at login, the React app sends `Authorization: Bearer <token>`, and the gateway validates it before routing. On AWS, **Amazon Cognito** could provide sign-up/sign-in and tokens, and an ALB can authenticate users with Cognito before forwarding. Name this as a known limitation and the planned improvement.

PART 5

Every change that makes the laptop app work in the cloud

On the laptop everything was "localhost": five Spring Boot apps, MySQL, Eureka and the React dev server. This part is the one place that lists EVERY edit made for AWS - in the code, in the repository and in the cloud settings - with the problem each one solves and what would break without it, followed by every shortcut script explained line by line.

5.0 The principle - one codebase, three environments

The business logic did not change at all: the same controllers, services, entities and React pages run on the laptop, on EC2 and on Fargate. What changes between environments is only **where things are** (Eureka, MySQL, the API, the website) and **how the program is started**. Those are exactly the things that were turned into configuration:

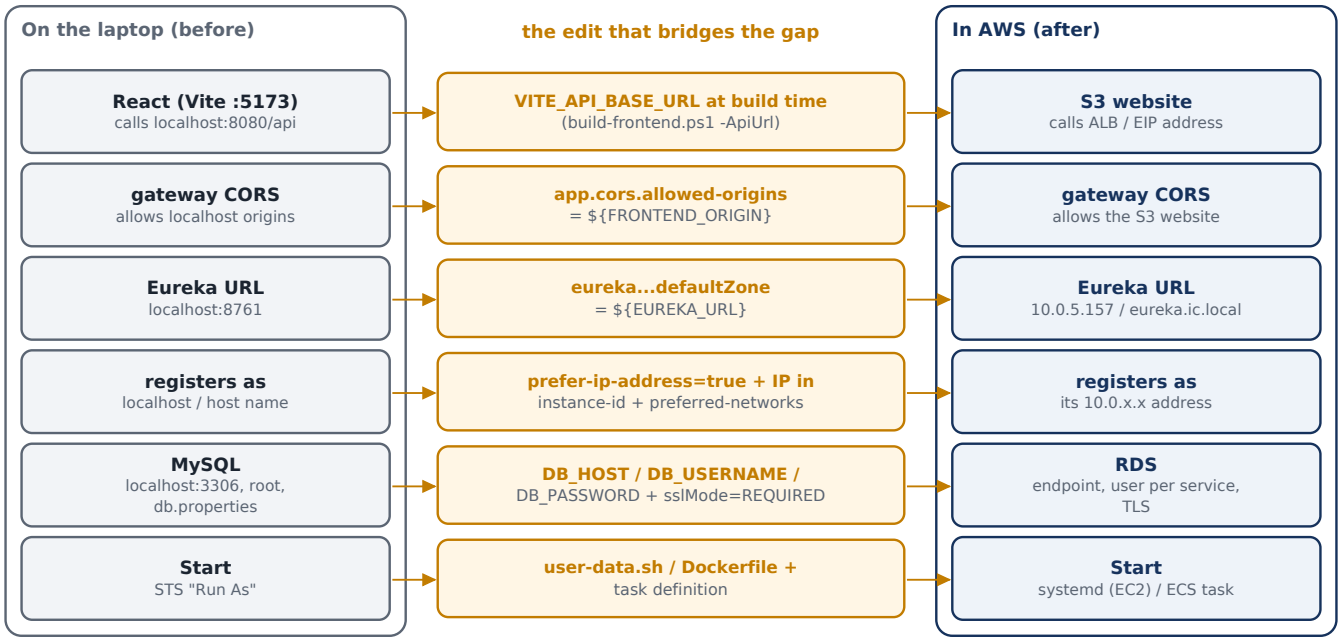


Figure 5.1 - Each laptop assumption, the edit that bridges it, and the cloud reality.

What would break if the laptop JARs were copied to AWS unchanged

Laptop assumption	What happens on AWS	Fixed by
Eureka is at `localhost:8761`	each app looks for Eureka on its own server / in its own task - nobody registers	edit 1, 2 + C1/C2
Each app registers as "localhost"	the gateway looks up user-service, gets "localhost:8081" and calls ITSELF -> errors	edit 1, 2 (prefer-ip-address)
Instance id = `name:port`	two copies of user-service get the same id and overwrite each other in Eureka	edit 1, 2 (IP in the instance-id)
MySQL at `localhost`, user root, password in db.properties	no MySQL on the server: "Communications link failure"	edit 1 + C1/C2
`useSSL=false`	database traffic unencrypted	edit 1 (sslMode=REQUIRED)
`createDatabaseIfNotExist=true`	the per-service MySQL users may not create schemas	edit 1 + the SQL script (edit 8)
db.properties must exist	a JAR built without it refuses to start	edit 5 (optional:)
CORS allows only localhost origins	the browser blocks every API call from the S3 site	edit 3, 4 + C5

Laptop assumption	What happens on AWS	Fixed by
React calls `http://localhost:8080/api`	"localhost" in a visitor's browser is the VISITOR's computer - nothing answers	C4 (build-frontend.ps1)
Apps are started with STS "Run As"	nothing starts the JAR on a server, and it would die with the SSH session	edit 7 (systemd) / edit 6 (Docker + ECS)
A computer has one network interface	a Fargate task has two; Spring picked the link-local 169.254 one	C3 (preferred networks)

5.1 All edits at a glance

Code and repository edits (all in `issue-tracking-system`, committed with the project):

#	Where	Kind	What changed	Why
1	`user-service`, `project-service`, `issue-service`: `src/main/resources/application-aws.properties`	new file	Eureka URL from `${EUREKA_URL}`; register by IP; IP in instance-id; JDBC URL from `${DB_HOST}` with TLS; user/password from variables	one JAR for laptop and cloud
2	`api-gateway/src/main/resources/application-aws.properties`	new file	Eureka lines as in 1; `app.cors.allowed-origins=${FRONTEND_ORIGIN}`	gateway finds Eureka and allows the S3 site
3	`api-gateway/src/main/resources/application.properties`	+1 property	`app.cors.allowed-origins=<localhost list>`	the CORS list becomes config
4	`api-gateway/.../config/GatewayConfig.java`	changed bean	`corsWebFilter` receives the origins with `@Value` instead of a fixed list	the only Java change
5	the three services' `application.properties`	1 word	`spring.config.import=optional:classpath:db.properties`	start without db.properties
6	`<app>/Dockerfile` (5 files)	new files	image recipe: Corretto 21 Alpine, non-root user, JAR, exec java	Milestone 2 images
7	`aws/ec2/user-data.sh`, `aws/ec2/app.env.example`	new files	first-boot script for each EC2; template of its settings file	repeatable servers (M1)
8	`aws/rds/01-schemas-and-users.sql`, `02-lambda-user.sql`	new files	three schemas, one least-privilege user per service; Lambda user	database per service
9	`aws/lambda/profile-photo-processor/`	new files	function, requirements, `package-lambda.sh`	designed S3 -> Lambda -> RDS
10	`aws/scripts/`	new files	`deploy-jar.ps1`, `build-frontend.ps1`, `copy-to-builder.ps1`, `build-and-push.sh`	one command instead of many (5.10)
11	`.gitignore`, `.gitattributes`	+6 lines / new	keys, app.env, zips never committed; LF line endings for Linux files	security, portability
12	`aws/README.md`	new file	what each deployment file is for	case-study documentation

Cloud-side settings (typed on the servers or in the console - never in Git, because they hold addresses of one deployment and passwords):

#	Where	What	Why
C1	`/etc/issue-tracker/app.env` on each EC2 (M1)	`SPRING_PROFILES_ACTIVE`, `EUREKA_URL`, `DB_*`, `FRONTEND_ORIGIN`	feeds the placeholders of edits 1-2
C2	environment variables of each task definition (M2)	the same variables, ECS values	same, for containers
C3	task definitions of the 4 Eureka clients (M2)	`SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0`	register the VPC IP, not 169.254.172.2
C4	the React build (both milestones)	`VITE_API_BASE_URL` = gateway EIP:8080/api (M1) or ALB/api (M2), set by `build-frontend.ps1`	the browser must call AWS, not localhost
C5	the gateway's `FRONTEND_ORIGIN`	M1: `ic-frontend-nlnu19` site; M2: `ic-frontend-m2-nlnu19` site	CORS must name the site that calls it

SAME CODE EVERYWHERE
What did **not** change: no Java in user-, project- or issue-service; nothing in eureka-server; the gateway routes (`lb://user-service`...); the React source code; the database tables (Hibernate creates them exactly as on the laptop). The laptop still works as before because without `SPRING_PROFILES_ACTIVE=aws` none of the aws files are loaded.

5.2 Edit 1 - the `aws` profile of the three services

Spring Boot loads `application.properties` and then, if the profile `aws` is active, `application-aws.properties` from the same folder; a property in the profile file wins. So the profile file only contains the lines that must differ in the cloud, and every value in it is a **placeholder** filled from an environment variable. The same JAR therefore runs on the laptop (no profile), on EC2 (variables from app.env) and on Fargate (variables from the task definition).

Listing 5.1 - `user-service/src/main/resources/application-aws.properties` (project-service and issue-service: same file with their name and `project_db` / `issue_db`)

```
# ========================================================================================================
# AWS profile of user-service - loaded in addition to application.properties when SPRING_PROFILES_ACTIVE=aws.
# Every value that differs between the laptop and AWS comes from an environment variable, so the SAME
# JAR runs locally (default profile), on EC2 (Milestone 1) and on ECS Fargate (Milestone 2).
# ========================================================================================================

# Eureka registry address, e.g. http://10.0.3.25:8761/eureka/ (EC2) or http://eureka.ic.local:8761/eureka/ (ECS)
eureka.client.service-url.defaultZone=${EUREKA_URL}
# Register this instance by its private IP instead of "localhost", so other machines can call it
eureka.instance.prefer-ip-address=true
# Unique per running copy - two tasks of the same service must not share an instance id
eureka.instance.instance-id=${spring.application.name}:${spring.cloud.client.ip-address}:${server.port}

# Amazon RDS (MySQL). DB_HOST = RDS endpoint. The schema is created by 01-schemas-and-users.sql.
# sslMode=REQUIRED: the connection to MySQL is encrypted.
spring.datasource.url=jdbc:mysql://${DB_HOST}:3306/user_db?sslMode=REQUIRED
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

Property	On the laptop (application.properties)	In the cloud (aws profile)	Why
`eureka.client.service-url.defaultZone`	`http://localhost:8761/eureka/`	`${EUREKA_URL}`	Eureka is on another machine: 10.0.5.157 (M1) or eureka.ic.local (M2)
`eureka.instance.prefer-ip-address`	not set (hostname = localhost)	`true`	other machines must be able to call the registered address
`eureka.instance.instance-id`	`${spring.application.name}:${server.port}`	`...:${spring.cloud.client.ip-address}:${server.port}`	unique per copy - two tasks never overwrite each other
`spring.datasource.url`	`jdbc:mysql://localhost:3306/user_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true`	`jdbc:mysql://${DB_HOST}:3306/user_db?sslMode=REQUIRED`	RDS endpoint; encrypted; the schema already exists (edit 8)
`spring.datasource.username` / `password`	from `db.properties` (root, your laptop password)	`${DB_USERNAME}` / `${DB_PASSWORD}`	a least-privilege user per service; password only on the server

- `${spring.cloud.client.ip-address}` is a value Spring Cloud computes at start-up: the IP that InetUtils chose. On EC2 that is the instance's private IP; on Fargate it was 169.254.172.2 until C3 fixed the choice - the same property, a different environment.
- `allowPublicKeyRetrieval=true` was a laptop workaround for MySQL 8 password exchange without TLS. With `sslMode=REQUIRED` the password travels inside the encrypted connection, so it is not needed.
- **An unset variable fails loudly:** "Could not resolve placeholder 'DB_HOST'" at start-up tells you immediately which line of app.env or the task definition is missing.

CHECKPOINT
Proof it worked: the Eureka dashboard (M1, through the tunnel) and the task logs (M2) showed `USER-SERVICE` registered at `10.0.x.x:8081`, and the log line "HikariPool-1 - Start completed" means the RDS login succeeded.

5.3 Edits 2, 3 and 4 - the gateway and CORS

The gateway needs the same Eureka lines (it registers itself and, more importantly, reads the registry to resolve `lb://` routes). It has no database. Its special job is CORS: it is the only component a browser talks to, so it is the one that must say "the S3 website may call me".

Listing 5.2 - `api-gateway/src/main/resources/application-aws.properties` **(edit 2)**

```
# ================================================================================================
# AWS profile of api-gateway - loaded when SPRING_PROFILES_ACTIVE=aws.
# ================================================================================================
eureka.client.service-url.defaultZone=${EUREKA_URL}
eureka.instance.prefer-ip-address=true
eureka.instance.instance-id=${spring.application.name}:${spring.cloud.client.ip-address}:${server.port}

# The React app's address on S3 (copied from the bucket's static website hosting panel), e.g.
# http://ic-frontend-nk7.s3-website-us-east-1.amazonaws.com  - read by GatewayConfig for CORS
app.cors.allowed-origins=${FRONTEND_ORIGIN}
```

Edit 3 - the laptop list of origins moved from Java code into `application.properties`, so the aws profile can replace it:

```
# Origins (scheme://host:port) allowed to call the API from a browser - read by GatewayConfig.corsWebFilter.
# (one line in the file; split with a backslash here to fit the page - both forms are valid)
app.cors.allowed-origins=http://localhost:8081,http://localhost:8082,http://localhost:8083,\
  http://localhost:3000,http://localhost:5173,http://localhost:4173
```

Edit 4 - `GatewayConfig.corsWebFilter` used to build its list with `List.of("http://localhost:5173", ...)` inside the code. Now Spring injects the property (comma-separated text becomes a `String[]` automatically):

```
    @Bean
    public CorsWebFilter corsWebFilter(@Value("${app.cors.allowed-origins}") String[] allowedOrigins) {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of(allowedOrigins));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("*"));
        config.setMaxAge(3600L);   // the browser may remember this answer for an hour

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return new CorsWebFilter(source);
    }
```

	Laptop	Milestone 1 gateway (EC2)	Milestone 2 gateway (ECS)
`app.cors.allowed-origins`	six localhost origins	`${FRONTEND_ORIGIN}`	`${FRONTEND_ORIGIN}`
`FRONTEND_ORIGIN`	-	`http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`	`http://ic-frontend-m2-nlnu19.s3-website-us-east-1.amazonaws.com`
Set in	application.properties	app.env on ic-gateway	task definition ic-api-gateway

DETAILS THAT MATTER
Only the gateway sets CORS headers; the services behind it do not, so the browser never receives the header twice (which is also an error). The origin must match the address bar exactly: scheme, host, port, no path, no trailing slash.

5.4 Edit 5 - db.properties becomes optional

On the laptop the MySQL user and password live in `src/main/resources/db.properties`, which is git-ignored so the password never reaches GitHub. `application.properties` imported it with `spring.config.import=classpath:db.properties` - a **mandatory** import. A JAR built on a machine without that file (a fresh clone, a CI server) would refuse to start, even though the aws profile provides the credentials from environment variables.

```
# before
spring.config.import=classpath:db.properties
# after - the file is used when it exists (laptop), silently skipped when it does not (cloud build)
spring.config.import=optional:classpath:db.properties
```

KEEP IN MIND
Your JARs are built on the laptop, so they DO contain your local db.properties (it is in `src/main/resources`). It holds only the laptop MySQL password and the aws profile overrides it in the cloud - but never publish the JARs or images publicly.

5.5 Edit 6 - five Dockerfiles

Milestone 2 needs each JAR as a container image. The Dockerfile sits in each application folder, next to `pom.xml`, so the build context contains `target/<app>-1.0.0.jar`. Every line is explained in 3.8 (Listing 3.4 and its table).

Listing 5.3 - `user-service/Dockerfile`

```
# Container image of user-service (Milestone 2).
# Build context = the user-service folder, after "mvn clean package" created target/user-service-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/user-service-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8081

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

File	Differs from Listing 5.3 only in	Port = server.port
`eureka-server/Dockerfile`	JAR name `eureka-server-1.0.0.jar`	`EXPOSE 8761`
`api-gateway/Dockerfile`	JAR name `api-gateway-1.0.0.jar`	`EXPOSE 8080`
`project-service/Dockerfile`	JAR name `project-service-1.0.0.jar`	`EXPOSE 8082`
`issue-service/Dockerfile`	JAR name `issue-service-1.0.0.jar`	`EXPOSE 8083`

Why the image needs no configuration inside it: all the cloud values arrive as environment variables from the task definition (C2), exactly like app.env on EC2. One image would run unchanged in a test and a production cluster.

5.6 Edits 7, 8 and 9 - the `aws/` folder

Everything needed to rebuild the deployment is versioned with the code (a case-study guideline). Placeholders such as `<RDS-ENDPOINT>` and `CHANGE_ME` passwords are replaced only on the servers or in the console.

```
issue-tracking-system/
  .gitignore  .gitattributes                edit 11
  <app>/Dockerfile                          edit 6 (five files)
  aws/
    README.md                               edit 12 - what each file is for
    ec2/user-data.sh                        edit 7 - first-boot script of each application EC2 (Listing 5.4)
    ec2/app.env.example                     edit 7 - template of /etc/issue-tracker/app.env (Listing 5.5)
    rds/01-schemas-and-users.sql            edit 8 - schemas + one user per service (Listing 5.6)
    rds/02-lambda-user.sql                  edit 8 - column-level user for the Lambda (Listing 5.7)
    lambda/profile-photo-processor/         edit 9 - lambda_function.py (Listing 5.8), requirements.txt,
                                                     package-lambda.sh (5.10.6)
    scripts/deploy-jar.ps1                  edit 10 - 5.10.2
    scripts/build-frontend.ps1              edit 10 - 5.10.3
    scripts/copy-to-builder.ps1             edit 10 - 5.10.4
    scripts/build-and-push.sh               edit 10 - 5.10.5
```

Edit 7 - the settings file template

Listing 5.5 - `aws/ec2/app.env.example`

```
# Template of /etc/issue-tracker/app.env on each application EC2 (owner root, chmod 600).
# The real file holds passwords: create it on the server only, NEVER commit it.
SPRING_PROFILES_ACTIVE=aws
EUREKA_URL=http://<EUREKA-PRIVATE-IP>:8761/eureka/
DB_HOST=<RDS-ENDPOINT>
DB_USERNAME=user_svc
DB_PASSWORD=<user_svc password>
# api-gateway only (no DB_ lines on the gateway):
# FRONTEND_ORIGIN=http://<bucket>.s3-website-us-east-1.amazonaws.com
# eureka-server: leave the file empty
```

systemd reads the real file (`EnvironmentFile=`) and hands every line to Java as an environment variable. It is owned by root with mode 600, so only root can read the passwords; the app still receives them because systemd (root) reads the file before starting Java as ec2-user.

Edit 8 - the database scripts

Listing 5.6 - `aws/rds/01-schemas-and-users.sql`

```
-- ================================================================================
-- 01-schemas-and-users.sql - run ONCE as the RDS master user, from ic-gateway:
--   mysql -h <RDS-ENDPOINT> -u admin -p --ssl < 01-schemas-and-users.sql
-- One schema per microservice and one MySQL user per service that can use only its own schema.
-- '10.0.%' = connections from inside ic-vpc only. Replace the three passwords before running.
-- ================================================================================
CREATE DATABASE IF NOT EXISTS user_db;
CREATE DATABASE IF NOT EXISTS project_db;
CREATE DATABASE IF NOT EXISTS issue_db;

CREATE USER IF NOT EXISTS 'user_svc'@'10.0.%'    IDENTIFIED BY 'CHANGE_ME_User#2026';
CREATE USER IF NOT EXISTS 'project_svc'@'10.0.%' IDENTIFIED BY 'CHANGE_ME_Project#2026';
CREATE USER IF NOT EXISTS 'issue_svc'@'10.0.%'   IDENTIFIED BY 'CHANGE_ME_Issue#2026';

-- Hibernate (ddl-auto=update) needs CREATE/ALTER/INDEX/REFERENCES to build its tables. No DROP, no other
» schema.
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, REFERENCES ON user_db.*    TO 'user_svc'@'10.0.%';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, REFERENCES ON project_db.* TO
» 'project_svc'@'10.0.%';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, REFERENCES ON issue_db.*   TO 'issue_svc'@'10.0.%';

SHOW DATABASES;
SELECT user, host FROM mysql.user WHERE user LIKE '%svc';
```

Line	Why
`CREATE DATABASE IF NOT EXISTS ...` x 3	database per service; `IF NOT EXISTS` makes the script safe to run twice

Line	Why
`'user_svc'@'10.0.%'`	the user exists only for connections from inside ic-vpc
`GRANT ... ON user_db.*`	the user sees one schema; no DROP, no GRANT OPTION
`CREATE, ALTER, INDEX, REFERENCES`	Hibernate `ddl-auto=update` creates and updates the tables on the first start

Listing 5.7 - `aws/rds/02-lambda-user.sql` **(designed, for the Lambda)**

```
-- ================================================================================
-- 02-lambda-user.sql - run AFTER user-service has started once (its tables must exist):
--   mysql -h <RDS-ENDPOINT> -u admin -p --ssl < 02-lambda-user.sql
-- Column-level grant: the Lambda may read user names and change the profile column - nothing else.
-- ================================================================================
CREATE USER IF NOT EXISTS 'photo_lambda'@'10.0.%' IDENTIFIED BY 'CHANGE_ME_Lambda#2026';
GRANT SELECT (username), UPDATE (profile) ON user_db.`user` TO 'photo_lambda'@'10.0.%';
SHOW GRANTS FOR 'photo_lambda'@'10.0.%';
```

Edit 9 - the Lambda function (designed, not deployed)

Listing 5.8 - `aws/lambda/profile-photo-processor/lambda_function.py`

```
"""
InfinityCastle - profile photo processor (AWS Lambda, Python 3.12).

Trigger : S3 "All object create events" on the photos bucket, prefix "profile-photos/".
Input   : uploaded files named profile-photos/<username>.<jpg|jpeg|png|gif|webp>
Work    : 1. check the key, the extension and the size (max 2 MB)
          2. ask S3 for the object's Content-Type (must be image/...)
          3. save the photo's public URL as that user's profile in RDS (user_db.user.profile)
Output  : a status per file; every decision is logged to CloudWatch Logs.

Environment variables: DB_HOST (RDS endpoint), DB_NAME (user_db), DB_USER (photo_lambda), DB_PASSWORD.
global-bundle.pem (the Amazon RDS certificates) is packaged next to this file, for TLS to MySQL.
"""
import logging
import os
import urllib.parse

import boto3
import pymysql
from pymysql.constants import CLIENT

logger = logging.getLogger()
logger.setLevel(logging.INFO)

PREFIX = "profile-photos/"
ALLOWED_EXTENSIONS = {"jpg", "jpeg", "png", "gif", "webp"}
MAX_SIZE_BYTES = 2 * 1024 * 1024
CA_BUNDLE = os.path.join(os.path.dirname(__file__), "global-bundle.pem")

# Created once per execution environment and reused by warm invocations
s3 = boto3.client("s3")


def lambda_handler(event, context):
    """Entry point. One S3 event may contain several records."""
    results = []
    for record in event.get("Records", []):
        bucket = record["s3"]["bucket"]["name"]
        key = urllib.parse.unquote_plus(record["s3"]["object"]["key"])   # keys arrive URL-encoded
        size = record["s3"]["object"].get("size", 0)
        results.append(process_photo(bucket, key, size))
    return {"results": results}


def process_photo(bucket, key, size):
    """Validates one upload and stores its URL. Returns a short status (also useful in console tests)."""
    if not key.startswith(PREFIX):
        return skip(key, "not under " + PREFIX)
    file_name = key[len(PREFIX):]
    if "/" in file_name or "." not in file_name:
        return skip(key, "expected profile-photos/<username>.<extension>")

    username, extension = file_name.rsplit(".", 1)
    username = username.lower()            # usernames are stored in lower case
    extension = extension.lower()
    if extension not in ALLOWED_EXTENSIONS:
        return skip(key, "." + extension + " is not an accepted image type")
```

```
    if size > MAX_SIZE_BYTES:
        return skip(key, "file is larger than 2 MB")

    # Read only the object's metadata (no download). Reached through the S3 gateway endpoint.
    content_type = s3.head_object(Bucket=bucket, Key=key).get("ContentType", "")
    if not content_type.startswith("image/"):
        return skip(key, "Content-Type is " + content_type)

    region = os.environ["AWS_REGION"]      # set by Lambda itself
    photo_url = "https://%s.s3.%s.amazonaws.com/%s" % (bucket, region, urllib.parse.quote(key))

    connection = pymysql.connect(
        host=os.environ["DB_HOST"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        database=os.environ.get("DB_NAME", "user_db"),
        ssl={"ca": CA_BUNDLE},             # encrypted connection, RDS certificate verified
        client_flag=CLIENT.FOUND_ROWS,     # rowcount = matched rows, even when the URL is unchanged
        connect_timeout=5,
    )
    try:
        with connection.cursor() as cursor:
            matched = cursor.execute(
                "UPDATE `user` SET profile = %s WHERE username = %s", (photo_url, username))
        connection.commit()
    finally:
        connection.close()

    if matched == 0:
        return skip(key, "no user with username '" + username + "'")
    logger.info("Profile photo of %s set to %s", username, photo_url)
    return {"key": key, "status": "UPDATED", "username": username, "url": photo_url}


def skip(key, reason):
    """Logs why an upload was ignored. The object stays in S3 and the database is not touched."""
    logger.warning("Skipped %s: %s", key, reason)
    return {"key": key, "status": "SKIPPED", "reason": reason}
```

`requirements.txt` contains one line, `pymysql>=1.1,<2` - the MySQL driver, which is not part of the Lambda Python runtime. How the function works is explained in 3.14; how it would be deployed in 6.14.

5.7 Edits 11 and 12 - Git hygiene and documentation

Listing 5.9 - the AWS block added to `.gitignore`

```
# AWS (Week 5): private keys, real settings files with passwords, Lambda build output
*.pem
*.ppk
app.env
*.zip
aws/lambda/**/build/
__pycache__/
```

Listing 5.10 - `.gitattributes`

```
# Keep Linux line endings (LF) for files that run on Linux (EC2, Docker, Lambda), even when the repository is
# cloned on Windows with core.autocrlf=true. A shell script with Windows line endings (CRLF) fails on Linux
# with errors like  $'\r': command not found.
*.sh        text eol=lf
Dockerfile  text eol=lf
*.sql       text eol=lf
*.py        text eol=lf
*.env.example text eol=lf
# PowerShell scripts are run on Windows
*.ps1       text eol=crlf
```

- `*.pem`, `*.ppk` - SSH private keys can never be committed by accident, even if one is copied into the project folder.
- `app.env` - a real settings file holds passwords; only `app.env.example` (placeholders) is in Git.
- `*.zip`, `build/` - generated Lambda packages are rebuilt by the script, never stored.
- **Line endings:** Windows writes CRLF, Linux expects LF. A shell script with CRLF fails on Linux with `$'\r': command not found` or `bad interpreter`. `.gitattributes` makes Git always check out the Linux files with LF and the PowerShell scripts with CRLF, whatever `core.autocrlf` says.

- `aws/README.md` - a table of every deployment file, where it is used and what it does (the guide is the full runbook).

5.8 Cloud-side settings C1-C5 - the values that feed the placeholders

The code edits create placeholders; these settings fill them. They are listed here without passwords - those exist only in each server's app.env and in the task definitions.

C1 - app.env on each Milestone 1 server

Server	SPRING_PROFILES_ACTIVE	EUREKA_URL	DB_HOST / DB_USERNAME	FRONTEND_ORIGIN
ic-gateway	`aws`	`http://10.0.5.157:8761/eureka/`	-	M1 site: `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`
ic-user	`aws`	same	RDS endpoint / `user_svc`	-
ic-project	`aws`	same	RDS endpoint / `project_svc`	-
ic-issue	`aws`	same	RDS endpoint / `issue_svc`	-

On **ic-eureka** the file stays empty: eureka-server has no aws profile, does not register with itself and needs no database - the laptop configuration works unchanged on its own server.

C2 + C3 - environment of each Milestone 2 task definition

Task definition	Environment variables (values)
`ic-eureka-server:1`	none
`ic-api-gateway:4`	`SPRING_PROFILES_ACTIVE=aws`, `EUREKA_URL=http://eureka.ic.local:8761/eureka/`, `FRONTEND_ORIGIN=http://ic-frontend-m2-nlnu19.s3-website-us-east-1.amazonaws.com`, `SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0`
`ic-user-service:4`	`SPRING_PROFILES_ACTIVE=aws`, `EUREKA_URL=` (as above), `DB_HOST=ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com`, `DB_USERNAME=user_svc`, `DB_PASSWORD=` (secret), `SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0`
`ic-project-service:4`	same with `DB_USERNAME=project_svc`
`ic-issue-service:4`	same with `DB_USERNAME=issue_svc`

- **C3 is the only cloud fix that is not in the code.** It is a Spring property set through relaxed binding (3.12), so no new JAR or image was needed - only new task-definition revisions. It could also be added to the aws profile as `spring.cloud.inetutils.preferred-networks=10.0` (harmless on EC2, whose IPs are 10.0.x too) in a future build.
- **C4** - the React API address is compiled into the JavaScript, so it is not a runtime variable: each milestone has its own build in its own bucket (5.10.3).
- **C5** - the two gateways each allow exactly their own site. Opening the M1 site against the M2 gateway would be blocked by CORS, and that is intended.

5.9 Why scripts at all

Every deployment step was first done by hand once, so you understand it; then the repeated ones were put into scripts so they are fast, identical every time and hard to get wrong. The figure shows where each script sits in the two deployment paths:

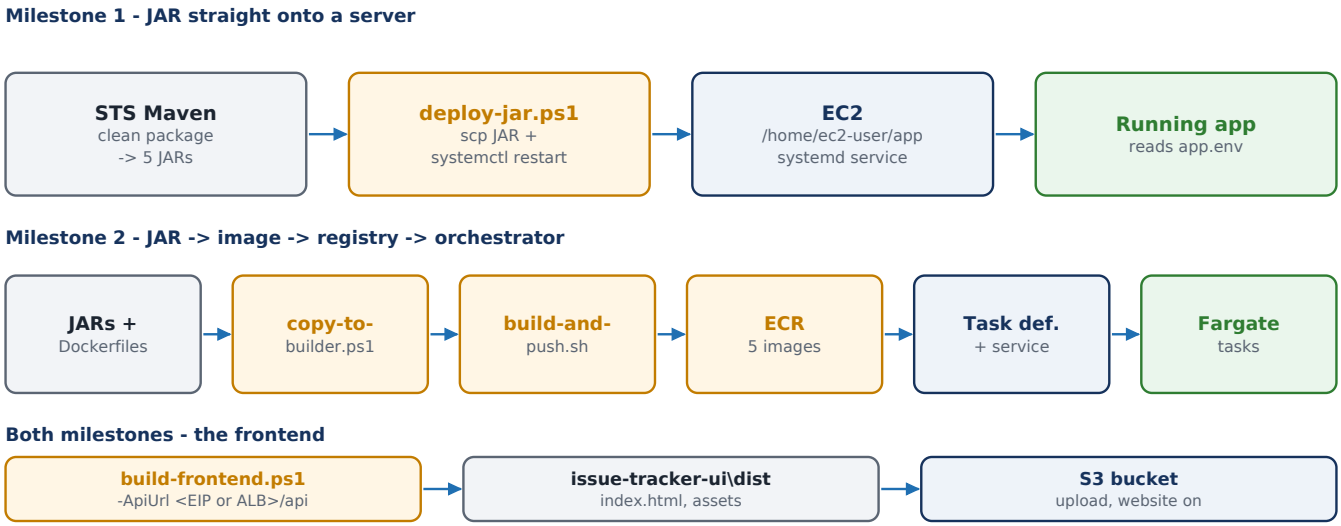


Figure 5.2 - The scripts in the Milestone 1 and Milestone 2 deployment paths.

Script	Runs on	When	Replaces by hand	Time
`user-data.sh`	each new EC2, automatically, as root	launch (M1)	~15 commands per server: install Java, client, agent; folders; service file; agent config	~2 min x 5 -> 0
`deploy-jar.ps1`	laptop (PowerShell)	every new JAR (M1)	scp + ssh + restart + status	~2 min -> 20 s
`build-frontend.ps1`	laptop	every React build for AWS	set variable, cd, build, unset, check	and no forgotten localhost build
`copy-to-builder.ps1`	laptop	before every image build (M2)	1 ssh + 11 scp with exact paths	~5 min -> 1 command
`build-and-push.sh`	ic-builder (bash)	every image build (M2)	5 builds + login + 5 tags + 5 pushes	16 commands -> 1
`package-lambda.sh`	an Amazon Linux EC2	Lambda build (designed)	pip into a folder, download certificate, zip correctly	error-prone by hand

HOW TO RUN

All laptop scripts are run from the `issue-tracking-system` folder in PowerShell (`cd "C:\Users\nlnu19\Documents\workspace-spring-tools-for-eclipse-5.4.0.RELEASE\IC App - AWS\issue-tracking-system"`). If Windows refuses with "running scripts is disabled", start them as `powershell -ExecutionPolicy Bypass -File .\aws\scripts\<name>.ps1 ...`.

5.10 Every script, line by line

5.10.1 user-data.sh - prepares an application server on its first boot (M1)

Listing 5.4 - `aws/ec2/user-data.sh`

```
#!/bin/bash
# ================================================================================================
# EC2 user data for ONE Issue Tracker application server (Milestone 1).
# Runs once, as root, on first boot. Change SERVICE (and nothing else) for each of the five instances:
#   eureka-server | api-gateway | user-service | project-service | issue-service
# It installs Java 21, the MySQL client and the CloudWatch agent, prepares folders, the systemd service
# and the CloudWatch agent configuration. The JAR and the settings file are added later (Part 6.7).
# ================================================================================================
SERVICE=user-service
JAR=${SERVICE}-1.0.0.jar

# 1. Software
dnf install -y java-21-amazon-corretto-headless mariadb105 amazon-cloudwatch-agent

# 2. Folders: the JAR, the settings (secret, root only) and the log file
mkdir -p /home/ec2-user/app /etc/issue-tracker /var/log/issue-tracker
chown ec2-user:ec2-user /home/ec2-user/app
touch /etc/issue-tracker/app.env
chmod 600 /etc/issue-tracker/app.env

# 3. systemd service: start at boot, restart on failure, run as ec2-user, log to a file
cat > /etc/systemd/system/issue-tracker.service <<EOF
[Unit]
Description=Issue Tracker - ${SERVICE}
After=network-online.target
Wants=network-online.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/app
EnvironmentFile=/etc/issue-tracker/app.env
ExecStart=/usr/bin/java -Xms128m -Xmx384m -jar /home/ec2-user/app/${JAR}
SuccessExitStatus=143
Restart=on-failure
RestartSec=10
StandardOutput=append:/var/log/issue-tracker/app.log
StandardError=append:/var/log/issue-tracker/app.log

[Install]
WantedBy=multi-user.target
EOF

# 4. CloudWatch agent: ship the log file + memory and disk usage (the \$ keeps ${aws:...} for the agent)
cat > /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json <<EOF
{
  "agent": { "run_as_user": "root" },
  "logs": { "logs_collected": { "files": { "collect_list": [
    { "file_path": "/var/log/issue-tracker/app.log",
      "log_group_name": "/issue-tracker/m1/${SERVICE}",
      "log_stream_name": "{instance_id}",
      "retention_in_days": 14 } ] } } },
  "metrics": {
    "namespace": "IssueTracker/EC2",
    "append_dimensions": { "InstanceId": "\${aws:InstanceId}" },
    "metrics_collected": {
      "mem":  { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] } } }
}
EOF
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -s \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json

# 5. Register the service (it is started by hand once the JAR and app.env are in place)
systemctl daemon-reload
systemctl enable issue-tracker
```

1. `SERVICE=user-service` - the only line you change per instance (`eureka-server`, `api-gateway`, `user-service`, `project-service`, `issue-service`). `JAR=` derives the file name from it.
2. `dnf install -y ...` - Java 21 (Corretto, headless = no desktop libraries), the MariaDB client (to run the SQL scripts and test RDS) and the CloudWatch agent. `-y` answers "yes" because nobody is there to type it.

3. `mkdir` / `chown` / `touch` / `chmod 600` - the JAR folder owned by ec2-user (so `scp` can write into it), the settings file readable only by root, the log folder.
4. `cat > ...issue-tracker.service <<EOF` - writes the systemd unit (every line explained in 3.5). `${SERVICE}` and `${JAR}` are filled in by bash while writing.
5. The CloudWatch agent configuration: ship `app.log` to the log group `/issue-tracker/m1/<service>` (one stream per instance, 14 days retention) and send memory and disk usage to the namespace `IssueTracker/EC2`. `\${aws:InstanceId}` is escaped so the agent, not bash, replaces it.
6. `amazon-cloudwatch-agent-ctl -a fetch-config ... -s` loads that configuration and starts the agent.
7. `systemctl daemon-reload` + `enable` - systemd reads the new unit and will start it at every boot. It is not started yet: the JAR and app.env do not exist until you deploy them.

> **CHECKPOINT**
> On a new server: `java -version` shows 21, `systemctl is-enabled issue-tracker` says `enabled`, and `sudo tail /var/log/cloud-init-output.log` ends without errors.

5.10.2 deploy-jar.ps1 - copy one JAR to its server and restart it (M1)

```
.\aws\scripts\deploy-jar.ps1 -App user-service -Server <public IP of ic-user>
.\aws\scripts\deploy-jar.ps1 -App api-gateway  -Server 52.1.6.53
```

Listing 5.11 - `aws/scripts/deploy-jar.ps1`

```
<#
  deploy-jar.ps1 - Milestone 1: copy ONE application's JAR to its EC2 and (re)start it.

  Run in PowerShell from the issue-tracking-system folder, after "mvn clean package" for that application:
      .\aws\scripts\deploy-jar.ps1 -App user-service -Server 3.91.24.180
  If Windows blocks scripts, run it like this instead:
      powershell -ExecutionPolicy Bypass -File .\aws\scripts\deploy-jar.ps1 -App user-service -Server
» 3.91.24.180

  The server must already have /etc/issue-tracker/app.env filled in (Part 6.7 of the guide).
  The key defaults to C:\Users\<you>\aws-keys\ic-key.pem - pass -Key to use another path.
#>
param(
    [Parameter(Mandatory = $true)]
    [ValidateSet("eureka-server", "api-gateway", "user-service", "project-service", "issue-service")]
    [string]$App,

    [Parameter(Mandatory = $true)]
    [string]$Server,                                  # public IP (or Elastic IP) of the EC2

    [string]$Key = "$env:USERPROFILE\aws-keys\ic-key.pem"
)

$jar = ".\$App\target\$App-1.0.0.jar"
if (-not (Test-Path $jar)) { Write-Error "Not found: $jar - run 'mvn clean package' for $App first."; exit 1 }
if (-not (Test-Path $Key)) { Write-Error "Key not found: $Key - pass -Key <path to ic-key.pem>."; exit 1 }

Write-Host "1/2 Copying $jar to $Server ..."
scp -i $Key $jar "ec2-user@${Server}:/home/ec2-user/app/"
if ($LASTEXITCODE -ne 0) { Write-Error "scp failed (security group, IP or key?)"; exit 1 }

Write-Host "2/2 Restarting the service on $Server (first start of Spring Boot takes about 30-60 s) ..."
ssh -i $Key "ec2-user@$Server" "sudo systemctl restart issue-tracker && sleep 3 && systemctl --no-pager status
» issue-tracker | head -n 5"
Write-Host "Follow the log with:  ssh -i $Key ec2-user@$Server 'tail -f /var/log/issue-tracker/app.log'"
```

1. `param(...)` - `-App` must be one of the five names (`ValidateSet` rejects typos before anything runs); `-Server` is the public IP; `-Key` defaults to `C:\Users\<you>\aws-keys\ic-key.pem`.
2. Two `Test-Path` checks: the JAR exists (did you run `clean package`?) and the key exists - clear messages instead of cryptic scp errors.
3. `scp -i $Key $jar ec2-user@${Server}:/home/ec2-user/app/` - copies the JAR, replacing the old one. `$LASTEXITCODE` stops the script if the copy failed (wrong IP after a stop/start, SG, key).

4. `ssh ... "sudo systemctl restart issue-tracker && sleep 3 && systemctl status ... | head -n 5"` - restarts the service and shows whether it is `active (running)`.
5. Prints the `tail -f` command to follow the start-up log.

5.10.3 build-frontend.ps1 - build React for AWS (both milestones)

```
.\aws\scripts\build-frontend.ps1 -ApiUrl http://52.1.6.53:8080/api                            # M1
.\aws\scripts\build-frontend.ps1 -ApiUrl http://ic-alb-1965966534.us-east-1.elb.amazonaws.com/api   # M2
```

Listing 5.12 - `aws/scripts/build-frontend.ps1`

```
<#
  build-frontend.ps1 - build the React app for AWS (the files you upload to the S3 website bucket).

  Run in PowerShell from the issue-tracking-system folder:
      Milestone 1:  .\aws\scripts\build-frontend.ps1 -ApiUrl http://<GATEWAY-ELASTIC-IP>:8080/api
      Milestone 2:  .\aws\scripts\build-frontend.ps1 -ApiUrl http://<ALB-DNS-NAME>/api
  Result: issue-tracker-ui\dist  ->  upload its CONTENTS (index.html, favicon.svg, assets folder) to the bucket.

  How it works: Vite bakes VITE_API_BASE_URL into the JavaScript at build time. A variable that already exists
  in the environment wins over the .env files, so the script sets it only for this build and removes it again.
  Local "npm run dev" keeps using http://localhost:8080/api.
#>
param(
    [Parameter(Mandatory = $true)]
    [string]$ApiUrl
)

if ($ApiUrl -notmatch '^https?://.+/api$') { Write-Error "ApiUrl must look like http://<host>[:port]/api"; exit
» 1 }

Push-Location .\issue-tracker-ui
try {
    $env:VITE_API_BASE_URL = $ApiUrl
    Write-Host "Building the React app with VITE_API_BASE_URL=$ApiUrl ..."
    npm run build
    if ($LASTEXITCODE -ne 0) { Write-Error "npm run build failed"; exit 1 }
    Write-Host "Done. Upload the contents of issue-tracker-ui\dist to the S3 bucket."
}
finally {
    Remove-Item Env:VITE_API_BASE_URL -ErrorAction SilentlyContinue
    Pop-Location
}
```

1. The regular expression `^https?://.+/api$` rejects a URL without `http://` or without the final `/api` - the two most common mistakes, which would produce a site that silently calls the wrong address.
2. `Push-Location .\issue-tracker-ui` - goes into the React project (and remembers where you were).
3. `$env:VITE_API_BASE_URL = $ApiUrl` - sets the variable for this PowerShell session only. Vite gives variables that already exist in the environment priority over `.env` files, so the build uses it.
4. `npm run build` - Vite replaces every `import.meta.env.VITE_API_BASE_URL` in the code with the text of the URL and writes `dist/` (index.html, favicon, assets).
5. `finally { Remove-Item Env:VITE_API_BASE_URL; Pop-Location }` - runs even after an error: the variable is removed (so a later `npm run dev` uses localhost again) and you are back in the original folder.
6. Then: upload the **contents** of `issue-tracker-ui\dist` to the bucket (M1: `ic-frontend-nlnu19`, M2: `ic-frontend-m2-nlnu19`).

> **WHY A VARIABLE, NOT A FILE**
> A `.env.production` file would also work, but it is one more file that can hold the wrong address and be committed by mistake; the variable leaves nothing behind. (The remote tools are also not allowed to write `.env` files on your laptop.)

5.10.4 copy-to-builder.ps1 - send Dockerfiles and JARs to ic-builder (M2)

```
.\aws\scripts\copy-to-builder.ps1 -Server <public IP of ic-builder>     # the IP only - no "ec2-user@"
```

Listing 5.13 - `aws/scripts/copy-to-builder.ps1`

```
<#
  copy-to-builder.ps1 - Milestone 2: copy the five Dockerfiles and JARs to the builder EC2 (ic-builder).

  Run in PowerShell from the issue-tracking-system folder, after "mvn clean package" for all five applications:
      .\aws\scripts\copy-to-builder.ps1 -Server <BUILDER-PUBLIC-IP>
  Result on the builder:  ~/build/<app>/Dockerfile  and  ~/build/<app>/target/<app>-1.0.0.jar
  Then on the builder:    bash build-and-push.sh
#>
param(
    [Parameter(Mandatory = $true)]
    [string]$Server,
    [string]$Key = "$env:USERPROFILE\aws-keys\ic-key.pem"
)

$apps = "eureka-server", "api-gateway", "user-service", "project-service", "issue-service"

ssh -i $Key "ec2-user@$Server" "mkdir -p
» ~/build/{eureka-server,api-gateway,user-service,project-service,issue-service}/target"
if ($LASTEXITCODE -ne 0) { Write-Error "ssh failed (security group, IP or key?)"; exit 1 }

foreach ($app in $apps) {
    $jar = ".\$app\target\$app-1.0.0.jar"
    if (-not (Test-Path $jar)) { Write-Error "Not found: $jar - run 'mvn clean package' first."; exit 1 }
    Write-Host "Copying $app ..."
    scp -i $Key ".\$app\Dockerfile" "ec2-user@${Server}:~/build/$app/"
    scp -i $Key $jar "ec2-user@${Server}:~/build/$app/target/"
}
scp -i $Key ".\aws\scripts\build-and-push.sh" "ec2-user@${Server}:~/build-and-push.sh"
Write-Host "Done. On the builder run:  bash build-and-push.sh"
```

1. `ssh ... "mkdir -p ~/build/{eureka-server,...}/target"` - one command on the builder creates all ten folders: bash expands `{a,b}` into a list, `-p` creates parents and does not complain if they exist.
2. For each of the five apps: check that the JAR exists, then `scp` the Dockerfile to `~/build/<app>/` and the JAR to `~/build/<app>/target/` - the same layout as on the laptop, so the Dockerfile's `COPY target/<app>-1.0.0.jar` works unchanged.
3. `scp ... build-and-push.sh` to the home folder of the builder.

> **SEEN ON THE DAY**
> Two things that happened: passing `-Server ec2-user@13.220.31.5` produced `ec2-user@ec2-user@...` (the script adds the user itself); and the eureka JAR once stopped with "broken pipe" (a dropped connection) - simply run the script again, it overwrites everything. The builder's public IP changes after every stop/start.

5.10.5 build-and-push.sh - build five images and push them to ECR (M2)

```
bash build-and-push.sh           # tag 1.0.0
bash build-and-push.sh 1.0.1     # a new version after code changes
```

Listing 5.14 - `aws/scripts/build-and-push.sh`

```
#!/bin/bash
# ================================================================================================
# build-and-push.sh - Milestone 2, run on ic-builder (Docker installed, IAM role ic-builder-role).
# Builds the five images from ~/build/<app> and pushes them to the ECR repositories issue-tracker/<app>.
# The repositories must exist (ECR console). No access keys are needed: the instance role is used.
#   Usage:  bash build-and-push.sh            (version tag 1.0.0)
#           bash build-and-push.sh 1.0.1      (another version tag)
# ================================================================================================
set -euo pipefail                     # stop at the first error, unknown variable or failed pipe

VERSION=${1:-1.0.0}
REGION=${AWS_REGION:-us-east-1}
APPS="eureka-server api-gateway user-service project-service issue-service"
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
REGISTRY=$ACCOUNT.dkr.ecr.$REGION.amazonaws.com

cd ~/build
```

```
for app in $APPS; do
  echo "== Building issue-tracker/$app:$VERSION"
  docker build -t "issue-tracker/$app:$VERSION" "./$app"
done

echo "== Logging Docker in to $REGISTRY (token valid 12 hours)"
aws ecr get-login-password --region "$REGION" | docker login --username AWS --password-stdin "$REGISTRY"

for app in $APPS; do
  echo "== Pushing $app"
  docker tag "issue-tracker/$app:$VERSION" "$REGISTRY/issue-tracker/$app:$VERSION"
  docker push "$REGISTRY/issue-tracker/$app:$VERSION"
done

echo
echo "Image URIs for the ECS task definitions:"
for app in $APPS; do echo "  $REGISTRY/issue-tracker/$app:$VERSION"; done
```

1. `set -euo pipefail` - stop at the first failing command (`-e`), treat an unset variable as an error (`-u`), and let a failure inside a pipe count (`pipefail`). Without it, a failed build would be followed by a push of an old image.
2. `VERSION=${1:-1.0.0}` - the first argument, or 1.0.0. `REGION=${AWS_REGION:-us-east-1}` likewise.
3. `aws sts get-caller-identity --query Account` - asks AWS "who am I?" with the instance role and extracts the account number, so the script contains no account-specific value.
4. `docker build -t issue-tracker/$app:$VERSION ./$app` x 5 - one image per folder.
5. `aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY` - the role `ic-builder-role` gets a 12-hour token; the pipe passes it without showing it on screen or in history.
6. `docker tag` + `docker push` x 5 - name each image with the registry address and upload it. Only layers ECR does not have yet are uploaded, so a second push is fast.
7. Prints the five image URIs to paste into the task definitions (or use "Browse ECR images").

> **NOTE**
> Prerequisites: the five ECR repositories `issue-tracker/<app>` exist (a push to a missing repository fails with "name unknown"), Docker runs and ec2-user is in the docker group (log out and in once after `usermod`), and the role is attached to the instance.

5.10.6 package-lambda.sh - build the Lambda zip (designed)

Listing 5.15 - `aws/lambda/profile-photo-processor/package-lambda.sh`

```
#!/bin/bash
# ================================================================================================
# package-lambda.sh - Milestone 1: build photo-lambda.zip for the profile-photo Lambda.
# Run on an Amazon Linux EC2 that has internet access (ic-gateway), in a folder containing
# lambda_function.py, requirements.txt and this script:   bash package-lambda.sh
# The zip holds, at its top level: our code + PyMySQL (not part of the Lambda runtime) + the Amazon RDS
# certificate bundle used to verify the TLS connection to MySQL.
# ================================================================================================
set -euo pipefail

sudo dnf install -y python3-pip zip > /dev/null
rm -rf build photo-lambda.zip
mkdir build
cp lambda_function.py build/
pip3 install --quiet --target build -r requirements.txt
curl -sS -o build/global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
(cd build && zip -qr ../photo-lambda.zip .)
echo "Created $(pwd)/photo-lambda.zip - copy it to your laptop and upload it in the Lambda console."
```

1. Installs `pip` and `zip` on the Amazon Linux machine (the same Linux family as the Lambda runtime, so compiled pieces of libraries fit).
2. `pip3 install --target build -r requirements.txt` - puts PyMySQL INTO the build folder instead of into the system.
3. Downloads `global-bundle.pem` (the Amazon RDS certificate authorities) so the function can verify the database certificate.

4. `cd build && zip -qr ../photo-lambda.zip .` - zips the CONTENTS: `lambda_function.py` must be at the top level of the zip, otherwise Lambda reports "Unable to import module 'lambda_function'".

5.11 Build the JARs and check that the laptop still works

1. In Spring Tools, for each of the five projects: right-click -> **Run As -> Maven build...** -> Goals `clean package` -> Run. Each `target/` folder now contains `<app>-1.0.0.jar` (the `.jar.original` file is not needed).
2. Start everything locally as usual (no profile active). Log in, open a board, create an issue: it works exactly as before, because the aws files are not loaded.
3. Optional local test of the profile: in a Run Configuration's Environment tab set `SPRING_PROFILES_ACTIVE=aws`, `EUREKA_URL=http://localhost:8761/eureka/`, `DB_HOST=localhost` and local credentials; the app starts and registers its IP (not "localhost") on http://localhost:8761.

> **CHECKPOINT**
> Five JARs exist; the app still works on the laptop; `git status` shows the new files and no `.pem`, `app.env` or `db.properties`.

PART 6

Milestone 1 - manual deployment, step by step

Rewritten after the real deployment of 19 October 2026 on the company laptop. Every click, command and value in the order that worked, the mistakes that happened and their fixes, what to write down, and what changes when the servers are stopped and started again.

> **ABOUT THE VALUES IN THIS PART**
> This part contains the **addresses** of your deployment (no passwords). Keep the PDF on your laptops; do not upload it to a public place. Every value marked *changes* must be looked up again after a stop/start - section 6.17 explains exactly which ones.

6.0 What is different from the first plan

The first version of this guide was written before anyone touched the console. The table lists everything that turned out differently, so you can explain each decision in the interview.

Area	First plan	What worked (use this)	Why
Laptop	personal laptop paths (`C:\Users\nisha`)	company laptop: project in `...\IC App - AWS\issue-tracking-system`, key in `C:\Users\nlnu19\aws-keys`	only the company laptop has AWS access
SSH rule	SSH 22 from **My IP**	SSH 22 from **Anywhere-IPv4 (0.0.0.0/0)**	the company network sends traffic through a proxy/VPN; the address the console detects is not the address SSH arrives from, so SSH timed out. Login still needs the private key.
RDS template	Free tier or Dev/Test, 20 GiB	**Sandbox** template, db.t3.micro	Dev/Test was refused: "not authorized to perform rds:CreateDBInstance" - the bootcamp policy only allows small instances; Dev/Test also proposed 200 GiB
DB subnet group	two private subnets	exactly **private1 (us-east-1a)** and **private2 (us-east-1b)**	two subnets in the same AZ gave "doesn't meet AZ coverage requirement"
ic-rds-sg	rule from ic-backend-sg	source must be the **security group** `sg-0065c0657e2fd6c17` (ic-backend-sg)	with a wrong source `mysql` hung at the password prompt (packets dropped)
ic-eureka	ic-backend-sg	ic-backend-sg **only**	ic-gateway-public-sg had been selected by mistake; it would publish port 8080 of a private server
Copying files	`scp file host:~`	never forget the `:~` (or `:/path`)	without the colon scp made a **local** copy named like the host
mysql command	console snippet	the guide's command with `--ssl`	the console snippet uses `--ssl-mode`, which the MariaDB client on Amazon Linux does not know
Order	all five servers, then deploy	Eureka deployed alone right after the first SSH test	it needs no database and confirms the whole EC2 setup early
Lambda, CloudWatch dashboards, full tests	part of Milestone 1	kept as **theory and design** (6.14-6.16, 3.13-3.14); logs and automatic metrics exist	the time went into Milestone 2; the design is complete, coded and explainable

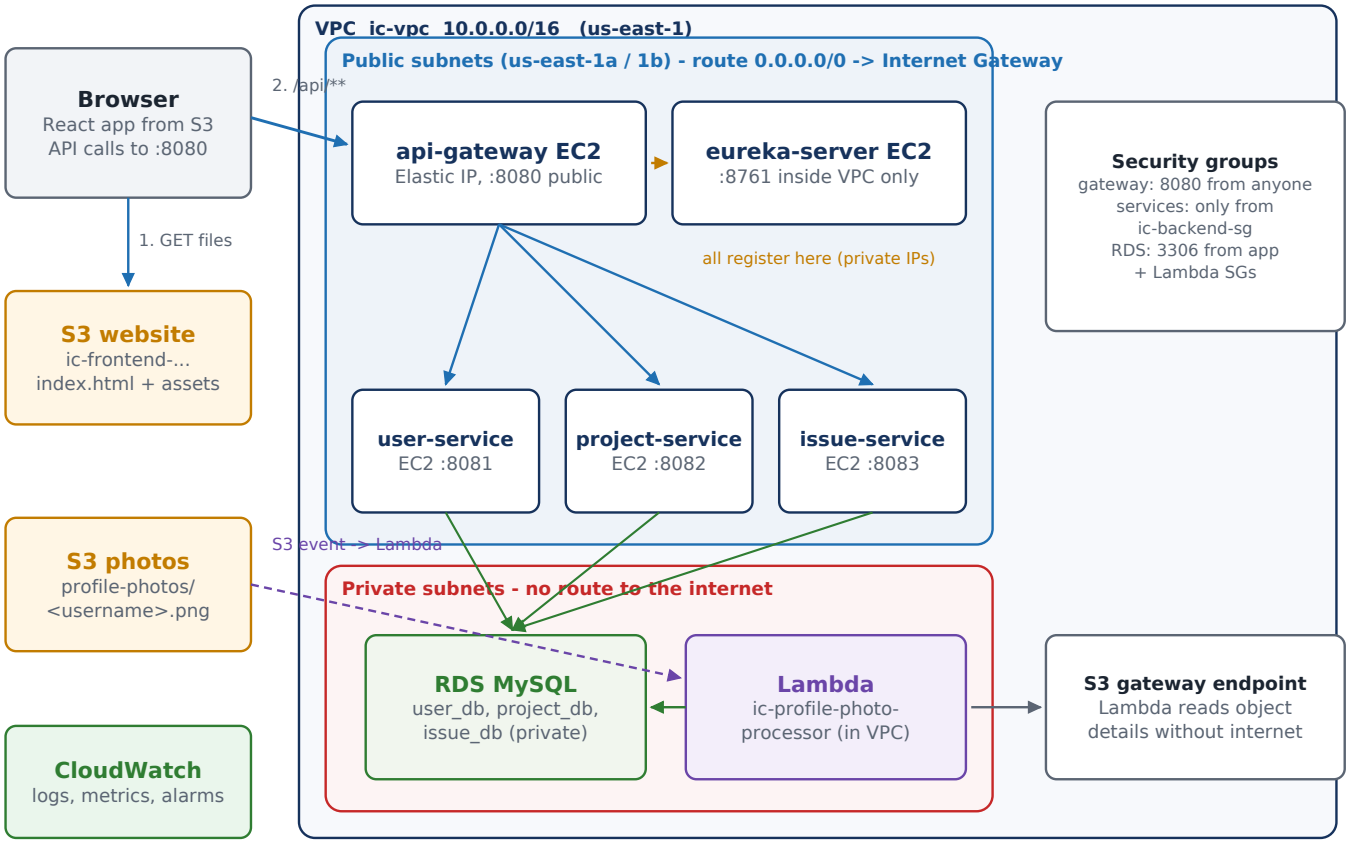


Figure 6.0 - Milestone 1 as deployed: browser -> S3 website; browser -> gateway Elastic IP -> services (found through Eureka) -> private RDS.

6.1 Your notepad worksheet - what to save, what not to save

Create the text file `C:\Users\nlnu19\aws-keys\ic-worksheet.txt` (outside the project, so it can never be committed). Copy table A into it now. Fill each value the moment the console shows it. The values below are those of the deployment of 19 October 2026.

A. Save these - they stay the same until you delete the resource

Value	Your value (19 Oct 2026)	Where to see it again
Region	`us-east-1` (N. Virginia)	top-right of every console page
AWS account ID	`169748358864`	top-right account menu; also inside every ECR image URI
Project folder	`...\IC App - AWS\issue-tracking-system`	File Explorer
Key file	`C:\Users\nlnu19\aws-keys\ic-key.pem`	File Explorer (download happens only once!)
VPC	`ic-vpc` = `vpc-0d7a543bc599a14f4`, 10.0.0.0/16	VPC -> Your VPCs
Subnets	public1 1a 10.0.0.0/20 (`subnet-06d3a5eefc60b1fbc`), public2 1b 10.0.16.0/20, private1 1a 10.0.128.0/20, private2 1b 10.0.144.0/20	VPC -> Subnets
ic-backend-sg	`sg-0065c0657e2fd6c17`	EC2 -> Security Groups
ic-gateway-public-sg	`sg-0decc7a07f80f9702`	EC2 -> Security Groups
Eureka private IP	`10.0.5.157`	EC2 -> ic-eureka -> Private IPv4; also the prompt `ip-10-0-5-157`
Gateway Elastic IP	`52.1.6.53` (private 10.0.27.246)	EC2 -> Elastic IPs
Private IPs of the services	user 10.0.4.115, project 10.0.20.49, issue 10.0.0.66	Eureka dashboard

Value	Your value (19 Oct 2026)	Where to see it again
RDS endpoint	`ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com` (port 3306, user `admin`)	RDS -> ic-mysql -> Connectivity & security -> Endpoints
Frontend bucket	`ic-frontend-nlnu19`	S3 -> Buckets
App address (Milestone 1)	`http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`	bucket -> Properties -> Static website hosting
API address (Milestone 1)	`http://52.1.6.53:8080/api`	built into the React files
Demo logins (not secret)	priya@its.com and karan@its.com / `Owner@123`; rahul@its.com and ananya@its.com / `Assignee@123`	DemoDataLoader.java

B. Do not rely on these - they change on every stop/start

Value	Example (19 Oct 2026)	How to get the current one
Public IP of ic-eureka	13.220.9.243	EC2 -> Instances -> select -> **Public IPv4 address** (copy icon)
Public IP of ic-user	3.239.150.112	same
Public IP of ic-project	18.212.15.96	same
Public IP of ic-issue	3.234.220.128	same
Public DNS names (`ec2-13-220-9-243...`)	follow the public IP	same page, Public IPv4 DNS
Your laptop's internet address	varies (proxy)	not needed - SSH is open to 0.0.0.0/0
ECS task IPs, builder public IP (Milestone 2)	-	ECS -> Tasks; EC2 -> ic-builder

C. Secrets - never in the project, Git, chat or screenshots

Secret	Where it may live
RDS master password (`admin`)	password manager, or your worksheet file in aws-keys
`user_svc`, `project_svc`, `issue_svc` passwords	same; they are also on each server in `/etc/issue-tracker/app.env` (root-only, chmod 600)
`photo_lambda` password	same; later in the Lambda's environment variables
`ic-key.pem`	only `C:\Users\nlnu19\aws-keys` - if lost, you cannot SSH to these servers again
Your copies of the two `.sql` files with real passwords	only aws-keys; delete them on the servers right after running them

WATCH OUT
Never show `sudo cat /etc/issue-tracker/app.env` during a screen share - it prints the database password. Section 6.19 has a masked version for the demo.

D. No need to save - look it up when needed

Instance IDs, AMI IDs, route-table and endpoint IDs, ARNs of roles. They never appear in a command of this guide.

6.2 Prepare the laptop (once)

1. **Region.** Sign in to the console and choose **United States (N. Virginia) us-east-1** top right. Check it on every page - resources in another region are invisible.
2. **JARs.** In STS, for each of the five projects: right-click -> Run As -> **Maven build...** -> Goals `clean package` -> Run. Each `target` folder then holds `<app>-1.0.0.jar`. Check in PowerShell, in the project folder: `dir .\*\target\*.jar` lists five files.
3. **React dependencies.** VS Code -> open `issue-tracker-ui` -> Terminal -> `npm ci` (once; it must end with 0 vulnerabilities and no error).
4. **Key folder.** Create `C:\Users\nlnu19\aws-keys`. The key and your worksheet live here.
5. **Terminal.** Windows Terminal with PowerShell (or cmd) - both have `ssh` and `scp` built in. Run the `.ps1` helper scripts from **PowerShell** in the project folder. Open a second tab (+) whenever a step says "in another window".

> **CONCEPT**
> Two folders, two jobs: commands that **copy project files** (deploy-jar, build-frontend, copy-to-builder, the Lambda folder) run in the **project folder**; commands that only need the key (ssh, the Eureka tunnel, copying the SQL files) run in **aws-keys**, where `.\ic-key.pem` is enough.

6.3 Key pair

1. EC2 -> **Key pairs** -> Create key pair -> name `ic-key`, type **RSA**, format **.pem** -> Create. The browser downloads `ic-key.pem` once; AWS keeps only the public half.
2. Move it to `C:\Users\nlnu19\aws-keys`.
3. Windows OpenSSH refuses a key that other users can read. In PowerShell, in aws-keys:

```
cd C:\Users\nlnu19\aws-keys
icacls .\ic-key.pem /inheritance:r
icacls .\ic-key.pem /grant:r "$($env:USERNAME):(R)"
```

6.4 Network: the VPC (wizard)

1. VPC console -> **Create VPC** -> **VPC and more**.
2. Name tag auto-generation **on**, value `ic`. IPv4 CIDR `10.0.0.0/16`. No IPv6. Tenancy Default.
3. Number of AZs **2**; public subnets **2**; private subnets **2** (default /20 sizes).
4. NAT gateways **None**. VPC endpoints **S3 Gateway**. DNS hostnames and DNS resolution both **ticked**.
5. **Create VPC** and wait until every step is green.

Subnet (wizard name)	AZ	CIDR	Used by
ic-subnet-public1-us-east-1a	us-east-1a	10.0.0.0/20	ic-eureka, ic-user, ic-issue (M2: ALB, tasks, builder)
ic-subnet-public2-us-east-1b	us-east-1b	10.0.16.0/20	ic-project, ic-gateway (M2: ALB, tasks)
ic-subnet-private1-us-east-1a	us-east-1a	10.0.128.0/20	RDS subnet group, Lambda
ic-subnet-private2-us-east-1b	us-east-1b	10.0.144.0/20	RDS subnet group, Lambda

> **CHECKPOINT**
> Your VPCs -> ic-vpc -> **Resource map**: 4 subnets, the public route table with the Internet Gateway, private route tables with the S3 endpoint.

6.5 IAM role for the EC2 servers

1. IAM -> Roles -> **Create role** -> Trusted entity **AWS service** -> Use case **EC2** -> Next.
2. Tick **CloudWatchAgentServerPolicy** -> Next.
3. Role name `ic-ec2-cloudwatch-role` -> **Create role**.

The role lets the CloudWatch agent on each server send logs and metrics without any access key. If the console ever says "not authorized ... iam:CreateRole", ask the trainer for an existing role.

6.6 Security groups (final, working rules)

EC2 -> Security Groups -> **Create security group**, VPC **ic-vpc** each time. Create them in this order (a rule can only point at a group that exists). To use a group as a source, choose **Custom** in the Source column and type `sg-` or the group name in the search box next to it.

Group	Inbound rules	Attached to
`ic-backend-sg` (sg-0065c0657e2fd6c17)	1) SSH, TCP 22, **Anywhere-IPv4 0.0.0.0/0**. Save. Then **Edit inbound rules** and add: 2) Custom TCP **8761**, source **ic-backend-sg**; 3) Custom TCP **8080-8083**, source **ic-backend-sg**	all five M1 servers
`ic-gateway-public-sg` (sg-0decc7a07f80f9702)	Custom TCP **8080**, Anywhere-IPv4 0.0.0.0/0	**ic-gateway only**
`ic-lambda-sg`	none (outbound only)	the Lambda (later)
`ic-rds-sg`	MySQL/Aurora **3306**, source **ic-backend-sg** (sg-0065c0657e2fd6c17); MySQL/Aurora 3306, source **ic-lambda-sg**	RDS (M2 adds ic-builder-sg and ic-ecs-sg)

WHY THESE RULES

A rule whose source is a **security group** means "any network card that is a member of that group" - not an IP range. When a server is replaced or gets a new address, the rule still works. The rule "8761 from ic-backend-sg" on ic-backend-sg itself means "the members of this group may talk to each other on Eureka's port".

SSH from 0.0.0.0/0 is acceptable here because login needs the private key `ic-key.pem` (Amazon Linux has password login switched off). In a real company you would use Session Manager or a VPN instead of an open port 22.

WATCH OUT

Check the **source column** after saving: it must show the `sg-...` id, not an IP range and not "My IP". The first ic-rds-sg rule had a wrong source and `mysql` hung.

6.7 RDS MySQL

a) DB subnet group

1. RDS -> **Subnet groups** -> Create DB subnet group.
2. Name `ic-db-subnet-group`, description "ic private subnets", VPC **ic-vpc**.
3. Availability Zones: tick **us-east-1a** and **us-east-1b**.
4. Subnets: tick exactly **10.0.128.0/20** (private1, 1a) and **10.0.144.0/20** (private2, 1b) -> Create.

WATCH OUT

"DB subnet group doesn't meet Availability Zone (AZ) coverage requirement" means both chosen subnets are in the same AZ (it happened with private2-1b + public2-1b). RDS needs subnets in at least two AZs even for a Single-AZ database, so it can move it later.

b) Create the database - the settings that the bootcamp account accepts

RDS -> Databases -> **Create database**. Only these fields change; leave the rest as proposed.

Section	Value
Creation method	**Full configuration** (Standard create)
Engine	**MySQL** (MySQL Community); version: the default (8.4.x was used)
Templates	**Sandbox** (on some consoles called Free tier) - **not** Dev/Test or Production
Availability	**Single-AZ DB instance**
DB instance identifier	`ic-mysql`
Master username / credentials	`admin`; **Self managed**; a strong password typed twice (letters, digits and #; avoid `$ \ " ' @ /` and spaces). Write it in your worksheet.
Instance class	Burstable -> **db.t3.micro**
Storage	gp2/gp3, the size the template proposes (20 GiB is plenty); untick **Enable storage autoscaling** if shown
Compute resource	**Don't connect to an EC2 compute resource**
VPC / subnet group	**ic-vpc** / **ic-db-subnet-group**
Public access	**No**
VPC security group	Choose existing -> **ic-rds-sg**; remove `default`
Availability Zone / port	No preference / 3306
Monitoring	Database Insights Standard; Performance Insights off; **Enhanced Monitoring off**
Additional configuration	Initial database name **empty** (the script creates three); backups as the template allows; encryption default; **Deletion protection off** (so you can clean up)

1. **Create database**. Status goes Creating -> Backing-up -> **Available** (about 10-15 minutes). You can launch the EC2 servers (6.8) meanwhile.
2. Open `ic-mysql` -> **Connectivity & security**. In "Connect using" choose **Endpoints** to see the endpoint with a copy button. Copy it into the worksheet:
   `ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com`.

WATCH OUT
"User ... is not authorized to perform: rds:CreateDBInstance ... because no identity-based policy allows" is **not** a network problem. The bootcamp role only allows certain settings. Go back, choose the Sandbox template and db.t3.micro, and create again. If that still fails, look at how batchmates' databases are configured or ask the trainer.

WATCH OUT
Do not use the console's ready-made `mysql ... --ssl-mode=VERIFY_IDENTITY` snippet. Our servers have the **MariaDB** client, which only understands `--ssl`.

CHECKPOINT
Status **Available**; Publicly accessible **No**; VPC security groups **ic-rds-sg (active)**; subnet group with the two private subnets.

6.8 The five EC2 application servers

Launch **ic-eureka first**, test SSH on it, then launch the other four. Each server gets the same first-boot script (`aws\ec2\user-data.sh`, Listing 5.4) with a different line 9.

a) Launch ic-eureka - every field

Section of "Launch an instance"	Value
Name and tags	`ic-eureka`
Application and OS Images	Quick Start -> **Amazon Linux** -> "Amazon Linux 2023 kernel-6.x AMI", **64-bit (x86)** (Free tier eligible)
Instance type	**t3.micro**
Key pair	**ic-key**
Network settings -> **Edit**	VPC **ic-vpc**; Subnet **ic-subnet-public1-us-east-1a**; Auto-assign public IP **Enable**
Firewall (security groups)	**Select existing security group** -> **ic-backend-sg** only (the Summary on the right must say "1 security group")
Configure storage	1 x **8 GiB gp3**; File systems: None
Advanced details -> IAM instance profile	**ic-ec2-cloudwatch-role**
Advanced details -> (everything else)	leave the defaults: Credit specification Unlimited, Metadata version V2 only, hop limit 2
Advanced details -> User data	open `aws\ec2\user-data.sh` in VS Code, change line 9 to `SERVICE=eureka-server`, select all, copy, paste here. Undo the change in VS Code (do not save it).

1. Before clicking **Launch instance**, scroll to the end of the User data box: the last two lines must be `systemctl daemon-reload` and `systemctl enable issue-tracker` (the whole 65-line script was pasted). The `=====` comment lines may look split - that is only the box wrapping long lines.
2. Launch. Wait until **Instance state Running** and **Status check 3/3 checks passed**, then about 3 more minutes for the script (it installs Java, the MySQL client and the CloudWatch agent).

b) First SSH test

```
cd C:\Users\nlnu19\aws-keys
ssh -i .\ic-key.pem ec2-user@<EUREKA-PUBLIC-IP>
#   first time only: "Are you sure you want to continue connecting (yes/no)?" -> type yes
java -version                                 # openjdk "21..." Corretto
systemctl status issue-tracker                # enabled; inactive (dead) = expected, no JAR yet
sudo systemctl status amazon-cloudwatch-agent  # active (running)
sudo tail -n 5 /var/log/cloud-init-output.log  # "Cloud-init ... finished"
exit
```

CONCEPT
The prompt `[ec2-user@ip-10-0-5-157 ~]$` contains the server's **private IP** (10.0.5.157). Write Eureka's into the worksheet - every other service needs it.

WATCH OUT
`Connection timed out` = a firewall dropped the packets. Check: the instance has a public IP; ic-backend-sg has SSH from 0.0.0.0/0; you typed the address without `< >`. `Permission denied (publickey)` = wrong key or user (must be `ec2-user`). `UNPROTECTED PRIVATE KEY FILE` = run the two `icacls` lines of 6.3.

c) Deploy Eureka straight away

Eureka needs no database and its `app.env` stays empty (the first-boot script created the file). In PowerShell, in the **project folder**:

```
cd "C:\Users\nlnu19\Documents\workspace-spring-tools-for-eclipse-5.4.0.RELEASE\IC App - AWS\issue-tracking-system"
dir .\eureka-server\target\eureka-server-1.0.0.jar
powershell -ExecutionPolicy Bypass -File .\aws\scripts\deploy-jar.ps1 -App eureka-server -Server <EUREKA-PUBLIC-IP>
ssh -i C:\Users\nlnu19\aws-keys\ic-key.pem ec2-user@<EUREKA-PUBLIC-IP> "tail -n 20 /var/log/issue-tracker/app.log"
```

CHECKPOINT
The script ends with `Active: active (running)`. The log shows `Started EurekaServerApplication` and then, every 10 seconds, `Running the evict task` - Eureka's normal heartbeat while no service is registered yet.

d) Launch the other four

Exactly as ic-eureka; only these fields change:

Name	Line 9 of user data	Subnet	Security groups
ic-user	`SERVICE=user-service`	public1 (1a)	ic-backend-sg
ic-project	`SERVICE=project-service`	public2 (1b)	ic-backend-sg
ic-issue	`SERVICE=issue-service`	public1 (1a)	ic-backend-sg
ic-gateway	`SERVICE=api-gateway`	public2 (1b)	ic-backend-sg **and** ic-gateway-public-sg

e) Elastic IP for the gateway

1. EC2 -> **Elastic IPs** -> Allocate Elastic IP address -> Allocate.
2. Select it -> Actions -> **Associate Elastic IP address** -> Resource type Instance -> **ic-gateway** -> Associate. Worksheet: gateway Elastic IP **52.1.6.53**.

WHY IT IS DONE THIS WAY
The gateway's address is compiled into the React files. A normal public IP changes on every stop/start; an Elastic IP stays with your account and moves with the instance, so the S3 site keeps working after a restart.

CHECKPOINT
Five instances **Running**, **3/3** status checks; ic-gateway shows the Elastic IP as its public IPv4 address.

6.9 Create the three schemas and the three database users

RDS has no public address, so the SQL runs **from ic-gateway** (a member of ic-backend-sg, which ic-rds-sg accepts). The MySQL client was installed by the first-boot script.

1. Copy the script to aws-keys (PowerShell, in the project folder):

```
copy .\aws\rds\01-schemas-and-users.sql C:\Users\nlnu19\aws-keys\
```

2. Open the **copy** in aws-keys with VS Code or Notepad. Replace the three `CHANGE_ME_...` passwords (letters, digits and `#` only). Save. Write the three passwords in your worksheet - the `app.env` files need them.
3. Copy it to the gateway and run it:

```
cd C:\Users\nlnu19\aws-keys
scp -i .\ic-key.pem .\01-schemas-and-users.sql ec2-user@52.1.6.53:~
ssh -i .\ic-key.pem ec2-user@52.1.6.53
ls                                  # app  01-schemas-and-users.sql
mysql -h ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com -u admin -p --ssl < 01-schemas-and-users.sql
rm 01-schemas-and-users.sql         # it contains passwords
exit
```

WATCH OUT

The `:~` at the end of the scp line is essential. Without it scp copies the file locally to a file named `ec2-user@...` ("1 file(s) copied."). If that happens, delete the stray file (it holds passwords): `del ".\ec2-user@ec2-52-1-6-53.compute-1.amazonaws.com"`.

What you see after typing the admin password	Meaning	Fix
the output below within 1-2 seconds	success	-
`ERROR 1045 ... Access denied for user 'admin'`	wrong master password	type it again; or RDS -> Modify -> New master password -> Apply immediately
nothing happens; after ~2 minutes `ERROR 2002 ... Can't connect`	packets dropped: ic-rds-sg does not allow ic-backend-sg (or RDS is not in ic-vpc)	Ctrl+C; fix the ic-rds-sg source (6.6); Up arrow and run again
`Unknown MySQL server host`	typo in the endpoint	copy it again from the console

```
Database                 user         host
information_schema       issue_svc    10.0.%
issue_db                 project_svc  10.0.%
mysql                    user_svc     10.0.%
performance_schema
project_db
sys
user_db
```

CONCEPT

The `mysql` password prompt appears **before** any connection is made. A wrong password therefore fails instantly, while a network block makes it hang - that is how you tell the two apart. `10.0.%` means the users may connect only from inside ic-vpc.

TRY IT

DBeaver through an SSH tunnel (the class used a public RDS instead): New connection -> MySQL -> Server host = the RDS endpoint, port 3306, user admin. **SSH** tab: Use SSH Tunnel, host 52.1.6.53, user ec2-user, authentication Public key = ic-key.pem. Driver properties: `sslMode=REQUIRED`. DBeaver connects to the gateway by SSH and from there to RDS - the database never needs a public address. Good to show in the interview.

6.10 Deploy user, project, issue and gateway

Order: **user-service -> project-service -> issue-service -> api-gateway** (the demo data of project and issue refer to user ids 1-4). Wait for "Started" in each log before the next.

a) Prepare the four app.env texts in Notepad

Open Notepad (do not save the file in the project). Type the four blocks once; later you copy a block and paste it on the server with a **right-click**.

```
# ---- ic-user
SPRING_PROFILES_ACTIVE=aws
EUREKA_URL=http://10.0.5.157:8761/eureka/
DB_HOST=ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com
DB_USERNAME=user_svc
DB_PASSWORD=<user_svc password>

# ---- ic-project: the same five lines with DB_USERNAME=project_svc and its password
# ---- ic-issue:   the same five lines with DB_USERNAME=issue_svc and its password

# ---- ic-gateway (no DB lines; the localhost origin is replaced in 6.12)
SPRING_PROFILES_ACTIVE=aws
EUREKA_URL=http://10.0.5.157:8761/eureka/
FRONTEND_ORIGIN=http://localhost:5173
```

WHY IT IS DONE THIS WAY

`/etc/issue-tracker/app.env` is read by systemd (`EnvironmentFile=`) and handed to Java as environment variables. The `aws` profile (`application-aws.properties`) turns them into Spring settings: `${EUREKA_URL}`, `${DB_HOST}`... The same JAR therefore runs on the laptop (default profile), on EC2 and in ECS - only the environment differs.

b) For each of the four servers

```
# 1. write the settings on the server   (aws-keys folder)
ssh -i .\ic-key.pem ec2-user@<SERVER-PUBLIC-IP>
sudo nano /etc/issue-tracker/app.env     # right-click = paste;  Ctrl+O, Enter = save;  Ctrl+X = exit
exit

# 2. copy the JAR and (re)start the service   (project folder, PowerShell)
powershell -ExecutionPolicy Bypass -File .\aws\scripts\deploy-jar.ps1 -App user-service -Server <IP>
#    -App: user-service | project-service | issue-service | api-gateway
#    -Server for the gateway: its Elastic IP 52.1.6.53

# 3. after about one minute, read the end of the log
ssh -i C:\Users\nlnu19\aws-keys\ic-key.pem ec2-user@<IP> "tail -n 30 /var/log/issue-tracker/app.log"
```

Log line	Meaning
`Demo users loaded (2 project owners, 2 assignees)`	user-service created its tables in user_db and inserted the demo data (only when the table is empty)
`Registering application PROJECT-SERVICE with eureka with status UP`	the service found Eureka at 10.0.5.157
`registration status: 204`	Eureka accepted the registration
`Started ProjectServiceApplication in 15.893 seconds`	start-up finished
`Demo projects loaded (3 projects)` / `Demo issues loaded (9 issues)`	demo data of the other services
`Discovery Client initialized ... initial instances count: 3`	(gateway) it already sees the three services
`Netty started on port 8080` / `Started ApiGatewayApplication`	the gateway is up
`Resolving eureka endpoints via configuration` (every 5 min)	normal

Error in the log	Cause	Fix
`Access denied for user 'user_svc'@'10.0...'`	password in app.env differs from the SQL copy	`sudo nano` the file again; `sudo systemctl restart issue-tracker`
`Communications link failure`	server cannot reach RDS	server must have ic-backend-sg; ic-rds-sg rule
`Could not resolve placeholder 'DB_HOST'` (or EUREKA_URL...)	line missing or misspelt in app.env	compare with the Notepad block
`Connection refused` to 10.0.5.157:8761 (repeating)	Eureka not running	start ic-eureka / its service first

6.11 Verify the backend

a) The Eureka dashboard through an SSH tunnel

Port 8761 is private. In a **new terminal tab** (aws-keys):

```
ssh -i .\ic-key.pem -L 8761:10.0.5.157:8761 ec2-user@52.1.6.53
```

Leave that window open (closing it, or typing `exit`, closes the tunnel). In the browser open `http://localhost:8761`.

> **CHECKPOINT**
> "Instances currently registered with Eureka": **API-GATEWAY** 10.0.27.246:8080, **ISSUE-SERVICE** 10.0.0.66:8083, **PROJECT-SERVICE** 10.0.20.49:8082, **USER-SERVICE** 10.0.4.115:8081 - all **UP (1)**. The red text "THE SELF PRESERVATION MODE IS TURNED OFF" is expected: we switched it off so stopped services disappear quickly.

> **CONCEPT**
> `-L 8761:10.0.5.157:8761` = "listen on port 8761 of my laptop and forward every connection through the SSH session to 10.0.5.157:8761, as seen from the gateway". The gateway is inside the VPC, so it can reach Eureka's private address.

b) The API through the gateway

Browser: `http://52.1.6.53:8080/api/users` - the four demo users as JSON (`"profile":null` because they have no photo). Optional login test in PowerShell:

```
Invoke-RestMethod -Method Post -ContentType "application/json" `
  -Uri http://52.1.6.53:8080/api/users/login `
  -Body '{"email":"priya@its.com","password":"Owner@123"}'
# answer: userId 1, username priya, role PROJECT_OWNER
```

6.12 The React frontend on S3

a) Build with the gateway address

```
cd "C:\Users\nlnu19\Documents\workspace-spring-tools-for-eclipse-5.4.0.RELEASE\IC App - AWS\issue-tracking-system"
powershell -ExecutionPolicy Bypass -File .\aws\scripts\build-frontend.ps1 -ApiUrl http://52.1.6.53:8080/api
```

It prints `Building the React app with VITE_API_BASE_URL=http://52.1.6.53:8080/api`, then `built in ...s` and the files in `issue-tracker-ui\dist`: `index.html`, `favicon.svg` and the `assets` folder (one .js, one .css, two icon fonts).

CONCEPT
Vite **replaces** `import.meta.env.VITE_API_BASE_URL` with the text `http://52.1.6.53:8080/api` while building. The files on S3 are plain static files - there is no server that could read a setting later. A new API address therefore means a new build and a new upload.

b) The bucket

1. S3 -> **Create bucket** -> General purpose -> name `ic-frontend-nlnu19` (bucket names are global; add a digit if taken) -> region us-east-1.
2. Object Ownership **ACLs disabled**. **Untick "Block all public access"** and tick the acknowledgement. Versioning **Enable**. Encryption SSE-S3 -> **Create bucket**.
3. Open the bucket -> **Upload** -> **Add files**: `index.html` and `favicon.svg` from `dist` -> **Add folder**: `dist\assets` -> **Upload**. `index.html` must be at the top level of the bucket, next to the `assets/` folder - not inside a `dist/` folder.
4. **Permissions** -> Bucket policy -> Edit -> paste Listing 6.1 -> Save.
5. **Properties** -> Static website hosting -> Edit -> **Enable** -> Host a static website -> Index document `index.html` -> Error document `index.html` -> Save. Copy the **Bucket website endpoint**: `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`.

Listing 6.1 - bucket policy of ic-frontend-nlnu19

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadWebsite",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::ic-frontend-nlnu19/*"
  }]
}
```

WHY IT IS DONE THIS WAY
The error document is `index.html` because React Router handles paths such as `/owner/dashboard` in the browser. S3 has no such file; returning index.html lets the app load and show the right page after F5.

c) Allow the website in the gateway (CORS)

```
ssh -i .\ic-key.pem ec2-user@52.1.6.53
sudo nano /etc/issue-tracker/app.env
#   change only the last line to (no trailing /):
#   FRONTEND_ORIGIN=http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com
sudo systemctl restart issue-tracker
exit
```

CHECKPOINT
Open `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`: the **InfinityCastle - Issue Tracker** login page. Log in as priya@its.com / Owner@123, role Project Owner: "Good morning, Priya", the board with Customer Portal and the demo issues. F5 on `/owner/dashboard` reloads the page. F12 -> Console: no CORS error.

CONCEPT
The browser shows "Not secure" because S3 website endpoints and port 8080 are plain HTTP. HTTPS needs CloudFront (for S3) and a certificate on a load balancer - see Part 10, "How would you add HTTPS?".

6.13 Milestone 1 status and evidence

Case-study task	Status (20 Oct 2026)	Evidence to keep
React on S3 static website	done	screenshot of the login page and the dashboard with the S3 address bar
One EC2 per microservice (5)	done	EC2 -> Instances, five Running; Eureka dashboard
RDS MySQL, private	done	RDS summary (Available, Publicly accessible No)
Services connected to RDS	done	`/api/users` JSON; logs with "Demo users loaded"
Lambda triggered by S3	theory only: designed and coded, not deployed (6.14, 3.14)	the code (Listing 5.8) and the design
CloudWatch logs/metrics	agent configured by user data; automatic metrics; dashboard and alarms as theory (6.15)	log groups
Testing	basic checks done (6.11); full table kept as the test plan (6.16)	screenshots of the checks

Take the screenshots with **Win+Shift+S** (sharper than phone photos) and keep them in a folder outside the project. Crop out anything that shows a password.

6.14 Lambda: the profile-photo processor (designed - not deployed)

> **STATUS**
> Decided on 20 October: the Lambda stays **theory**. The function is written and tested locally against sample events (Listing 5.8), and everything needed to deploy it is below - but it was not created in the account. In the interview, present it as a complete design: the theory is in 3.14, the code in 5.6. If it is ever deployed, these are the exact steps; ic-gateway must be running for b) and c).

Because the Lambda would write to the shared RDS, a photo would appear in both the Milestone 1 and the Milestone 2 site without any change to the Spring Boot services.

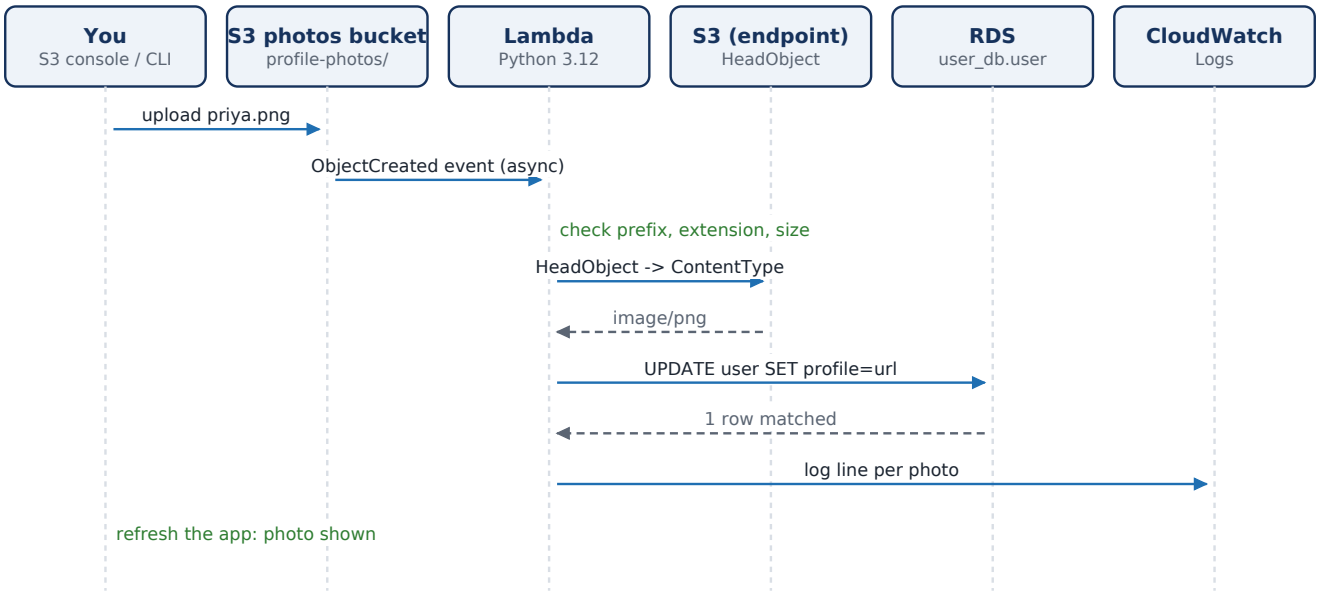


Figure 6.1 - Uploading `profile-photos/priya.png` sets Priya's photo in the app, without any change to the Spring Boot services.

0) One security-group check

ic-rds-sg must have **MySQL 3306 from ic-lambda-sg** (6.6). Add it now if it is missing.

a) The photos bucket

1. S3 -> Create bucket `ic-photos-nlnu19` (us-east-1, ACLs disabled, **untick Block all public access**, versioning on).
2. Permissions -> Bucket policy -> Listing 6.2 (public **read** of `profile-photos/*` only - the browser loads the photos directly from S3).
3. Create folder `profile-photos` (Create folder button).

Listing 6.2 - bucket policy of ic-photos-nlnu19

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadProfilePhotos",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::ic-photos-nlnu19/profile-photos/*"
  }]
}
```

b) Build the deployment package on ic-gateway (it has internet access)

```
# laptop, PowerShell, PROJECT folder: copy the Lambda folder to the gateway
scp -i C:\Users\nlnu19\aws-keys\ic-key.pem -r `
    .\aws\lambda\profile-photo-processor ec2-user@52.1.6.53:~/photo-lambda

# on the gateway: code + PyMySQL + the RDS certificate bundle, all at the root of the zip
ssh -i C:\Users\nlnu19\aws-keys\ic-key.pem ec2-user@52.1.6.53
cd ~/photo-lambda && bash package-lambda.sh
exit

# laptop, aws-keys folder: fetch the zip
cd C:\Users\nlnu19\aws-keys
scp -i .\ic-key.pem ec2-user@52.1.6.53:~/photo-lambda/photo-lambda.zip .
```

WATCH OUT

If the script stops with `$'\r': command not found`, the file got Windows line endings on the way. On the gateway: `sed -i 's/\r$//' package-lambda.sh` and run it again.

c) The database user for the Lambda

```
copy .\aws\rds\02-lambda-user.sql C:\Users\nlnu19\aws-keys\        # project folder
```

1. Set the password in the **copy** in aws-keys; write it in the worksheet.
2. Same three commands as 6.9 with this file name: `scp ... ec2-user@52.1.6.53:~` (with `:~`), `mysql -h ... -u admin -p --ssl < 02-lambda-user.sql`, `rm 02-lambda-user.sql`.
3. The output shows `GRANT SELECT (username), UPDATE (profile) ON user_db.user TO photo_lambda@10.0.%`.

d) The execution role

1. IAM -> Roles -> Create role -> AWS service -> **Lambda** -> attach **AWSLambdaVPCAccessExecutionRole** -> name `ic-photo-lambda-role` -> Create.
2. Open the role -> Add permissions -> **Create inline policy** -> JSON -> Listing 6.3 -> name `ic-read-profile-photos` -> Create.

Listing 6.3 - inline policy of ic-photo-lambda-role

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "ReadProfilePhotoMetadata",
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::ic-photos-nlnu19/profile-photos/*"
  }]
}
```

e) Create the function

1. Lambda -> **Create function** -> Author from scratch -> name `ic-profile-photo-processor` -> Runtime **Python 3.12** -> Architecture x86_64.
2. Permissions -> Change default execution role -> **Use an existing role** -> ic-photo-lambda-role.
3. Additional configurations -> **Enable VPC** -> VPC ic-vpc -> the two **private** subnets -> security group **ic-lambda-sg** -> Create function.
4. Code -> **Upload from -> .zip file** -> `photo-lambda.zip` from aws-keys -> Save. (Handler stays `lambda_function.lambda_handler`.)
5. Configuration -> General configuration -> Edit -> Timeout **15 sec**, memory 128 MB -> Save.
6. Configuration -> Environment variables -> Edit: `DB_HOST` = `ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com`, `DB_NAME` = `user_db`, `DB_USER` = `photo_lambda`, `DB_PASSWORD` = its password -> Save.

f) The S3 trigger

1. S3 -> ic-photos-nlnu19 -> Properties -> **Event notifications** -> Create event notification.
2. Name `ic-photo-uploaded`; Prefix `profile-photos/`; Event types **All object create events**; Destination **Lambda function** -> ic-profile-photo-processor -> Save. (S3 adds the permission to invoke the function.)

g) Test it

1. Rename any small PNG to `priya.png` and upload it into `profile-photos/` of ic-photos-nlnu19.
2. CloudWatch -> Log groups -> `/aws/lambda/ic-profile-photo-processor` -> newest stream: "Profile photo of priya set to https://ic-photos-nlnu19.s3.us-east-1.amazonaws.com/profile-photos/priya.png".
3. In the app, log out and in as Priya (or Ctrl+F5): the sidebar shows the photo instead of "PS".
4. Negative tests: upload `notes.txt` -> "Skipped ... .txt is not an accepted image type"; upload `nobody.png` -> "no user with username 'nobody'". Nothing in the database changes.

Lambda log shows	Cause
time-out on `head_object`	the S3 gateway endpoint is not on the private route tables (VPC -> Endpoints -> Route tables)
time-out on `connect`	ic-rds-sg does not allow ic-lambda-sg (step 0)
`Access denied for user 'photo_lambda'`	password differs between c) and the environment variable
`No module named 'pymysql'`	the zip was built wrongly - repeat b)

6.15 CloudWatch dashboards and alarms (designed)

What exists without any extra work: the agent configured by user data on the five servers (`/issue-tracker/m1/<app>` log groups, memory and disk metrics), the `/ecs/ic-<app>` groups of Milestone 2, Container Insights, and the automatic metrics of EC2, RDS, ECS and the ALB. The dashboard and alarms below are the **design** - the steps to build them in about 20 minutes (theory in 3.13).

1. **Logs:** CloudWatch -> Log groups: `/issue-tracker/m1/<app>` (one per server, made by the agent), `/ecs/ic-<app>` (Milestone 2), `/aws/lambda/ic-profile-photo-processor`. For each: Actions -> **Edit retention setting** -> 14 days (default is "never expire").
2. **Logs Insights:** select the five `/issue-tracker/m1/...` groups (and the `/ecs/...` ones) and run the query below to see every error across the services in one list.
3. **Dashboard** `ic-dashboard`: line widgets for EC2 CPUUtilization (5 instances), IssueTracker/EC2 mem_used_percent, Lambda Invocations + Errors, RDS DatabaseConnections + CPUUtilization, ECS service CPU, ALB RequestCount.
4. **Alarms** (optional, quick): SNS topic `ic-alerts` with your e-mail (confirm the mail). Then `ic-gateway-cpu-high` (CPUUtilization of ic-gateway > 70 for 2 x 5 min), `ic-photo-lambda-errors` (Errors sum >= 1 in 5 min), `ic-rds-storage-low` (FreeStorageSpace < 2 GB), each notifying ic-alerts.

```
fields @timestamp, @logStream, @message
| filter @message like /ERROR|Exception/
| sort @timestamp desc
| limit 50
```

> **CHECKPOINT**
> Each service has a log group with recent lines; the dashboard shows data in every widget; alarms are OK (or "Insufficient data" at first).

6.16 Test plan (Milestone 1, task F)

The basic checks of 6.11 were done during the deployment. This table is the full end-to-end test plan to run (and screenshot) before or during the demo; tests 6 and 10 need the Lambda and the dashboard of 6.14-6.15.

#	Test	Expected
1	Open `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com`	login page; favicon "IC"
2	Sign up a new assignee without a photo, log in	success message; initials in the sidebar
3	Owner: board, search `^add`, filters, project switch, F5	same behaviour as on the laptop
4	Create a project and an issue; edit it	saved; still there after refresh (stored in RDS)
5	Assignee (rahul@its.com / Assignee@123): change a status	owner's board shows the new column
6	Upload `<username>.png` to the photos bucket (after 6.14)	photo appears for that user
7	GET `http://52.1.6.53:8080/api/issues/999`	404 with "Issue not found with id 999"
8	`sudo systemctl stop issue-tracker` on ic-issue, reload the board	friendly "service not available" error + Try again
9	`sudo systemctl start issue-tracker`, wait 30 s, press Try again	board loads (Eureka re-registered it)
10	CloudWatch Logs Insights (after 6.15)	the errors of test 8 are listed
11	EC2 -> ic-user -> Instance state -> **Reboot**	after ~1 min the service is back by itself (systemd, enabled)

6.17 What persists and what changes

The interview may be days after the deployment, and the bootcamp account may stop resources in between. This table tells you, for every event, what survives and what you must redo.

Event	Stays the same	Changes / is lost	What you must do
Reboot an EC2	everything, including the public IP	nothing (the app restarts)	wait 1-2 min; the service is `enabled`, systemd starts it at boot
Stop, then Start an EC2	instance ID, **private IP**, disk (JAR, `app.env`, systemd service, logs, agent config), security groups, role, the **Elastic IP** of ic-gateway	**public IPv4 and public DNS** of ic-eureka, ic-user, ic-project, ic-issue; SSH asks "yes/no" again for the new address	copy the new public IPs into worksheet table B (only needed for SSH and deploy-jar). Nothing in app.env changes - it uses private IPs and the RDS endpoint
Terminate an EC2	nothing of that server	its disk, its private IP; a new server gets new IPs	relaunch it (6.8 a/d), write app.env, deploy the JAR. If it was **ic-eureka**: put its new private IP in EUREKA_URL of all four app.env files and restart them. If it was ic-gateway: associate the Elastic IP again
Stop RDS temporarily	endpoint, schemas, users, data	AWS starts it again by itself after **7 days**	start it before the EC2s and wait for Available (5-10 min)
Delete and recreate RDS with the **same identifier**	endpoint (it is built from identifier + account + region)	all data, users and schemas	run 01 and 02 SQL again; restart the services (they create the tables and demo data)
Elastic IP	the address, while allocated; stays associated over stop/start	released = gone for ever; a new one has another address	never release it before the interview. A new address means a new React build + upload (6.12)
S3 buckets and website	everything; always on	-	nothing
VPC, subnets, SGs, roles, key pair	everything	-	nothing
Data in RDS	all rows, including what you created during tests	-	demo data is inserted only into empty tables
Eureka registry	-	it is in memory: empty after Eureka restarts	services re-register by themselves within ~30 s
A new day / another network on the laptop	SSH rule (0.0.0.0/0)	the SSH tunnel and console login end; the **office network blocks port 8080** (and may block 22)	open the tunnel again; sign in again; for Milestone 1 use a **personal network or phone hotspot**
CloudWatch logs	kept (until the retention you set)	-	nothing
ECS service desired tasks 1 -> 0 -> 1 (M2)	task definitions, ALB and its DNS name, Cloud Map name `eureka.ic.local`, ECR images	every task gets a new IP	nothing - tasks re-register in Eureka; `eureka.ic.local` points to the new Eureka task (details: 7.14)
ic-builder stop/start (M2)	disk with images and files	public IP	use the new IP for SSH and copy-to-builder

WATCH OUT

Ask the trainer whether the bootcamp account **stops or deletes** resources automatically (at night, at the end of a session, or after some days). Stopped is harmless - follow 6.19. Deleted means rebuilding with this part, which is why the worksheet matters.

6.18 Pausing between now and the interview

Option	How	Approximate cost per day	Ready for the demo in
Keep everything running	do nothing	5 x t3.micro ~$1.25 + 5 public IPv4 ~$0.60 + RDS ~$0.41 = ~$2.3	immediately
Stop the five EC2s (recommended)	EC2 -> Instances -> select ic-eureka, ic-user, ic-project, ic-issue, ic-gateway -> Instance state -> **Stop instance**	disks ~$0.10 + Elastic IP ~$0.12 + RDS ~$0.41	~10 minutes (6.19)
Also stop RDS	RDS -> ic-mysql -> Actions -> **Stop temporarily** (if the account allows it)	adds only storage ~$0.08 instead of ~$0.41	~20 minutes; restarts by itself after 7 days

WATCH OUT
Never, before the interview: **terminate** an instance, **release** the Elastic IP, **delete** RDS, the key pair or the buckets. Each of these forces a partial rebuild.

6.19 Demo-day runbook: bring Milestone 1 back and show it

a) Start-up (30 minutes before)

WATCH OUT
Network first. The office network lets only standard web ports through: the app on S3 loads, but every call to `http://52.1.6.53:8080` fails with "Cannot reach the server" (seen on 20 Oct 2026). Demonstrate Milestone 1 from a **personal network or phone hotspot**. Milestone 2 uses the load balancer on port 80, which office networks normally allow.

1. Sign in to the console; region **us-east-1**.
2. **RDS first.** RDS -> Databases -> ic-mysql. If the status is *Stopped*: Actions -> **Start** and wait for **Available** (5-10 min). The services cannot start without it.
3. **Eureka next.** EC2 -> Instances -> ic-eureka -> Instance state -> **Start instance**. Wait for 3/3 checks (about 2 min).
4. **The other four together:** select ic-user, ic-project, ic-issue, ic-gateway -> Start instance. Wait for 3/3, then about 2 more minutes - systemd starts every application by itself.
5. **Worksheet table B:** copy the new public IPs of ic-eureka, ic-user, ic-project and ic-issue. ic-gateway keeps 52.1.6.53.
6. **Quick check (browser):** `http://52.1.6.53:8080/api/users` returns the four users. A 503 or "connection refused" in the first minutes means "not registered yet" - wait one more minute.
7. **Eureka tunnel** in a new terminal tab (leave it open), then `http://localhost:8761` in the browser shows the four services UP:

```
cd C:\Users\nlnu19\aws-keys
ssh -i .\ic-key.pem -L 8761:10.0.5.157:8761 ec2-user@52.1.6.53
```

8. **The app:** open `http://ic-frontend-nlnu19.s3-website-us-east-1.amazonaws.com` and log in as Priya. Log out again so the demo starts at the login page.

b) If something is missing

```
ssh -i .\ic-key.pem ec2-user@<PUBLIC-IP-OF-THAT-SERVER>
systemctl status issue-tracker                        # active (running)?
tail -n 40 /var/log/issue-tracker/app.log             # the reason, see the table in 6.10
sudo systemctl restart issue-tracker                  # start it again (e.g. after RDS became available)
```

If a service started while RDS was still starting, it failed and systemd retries it every 10 seconds (`Restart=on-failure`) - it usually comes up alone once RDS is Available; a manual restart is quicker.

c) What to show, in this order

#	Show	Say (in your own words)
1	Figure 1.1 (architecture)	S3 serves the React files; the browser calls the gateway on its Elastic IP; the gateway finds the services through Eureka; one private RDS with a schema and a user per service.
2	VPC -> ic-vpc -> Resource map	two public subnets with a route to the Internet Gateway, two private subnets without it, S3 gateway endpoint, two AZs.
3	Security groups: ic-rds-sg, ic-backend-sg	rules reference groups, not IPs; only the gateway's port 8080 is public; RDS accepts only the app servers.
4	RDS -> ic-mysql	Publicly accessible No; endpoint; private subnet group in two AZs; the services connect over TLS (`sslMode=REQUIRED`) with their own limited users.
5	EC2 -> Instances, Elastic IPs	five t3.micro, spread over two AZs; the gateway's fixed address.
6	The app: log in, board, create an issue, F5	data is in RDS; F5 works thanks to the error document.
7	Eureka dashboard via the tunnel	services registered with private IPs (the `aws` profile); Eureka itself is not reachable from the internet.
8	SSH to ic-user: `systemctl cat issue-tracker`, masked app.env, `tail -f` the log while you click in the app	systemd keeps the app running and restarts it; configuration comes from environment variables, one JAR for every environment.
9	Stop issue-service, reload the board, start it, Try again	friendly error, recovery through Eureka re-registration.
10	CloudWatch log groups (`/issue-tracker/m1/<app>`), Logs Insights query; then the Lambda design (Figure 3.33)	central logs; the event-driven serverless step as a design: trigger, role, VPC, idempotency.

Masked view of the settings for step 8 (never `cat` the file on a shared screen):

```
sudo sed 's/PASSWORD=.*/PASSWORD=********/' /etc/issue-tracker/app.env
```

d) After the demo

Pause again (6.18) if more demos may follow, or clean up everything (Part 9.3).

6.20 Problems met during the real deployment

What you saw	Cause	Fix
"DB subnet group doesn't meet Availability Zone (AZ) coverage requirement"	both subnets in us-east-1b	private1 (1a) + private2 (1b)
"not authorized to perform: rds:CreateDBInstance ... no identity-based policy allows"	bootcamp policy rejects the Dev/Test settings (and 200 GiB)	Sandbox template, db.t3.micro
`ssh: connect to host ... port 22: Connection timed out`	SSH rule "My IP" does not match the company network's exit address	SSH 22 from 0.0.0.0/0
`Could not resolve hostname <13.220.9.243>`	the `< >` of the placeholder were typed	type only the address
`scp` printed "1 file(s) copied."	destination without `:~`, so a local copy was made	add `:~`; delete the stray file
`-bash: 01-schemas-and-users.sql: No such file or directory`	consequence of the previous line	copy again with `:~`
`mysql` waits forever after the password	ic-rds-sg did not allow ic-backend-sg	rule MySQL 3306 from sg-0065c0657e2fd6c17
Eureka log: `Running the evict task` every 10 s	normal heartbeat	none

What you saw	Cause	Fix
Eureka page: "SELF PRESERVATION MODE IS TURNED OFF"	set on purpose in eureka-server	none
Gateway does not start with the aws profile	`FRONTEND_ORIGIN` not set yet	temporary `http://localhost:5173`, real value in 6.12
Browser: "Not secure"	plain HTTP (S3 website, port 8080)	expected; HTTPS = CloudFront + ACM
At the office: login page says "Cannot reach the server (API Gateway on port 8080)"	the office firewall blocks port 8080; nothing was wrong in AWS	personal network / hotspot (M2: ALB on port 80)
ic-eureka had two security groups in the launch summary	ic-gateway-public-sg selected by mistake	remove it before launching

PART 7

Milestone 2 - containers on ECR and ECS Fargate, step by step

The record of the real deployment of 19-20 October 2026: every step in the order that finally worked, every value, each error message and its fix, what to write down, and how to pause and bring the system back for the demo. The theory behind each step is in Part 3 (3.8-3.13); the code changes it relies on are in Part 5.

> **ABOUT THE VALUES IN THIS PART**
> This part contains addresses, never passwords. Wherever a password is needed it is typed into the console or a hidden `read -s` prompt.

7.0 What changes from Milestone 1

Aspect	Milestone 1	Milestone 2
Packaging	JAR copied to a server	JAR inside a Docker image, stored in ECR
Where it runs	five EC2 instances you manage	five ECS services on Fargate - no servers
Starting / restarting	systemd (`Restart=on-failure`)	the ECS service scheduler (desired count)
Configuration	`/etc/issue-tracker/app.env`	environment variables in task definitions
Finding Eureka	its private IP 10.0.5.157	the name `eureka.ic.local` (Service Connect)
Entry point for the browser	Elastic IP 52.1.6.53, port 8080	ALB DNS name, port 80
Security groups	`ic-backend-sg`, `ic-gateway-public-sg`	`ic-alb-sg`, `ic-ecs-sg`
Logs	CloudWatch agent ships app.log	awslogs driver -> `/ecs/ic-<app>`
Frontend	bucket `ic-frontend-nlnu19`	bucket `ic-frontend-m2-nlnu19` (own build)
Database	RDS ic-mysql	**the same** instance, schemas and users

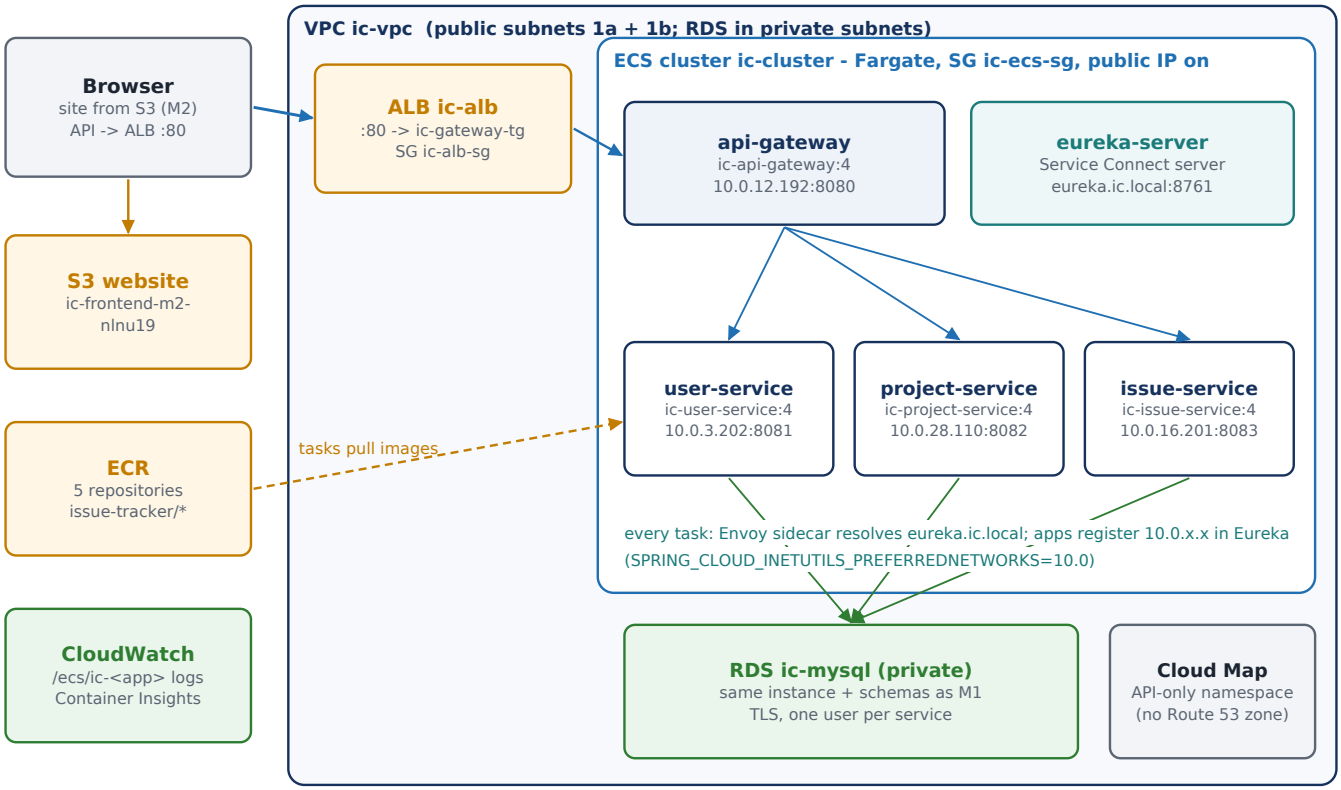


Figure 7.1 - Milestone 2 as deployed.

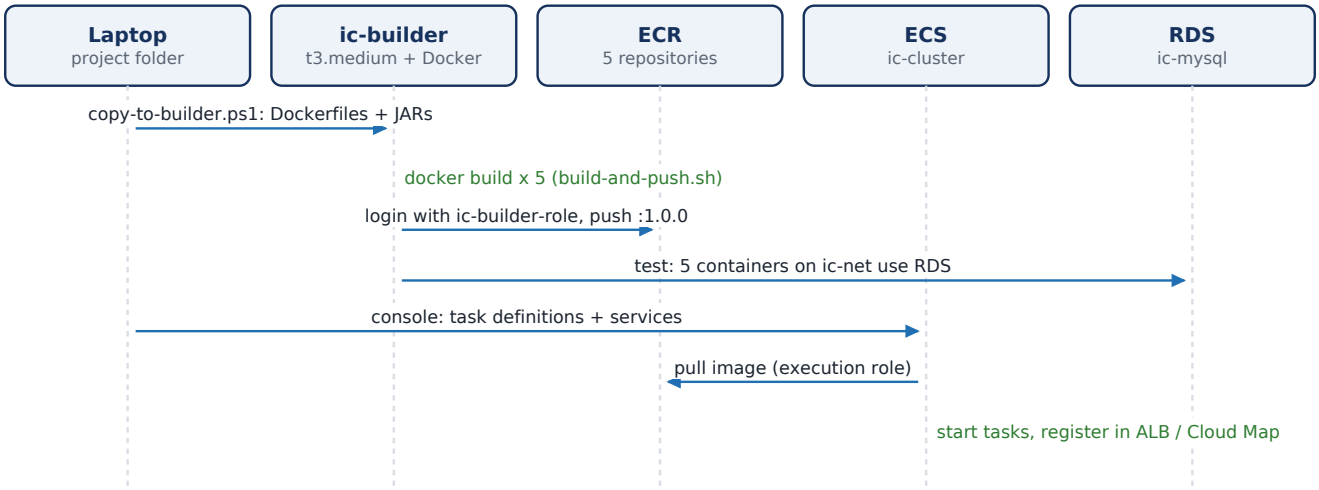


Figure 7.2 - The order of events: build, push, test, then let ECS pull and run.

Step	Section	Rough time
Builder EC2 (role, SG, launch)	7.2	15 min
ECR repositories, copy, build, push	7.3	20 min
Container test on the builder	7.4	10 min
Security groups, namespace, cluster, target group, ALB, IAM fix	7.5-7.6	30 min
Five task definitions	7.7	25 min
Five services	7.8	25 min + waiting
Roll out fixes, verify, React M2 site	7.9-7.10	30 min

7.1 Worksheet additions for Milestone 2

Add these to `ic-worksheet.txt` (6.1). Same four groups as for Milestone 1.

A. Save these - stable until you delete the resource

Value	Your value	Where to see it again
ECR registry	`169748358864.dkr.ecr.us-east-1.amazonaws.com`	ECR -> any repository -> URI
Repositories	`issue-tracker/eureka-server`, `/api-gateway`, `/user-service`, `/project-service`, `/issue-service`	ECR -> Repositories
Image tag	`1.0.0`	ECR -> repository -> Images
Builder role / SG	`ic-builder-role` / `ic-builder-sg`	IAM / EC2
ALB DNS name	`ic-alb-1965966534.us-east-1.elb.amazonaws.com`	EC2 -> Load balancers -> ic-alb
API address (M2)	`http://ic-alb-1965966534.us-east-1.elb.amazonaws.com/api`	built into the M2 React files
Target group	`ic-gateway-tg` (IP, HTTP 8080, `/actuator/health`)	EC2 -> Target groups
Security groups	`ic-alb-sg`; `ic-ecs-sg` = `sg-038843b3b0866b4b9`	EC2 -> Security Groups
Cloud Map namespace	`ic.local` (API calls only)	Cloud Map -> Namespaces
Cluster / services	`ic-cluster` / `ic-<app>-svc`	ECS -> Clusters
Task definitions in use	`ic-eureka-server:1`, `ic-api-gateway:4`, `ic-user-service:4`, `ic-project-service:4`, `ic-issue-service:4`	ECS -> service -> Deployments
Eureka name for clients	`http://eureka.ic.local:8761/eureka/`	task definition environment
Frontend bucket (M2)	`ic-frontend-m2-nlnu19`	S3 -> Buckets
App address (M2)	`http://ic-frontend-m2-nlnu19.s3-website-us-east-1.amazonaws.com`	bucket -> Properties -> Static website hosting

B. Do not rely on these - they change

Value	Example	Changes when
Builder public IP	13.220.31.5	every stop/start of ic-builder
Task private IPs	user 10.0.3.202, project 10.0.28.110, issue 10.0.16.201, gateway 10.0.12.192	every new task (deployment, crash, scale, restart from 0)
Task public IPs	-	every new task
Image digests `@sha256:...`	-	every push of changed code
Revision numbers	:4	every edit of a task definition

C. Secrets and D. No need to save

C: the three service passwords - typed into the task definitions and into the `read -s` prompts of 7.4, kept in the worksheet in aws-keys, nowhere else. **D:** task IDs, ENI IDs, CloudFormation stack names, ALB node IPs, the Envoy addresses 127.255.0.x - ECS manages them.

7.2 Step 1 - the builder EC2

Docker images are built on a temporary EC2 in the cloud, as in class: it needs no Docker Desktop on the company laptop, builds x86 images that match Fargate, and pushes to ECR with a role instead of access keys.

1. **Role:** IAM -> Roles -> Create role -> Trusted entity **AWS service -> EC2** -> attach **AmazonEC2ContainerRegistryPowerUser** -> name `ic-builder-role` -> Create. (Push and pull images; it cannot delete repositories.)
2. **Security group** `ic-builder-sg` in ic-vpc: inbound SSH 22 from **Anywhere-IPv4 0.0.0.0/0** (same reason as in Milestone 1: the company network breaks "My IP"; login still needs the key).
3. **ic-rds-sg** -> Edit inbound rules -> add MySQL/Aurora 3306, source **ic-builder-sg** (only needed for the container test in 7.4).
4. **Launch** EC2 -> Launch instance: name `ic-builder`, Amazon Linux 2023, **t3.medium** (Docker builds need memory), key `ic-key`, VPC ic-vpc, subnet **public1**, auto-assign public IP **Enable**, security group **ic-builder-sg** only, storage **20 GiB** gp3, Advanced -> IAM instance profile **ic-builder-role**, user data:

```
#!/bin/bash
# Builder: Docker for building and testing images (the AWS CLI is already part of Amazon Linux 2023)
dnf install -y docker
systemctl enable --now docker
usermod -aG docker ec2-user
```

5. Wait for "3/3 checks passed" plus two minutes, copy the **public IPv4**, and from the aws-keys folder:

```
ssh -i .\ic-key.pem ec2-user@<BUILDER-PUBLIC-IP>
docker --version && aws --version
aws sts get-caller-identity     # Account 169748358864, ...assumed-role/ic-builder-role/... (no keys)
```

WHAT WE ACTUALLY DID
The builder was a new EC2 (not one of the Milestone 1 servers). It ran at `13.220.31.5`; after any stop/start it has a new address. `get-caller-identity` showing `assumed-role/ic-builder-role` proves the role works.

WATCH OUT
`permission denied while trying to connect to the Docker daemon socket`: the docker group applies only to new logins - type `exit`, SSH in again.

7.3 Step 2 - ECR repositories, copy, build and push

1. **Repositories first** (a push to a missing repository fails): ECR -> Private registry -> Repositories -> **Create repository**, five times, default settings: `issue-tracker/eureka-server`, `issue-tracker/api-gateway`, `issue-tracker/user-service`, `issue-tracker/project-service`, `issue-tracker/issue-service`.
2. **JARs:** in Spring Tools, `clean package` for all five projects (5.11).
3. **Copy** - laptop PowerShell, in the project folder, with the builder's IP **only**:

```
powershell -ExecutionPolicy Bypass -File .\aws\scripts\copy-to-builder.ps1 -Server 13.220.31.5
```

WHAT WE ACTUALLY DID
First run with `-Server ec2-user@13.220.31.5` -> the script built `ec2-user@ec2-user@...` and failed: it adds the user itself. Second run: the eureka-server JAR stopped with "broken pipe" (connection dropped mid-copy) - running the script again copied everything cleanly. It is safe to repeat: every file is simply overwritten.

4. **Build and push** - on the builder (the script is in the home folder):

```
bash build-and-push.sh
docker images          # five issue-tracker/* images
```

The script builds the five images, logs Docker in to ECR with the instance role (token valid 12 hours), tags and pushes them, and prints the five image URIs (5.10.5 explains every line).

CHECKPOINT
ECR -> each repository shows an image with tag `1.0.0`. Image URI form: `169748358864.dkr.ecr.us-east-1.amazonaws.com/issue-tracker/user-service:1.0.0`.

7.4 Step 3 - test the five containers on the builder

Before ECS, the images were started together on one Docker network, with the services connected to RDS - the same settings the tasks get later. If this works, anything that fails on ECS is configuration, not the image. `read -s` keeps the passwords off the screen and out of the history.

```
docker network create ic-net
read -p "RDS endpoint: " DB
read -s -p "user_svc pw: " PU; echo
read -s -p "project_svc pw: " PP; echo
read -s -p "issue_svc pw: " PI; echo
COMMON="--network ic-net -e SPRING_PROFILES_ACTIVE=aws -e EUREKA_URL=http://eureka:8761/eureka/ -e DB_HOST=$DB"

docker run -d --name eureka --network ic-net issue-tracker/eureka-server:1.0.0
docker run -d --name user-service $COMMON -e DB_USERNAME=user_svc \
  -e DB_PASSWORD="$PU" issue-tracker/user-service:1.0.0
docker run -d --name project-service $COMMON -e DB_USERNAME=project_svc \
  -e DB_PASSWORD="$PP" issue-tracker/project-service:1.0.0
docker run -d --name issue-service $COMMON -e DB_USERNAME=issue_svc \
  -e DB_PASSWORD="$PI" issue-tracker/issue-service:1.0.0
docker run -d --name api-gateway --network ic-net -p 8080:8080 -e SPRING_PROFILES_ACTIVE=aws \
  -e EUREKA_URL=http://eureka:8761/eureka/ -e FRONTEND_ORIGIN=http://localhost \
  issue-tracker/api-gateway:1.0.0

docker ps                                  # five containers "Up"
docker logs -f user-service                # "Started UserServiceApplication", Ctrl+C
sleep 60; curl -s localhost:8080/api/users | head -c 300; echo
docker rm -f eureka user-service project-service issue-service api-gateway   # clean up
```

HOW TO PASTE THIS BLOCK
Paste the five `read` lines **one at a time** and answer each prompt before pasting the next. A `read` consumes the next input line - if the whole block is pasted at once, the following commands are swallowed as "passwords". A line ending in `\` continues on the next line: copy both together.

WHAT WE ACTUALLY DID
`docker ps -a` showed issue-service as **Exited**. `docker logs issue-service` pointed at the database login (MySQL answers a wrong password with "Access denied for user ...") - the issue_svc password had been mistyped at the prompt. Removing that container and running its two lines again with the right password fixed it; the curl through the containerised gateway then returned the users from RDS.

CHECKPOINT
Five containers Up, `/api/users` JSON from the gateway container. Clean up with the last line; the builder can now be **stopped** (images are in ECR).

7.5 Step 4 - security groups and the Cloud Map namespace

a) Security groups (VPC ic-vpc)

Group	Inbound rules	Attached to
`ic-alb-sg`	HTTP 80 from Anywhere-IPv4	the ALB
`ic-ecs-sg`	TCP 8080 from **ic-alb-sg**; TCP 8761 from **ic-ecs-sg**; TCP 8080-8083 from **ic-ecs-sg**	every task
`ic-rds-sg` (edit)	add MySQL 3306 from **ic-ecs-sg** (ic-backend-sg and ic-builder-sg stay)	RDS

`ic-ecs-sg` references itself: a rule whose source is the same group means "any task may reach any other task on these ports" - Eureka (8761) and the services (8080-8083). Create the group with the ALB rule first, save, then edit it and add the two self-rules (the group must exist before it can be chosen as a source).

b) The namespace - three attempts

Attempt	What happened	Why	Lesson
1 name `" ic.local"`	the console rejected the name	a leading space (pasted) - not a valid DNS name	type names, check for spaces
2 `ic.local`, "API calls and DNS queries in VPCs"	`not authorized to perform route53:CreateHostedZone`	a DNS namespace creates a Route 53 private hosted zone; the sandbox forbids it	implicit deny - change the design
3 `ic.local`, **"API calls"** only	created	no Route 53 zone needed	names reach the tasks through Service Connect (7.8)

1. Cloud Map -> Namespaces -> **Create namespace** -> name `ic.local` -> Instance discovery **API calls** -> Create.

> **WHY IT IS DONE THIS WAY**
> An API-only namespace stores names that only the AWS API (and ECS Service Connect) can answer - there is no DNS record a normal program could resolve. Service Connect bridges that: it puts `eureka.ic.local` into each task's /etc/hosts and proxies the connection (3.11, Figure 3.29).

7.6 Step 5 - cluster, load balancer and the IAM fix

a) Cluster

1. ECS -> Clusters -> **Create cluster** -> name `ic-cluster` -> Infrastructure **AWS Fargate (serverless)** only -> Monitoring: **Container Insights with enhanced observability** -> Create (the default namespace setting can be left; services choose `ic.local` themselves).

b) Target group and ALB

1. EC2 -> Target groups -> **Create target group** -> type **IP addresses** -> name `ic-gateway-tg` -> HTTP **8080** -> VPC ic-vpc -> health check path `/actuator/health` -> Next -> register nothing (ECS does it) -> Create.
2. EC2 -> Load balancers -> Create -> **Application Load Balancer** -> `ic-alb` -> Internet-facing, IPv4 -> VPC ic-vpc -> AZs 1a and 1b with their **public** subnets -> security group **ic-alb-sg** only (remove default) -> listener HTTP:80 -> forward to **ic-gateway-tg** -> Create. Copy the DNS name into the worksheet.

WHAT WE ACTUALLY DID
The first attempt failed with "explicit deny" on `elasticloadbalancing:DescribeTargetGroups` and `ec2:DescribeVpcs`: the bootcamp account had been **temporarily suspended**. Nothing was wrong with the steps; after reactivation the target group and ALB were created as above.

c) Let the execution role create log groups - before the first service

The task definitions use "Use log collection" with `awslogs-create-group=true`: ECS creates `/ecs/ic-<app>` when the first task starts - acting as `ecsTaskExecutionRole`. Its managed policy may write logs but not create groups, so the first service failed (circuit breaker, CloudFormation `CREATE_FAILED`, task stopped with `ResourceInitializationError ... logs:CreateLogGroup`). The fix:

1. IAM -> Roles -> **ecsTaskExecutionRole** -> Add permissions -> **Create inline policy** -> JSON.
2. Paste Listing 3.1 (action `logs:CreateLogGroup`, resource `arn:aws:logs:us-east-1:169748358864:log-group:/ecs/*`) -> Next -> name `ic-ecs-create-log-groups` -> Create policy.
3. The warning "not authorized to perform access-analyzer:ValidatePolicy" is only the console's optional policy checker - ignore it.

NOTE
`ecsTaskExecutionRole` itself is created by the console the first time a task definition chooses "Create new role". If it already exists, just add the inline policy.

7.7 Step 6 - the five task definitions (final values)

ECS -> Task definitions -> **Create new task definition**, once per application. The values below are those of the revisions that run now; 7.9 explains how revisions 2-4 came about.

Field	Value (all five)
Family	`ic-eureka-server`, `ic-api-gateway`, `ic-user-service`, `ic-project-service`, `ic-issue-service`
Launch type / OS	AWS Fargate; **Linux/X86_64** (the builder is x86)
Task size	**0.5 vCPU**, **1 GB**
Task role	none
Task execution role	`ecsTaskExecutionRole` (+ inline policy of 7.6 c)
Container name	the app name: `eureka-server`, `api-gateway`, `user-service`...
Image URI	**Browse ECR images** -> repository -> tag 1.0.0 (stored as `...@sha256:` digest)
Essential	Yes
Port mapping	container port = the app's `server.port` (table below), TCP, app protocol HTTP
Log collection	Use log collection -> Amazon CloudWatch (group `/ecs/ic-<app>`, created automatically)

Task definition	Container port	Environment variables
`ic-eureka-server:1`	**8761**	none
`ic-api-gateway:4`	**8080**	`SPRING_PROFILES_ACTIVE=aws`, `EUREKA_URL=http://eureka.ic.local:8761/eureka/`, `FRONTEND_ORIGIN=http://ic-frontend-m2-nlnu19.s3-website-us-east-1.amazonaws.com`, `SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0`
`ic-user-service:4`	**8081**	`SPRING_PROFILES_ACTIVE=aws`, `EUREKA_URL=` (as above), `DB_HOST=ic-mysql.cijoc4i6c1pv.us-east-1.rds.amazonaws.com`, `DB_USERNAME=user_svc`, `DB_PASSWORD=` (typed, secret), `SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0`

Task definition	Container port	Environment variables
ic-project-service:4	**8082**	as user-service, DB_USERNAME=project_svc and its password
ic-issue-service:4	**8083**	as user-service, DB_USERNAME=issue_svc and its password

THE PORT SLIP
Revision 1 of four task definitions had the ports shifted by one row (user 8080, project 8081, issue 8082, gateway 8083). The apps listened on their real ports, so nothing matched. Check: the port in the task definition = `server.port` in that app's application.properties.

THE PREFERRED-NETWORKS VARIABLE
`SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0` was added after the first working rollout showed every service registered as `169.254.172.2` (Fargate's internal interface, 3.9). It tells Spring to register the 10.0.x.x VPC address. Eureka itself registers nothing, so it does not need it and stayed at revision 1.

7.8 Step 7 - the five services

ic-cluster -> Services -> **Create**, in this order: **eureka** -> user -> project -> issue -> **gateway**. Eureka first, so the name `eureka.ic.local` exists before any client starts.

Section	Value (all services)
Compute options	Launch type **FARGATE**, platform version LATEST (1.4.0)
Deployment configuration	Application type **Service**; task definition family `ic-<app>`, latest revision; service name `ic-<app>-svc`; desired tasks **1**
Deployment options	rolling update, deployment circuit breaker with rollback (defaults)
Networking	VPC ic-vpc; subnets **public1 + public2** only; security group **ic-ecs-sg** only (remove default); **Public IP: turned on** (to pull images from ECR without a NAT gateway)
Service Connect	turned on, namespace **ic.local** (mode per service below)
Service auto scaling	leave off - the sandbox denies `application-autoscaling` (explicit deny)

Service	Service Connect	Load balancing
ic-eureka-server-svc	**Client and server**: port alias = the 8761 mapping, discovery name eureka, client alias DNS eureka.ic.local, port 8761	none
ic-user-service-svc, ic-project-service-svc, ic-issue-service-svc	**Client side only**	none
ic-api-gateway-svc	**Client side only**	Application Load Balancer -> **existing** ic-alb -> container api-gateway 8080:8080 -> existing listener 80:HTTP -> existing target group ic-gateway-tg -> **health check grace period 120 s**

- Each create takes 1-3 minutes: the console builds a CloudFormation stack (`ECS-Console-V2-Service-...`), ECS starts the task, pulls the image, Spring Boot starts (about a minute on 0.5 vCPU).
- Watch: service -> **Tasks** (Running), **Logs** (or CloudWatch `/ecs/ic-<app>`), and for the gateway the target group -> Targets (healthy).
- The grace period keeps ECS from killing the gateway while Spring Boot is still starting and `/actuator/health` does not yet answer.

WHAT WE ACTUALLY DID
The first eureka service hit the log-group error (7.6 c) and the circuit breaker; after the inline policy it ran. The other four were created the same way. The gateway answered with a **500 Whitelabel** page - the services were registered as 169.254.172.2 - which led to the port fix and the preferred-networks revisions (7.9).

7.9 Step 8 - changing a running service: new revision, then deploy it

A task definition revision cannot be edited. Every fix is: create a new revision, then tell the service to use it. Creating the revision alone changes nothing that runs - exactly what happened once: after "Update service" without choosing the new revision, the service stayed on revision 2.

1. ECS -> Task definitions -> `ic-user-service` -> select the latest revision -> **Create new revision** -> change the field (port, environment variable) -> Create. A new number appears (`:4`).
2. With that revision selected: **Deploy -> Update service** -> cluster ic-cluster -> service `ic-user-service-svc` -> Update. (Equivalent: service -> Update service -> pick the revision explicitly.)
3. Service -> **Deployments**: the new deployment shows the new revision as PRIMARY; the old one drains.
4. Repeat for project, issue and the gateway.
5. **Force new deployment** (Update service -> tick it) restarts tasks with the SAME revision - for example after pushing a new image under the same tag.

Revision history	What changed
`ic-eureka-server:1`	unchanged since creation
`:1` of the four others	first version - ports shifted for some
`:2` / `:3`	port corrections and environment adjustments while debugging
`:4` (running)	correct port + `SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0`

NOTE
During a deployment the old gateway task stays in the target group as "draining" for up to 5 minutes (deregistration delay 300 s) - normal, not an error.

7.10 Step 9 - verify, then the React site for Milestone 2

1. CloudWatch -> Log groups -> `/ecs/ic-user-service` -> newest stream: "Registering application USER-SERVICE with eureka with status UP" and "registration status: 204". The instance id contains **10.0.x.x**.
2. EC2 -> Target groups -> `ic-gateway-tg` -> Targets: one **healthy** target 10.0.12.192:8080.
3. Browser: `http://ic-alb-1965966534.us-east-1.elb.amazonaws.com/api/users` -> the users as JSON (the same data as Milestone 1: one database).

WHAT WE ACTUALLY DID
Final registrations: user 10.0.3.202:8081, project 10.0.28.110:8082, issue 10.0.16.201:8083, gateway 10.0.12.192:8080. `/api/users` returned JSON through the ALB.

The React site - a second build, because the API address is compiled in (5.10.3):

```
.\aws\scripts\build-frontend.ps1 -ApiUrl http://ic-alb-1965966534.us-east-1.elb.amazonaws.com/api
# if scripts are blocked: powershell -ExecutionPolicy Bypass -File .\aws\scripts\build-frontend.ps1 -ApiUrl ...
```

4. S3 -> Create bucket `ic-frontend-m2-nlnu19` (us-east-1) -> **untick Block all public access** and acknowledge -> Create.
5. Permissions -> Bucket policy -> Listing 3.3 (`PublicReadWebsite`, resource `arn:aws:s3:::ic-frontend-m2-nlnu19/*`) -> Save.
6. Properties -> Static website hosting -> Enable, index document `index.html`, error document `index.html` -> Save; copy the website endpoint.
7. Objects -> Upload -> the **contents** of `issue-tracker-ui\dist` (index.html, favicon, assets folder) -> Upload.
8. The gateway's `FRONTEND_ORIGIN` (7.7) is exactly this website address - otherwise CORS blocks the calls.
9. Open `http://ic-frontend-m2-nlnu19.s3-website-us-east-1.amazonaws.com`, log in as priya@its.com, open a board.

> **WHY IT IS DONE THIS WAY**
> Two buckets let both milestones be demonstrated side by side: the M1 site talks to the Elastic IP, the M2 site to the ALB, both read and write the same RDS data.

7.11 Milestone 2 status

Case-study task	Status	Evidence
A. Containerise: Dockerfiles, build on EC2	done	5 Dockerfiles; builder ic-builder; container test with RDS
B. ECR repositories and push	done	5 repositories, tag 1.0.0
C. ECS cluster, task definitions, services	done	ic-cluster, 5 services 1/1 running, revisions :1/:4
C. Service discovery	done	Eureka + Service Connect name `eureka.ic.local` (API-only Cloud Map)
C. Load balancing	done	ALB ic-alb -> ic-gateway-tg, healthy
D. RDS from containers	done	same RDS via environment variables
E. CloudWatch	logs + automatic metrics + Container Insights; dashboards/alarms as theory	`/ecs/ic-<app>`
F. Orchestration proof (stop a task, scale)	to show in the demo (7.12)	-
React for M2	done	`http://ic-frontend-m2-nlnu19.s3-website-us-east-1.amazonaws.com`
Lambda	theory only (designed and coded)	3.14, 5.6, 6.14

7.12 Proving orchestration in the demo

1. **Self-healing:** ECS -> ic-cluster -> `ic-user-service-svc` -> Tasks -> select the task -> **Stop**. Within about a minute a new task appears with a new 10.0.x.x address; its log shows a new Eureka registration. The app recovers after a short error while the instance is replaced.
2. **Scale out:** Update service -> desired tasks **2** -> Update. Two tasks run, both register (different instance ids thanks to the IP in the id), and the gateway alternates between them - visible as requests in both log streams. Set it back to 1 afterwards.
3. **Auto scaling:** explain target tracking (keep CPU at 60%, min 1, max 3) and that the sandbox explicitly denies `application-autoscaling`, so it was demonstrated by hand.
4. **Rolling update:** show the Deployments tab of a service - revisions, PRIMARY vs ACTIVE, circuit breaker.

7.13 CloudWatch for Milestone 2 - what exists, what is theory

Item	Status	Where
Container logs	exist	CloudWatch -> Log groups -> `/ecs/ic-<app>`, one stream per task
Service CPU / memory	exist (automatic)	ECS -> service -> Health and metrics; metrics `AWS/ECS`
Container Insights	exists (enabled on the cluster)	CloudWatch -> Insights -> Container Insights
ALB metrics	exist (automatic)	RequestCount, TargetResponseTime, HTTPCode_Target_5XX_Count, UnHealthyHostCount
Logs Insights query	available any time	query of 3.13 over all `/ecs/` groups
Dashboard `ic-dashboard`, alarms + SNS e-mail	theory (designed in 6.15)	e.g. UnHealthyHostCount >= 1 for 2 min; service CPU > 70%
Log retention	recommended: set 14 days	log group -> Actions -> Edit retention

7.14 What persists, pausing and the demo day

Resource	While paused (services at 0, builder stopped)	After resuming
Task definitions, revisions	kept, free	unchanged
ECR images	kept (cents per month)	unchanged - no rebuild needed
Services at desired 0	kept; **no task cost**	set back to 1
Tasks	deleted	new tasks, **new private and public IPs** - Eureka, Service Connect and the ALB adapt by themselves; nothing to edit
ALB + target group	keep running and **keep billing** (~$0.55/day)	same DNS name - the M2 React build stays valid
Cloud Map namespace	kept	unchanged
Log groups	kept	new streams for new tasks
ic-builder	stopped: only its 20 GiB disk is billed	new public IP (only needed for a rebuild)
RDS (shared with M1)	may be stopped up to 7 days	start it first and wait for "Available"
S3 site	always on	unchanged

Pause

1. ECS -> ic-cluster -> each service -> **Update service** -> desired tasks **0** -> Update (gateway first, Eureka last).
2. EC2 -> ic-builder -> Instance state -> **Stop** (if it is running).
3. Optional, only for a long pause: RDS -> Stop temporarily (7 days max, then AWS starts it). Do **not** delete the ALB unless pausing for weeks - a new ALB has a new DNS name and needs a new React build.

Demo day - bring Milestone 2 back (about 10 minutes)

1. RDS -> ic-mysql -> status **Available** (start it if stopped; ~5 minutes).
2. `ic-eureka-server-svc` -> desired **1** -> wait until the task is Running (~1-2 minutes).
3. `ic-user-service-svc`, `ic-project-service-svc`, `ic-issue-service-svc` -> desired **1** each.
4. `ic-api-gateway-svc` -> desired **1** -> target group shows **healthy** (grace period 120 s).
5. Check `http://ic-alb-1965966534.us-east-1.elb.amazonaws.com/api/users`, then open the M2 site and log in.
6. Show the self-healing and scaling steps of 7.12, the Deployments tab, the log groups and Container Insights.
7. Afterwards: desired 0 again (Pause above).

WHY RESUMING IS EASY

Nothing in the M2 configuration contains a task IP, so a resume needs no edits at all. The only addresses that matter - the ALB DNS name, `eureka.ic.local` and the RDS endpoint - never change while those resources exist.

7.15 Problems met during Milestone 2

#	What we saw	Cause	Fix
1	copy-to-builder failed to connect	`-Server` given as `ec2-user@IP` (the script adds the user)	`-Server` = IP only
2	"broken pipe" while copying the eureka JAR	connection dropped	run the script again
3	pasted test block: commands vanished into prompts	`read` takes the next line as input	paste `read` lines one at a time
4	issue-service container Exited	mistyped issue_svc password	`docker logs`, re-run with the right one
5	`elasticloadbalancing:DescribeTargetGroups` / `ec2:DescribeVpcs` ... explicit deny	account temporarily suspended	wait for reactivation
6	namespace name rejected	leading space `" ic.local"`	type `ic.local`
7	`not authorized to perform route53:CreateHostedZone`	DNS namespace needs Route 53; sandbox denies	API-only namespace + Service Connect
8	circuit breaker, CloudFormation `CREATE_FAILED`, task stopped: `... logs:CreateLogGroup`	execution role cannot create log groups	inline policy `ic-ecs-create-log-groups`
9	task definitions: ports shifted by one row (user 8080 ... gateway 8083)	a slip in the form	new revisions with 8761/8080/8081/8082/8083
10	gateway 500 Whitelabel; logs show `169.254.172.2` registrations	Spring chose Fargate's link-local interface	`SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0`
11	still old behaviour after "Update service"	service still on revision 2	Task definition -> Deploy -> Update service, check Deployments
12	`application-autoscaling:...` explicit deny	sandbox restriction	scale by desired count
13	`access-analyzer:ValidatePolicy` not authorized	optional console check	ignore
14	clean-up before M2: "cluster cannot be deleted" (old class cluster)	tasks still stopping	stop tasks / services first, wait, then delete

INTERVIEW ANGLE

"The hardest part of Milestone 2 was not Docker but the platform details: a sandbox that denied Route 53, so we used Service Connect on an API-only namespace; an execution role that could not create log groups; and Fargate's link-local interface, which made Spring register 169.254.172.2 in Eureka. Each one was found in the task's stopped reason or its CloudWatch log."

PART 8

Milestone 3 - GitHub

Commit the deployment changes and push the build to the repository - Deloitte GitHub Enterprise, as the case-study guidelines require.

8.1 What goes into Git

Commit	Never commit
All source code, `application-aws.properties`, Dockerfiles, `aws/` scripts with `CHANGE_ME` placeholders, `app.env.example`, README updates	`*.pem`, `*.ppk`, real `app.env`, `db.properties`, SQL files with real passwords, `photo-lambda.zip`, `target/`, `node_modules/`, `dist/`, AWS access keys

8.2 Steps

```
cd issue-tracking-system
git status                          # check: no .pem, app.env, db.properties, zip in the list
git switch -c feature/aws-deployment
git add .
git commit -m "AWS deployment: aws profile, configurable CORS, Dockerfiles, Lambda, scripts"
git push -u origin feature/aws-deployment
# on GitHub Enterprise: open a Pull Request into main, review the diff, merge
git switch main && git pull
git tag v1.0-aws && git push origin v1.0-aws
```

1. If the repository does not exist yet: create it in the Deloitte GitHub Enterprise organisation (no README, no .gitignore - the project has them), then `git remote add origin <https URL>` and push `main` first.
2. Add a "Deployment" section to the main README: the architecture figures, the environment variables of each application, and a link to this runbook.

WATCH OUT
If a secret was ever committed, deleting the file later does not remove it from the history. Change the password (RDS: Modify -> new master password; MySQL: `ALTER USER ... IDENTIFIED BY ...`) and rotate any key.

PART 9

Troubleshooting, cost and clean-up

The problems you are most likely to meet, what causes them and how to fix them; what costs money; and how to remove everything safely.

9.1 Troubleshooting

Symptom	Likely cause	Fix
`ssh` times out	SSH rule limited to "My IP" (company proxy); instance in a private subnet; no public IP	SSH 22 from 0.0.0.0/0 in ic-backend-sg / ic-builder-sg; public subnet with auto-assign public IP
"UNPROTECTED PRIVATE KEY FILE"	Windows file permissions on ic-key.pem	the two `icacls` commands in 6.3
`mysql` hangs after the password	ic-rds-sg does not allow the caller's security group	add MySQL 3306 from that group (6.6)
scp says "1 file(s) copied."	destination without `:~` - local copy	add `:~`; delete the stray file
Service exits at start: "Could not resolve placeholder EUREKA_URL"	app.env missing a line or the profile variable	check `/etc/issue-tracker/app.env`; `sudo systemctl restart issue-tracker`
"Communications link failure" to MySQL	ic-rds-sg does not allow the caller's SG; wrong endpoint	add the SG rule; copy the endpoint again
"Access denied for user 'user_svc'@'10.0...'"	wrong password, or the user was created for another host	run the SQL again / `ALTER USER`
Eureka lists services as `localhost`	`aws` profile not active	`SPRING_PROFILES_ACTIVE=aws` in app.env or the task definition
Gateway returns 503 for /api/...	target service not registered yet or stopped	wait 30 s; check Eureka and the service log
Browser: "blocked by CORS policy"	`FRONTEND_ORIGIN` differs from the address bar (http vs https, trailing slash, other bucket endpoint form)	copy the website endpoint exactly; restart the gateway / new task revision
React app shows "Cannot reach the server"	wrong `VITE_API_BASE_URL` in the build, gateway down, port 8080 not open	rebuild with the right address; check ic-gateway-public-sg
404 when refreshing a page on S3	error document not `index.html`	Static website hosting -> Error document index.html
403 on the S3 website	Block Public Access still on or no bucket policy	turn off BPA for the bucket; add Listing 6.1
Lambda time-out	S3 endpoint missing on private route tables, or RDS SG	see the table in 6.14 g
Lambda "No module named pymysql"	library not at the root of the zip	rebuild: `pip3 install --target .` inside the folder
ECS task stops: "CannotPullContainerError"	no public IP on the task, wrong image URI, or missing execution role	turn Public IP on; copy the URI from ECR; select ecsTaskExecutionRole
ECS gateway task keeps restarting	ALB health check fails during start-up	grace period 120-180 s; health path /actuator/health
Out of memory / very slow on t3.micro	1 GiB is tight for the JVM + OS	`-Xmx320m`, or t3.small
"not authorized to perform iam:CreateRole"	bootcamp account restrictions	ask the trainer for an existing role
ECS: circuit breaker / CloudFormation `CREATE_FAILED`; task stopped with `ResourceInitializationError ... logs:CreateLogGroup`	the execution role may not create the `/ecs/ic-<app>` log group	inline policy Listing 3.1 (7.6 c)
Cloud Map: `not authorized to perform route53:CreateHostedZone`	DNS namespaces need a Route 53 zone; sandbox denies it	API-only namespace + Service Connect (7.5, 7.8)

Symptom	Likely cause	Fix
Gateway 500 Whitelabel on ECS; Eureka shows `169.254.172.2`	Spring registered Fargate's link-local interface	`SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0`, new revision (3.9)
Task running but unreachable / unhealthy	container port in the task definition differs from `server.port`	new revision with 8761/8080/8081/8082/8083
Change made, behaviour unchanged	the service still runs the old revision	Deploy -> Update service; check the Deployments tab
Client cannot reach `eureka.ic.local`	Service Connect off on that service, or the Eureka service is not "Client and server"	Update service -> Service Connect (7.8); redeploy
`explicit deny` on describe calls everywhere	the bootcamp account is (temporarily) suspended	wait for reactivation
M1 site works at home, times out in the office	the office network blocks port 8080	demo M1 from a personal network or hotspot; M2 uses port 80

9.2 What costs money (approximate, us-east-1)

Resource	Approximate price	Tip
EC2 t3.micro x5 / t3.medium builder	~$0.0104 / ~$0.0416 per hour each	stop them when not demonstrating
Public IPv4 addresses (incl. Elastic IP)	~$0.005 per hour each	release the Elastic IP after the demo
RDS db.t3.micro + 20 GB	~$0.017 per hour + storage	stop it (up to 7 days) or delete with a snapshot
Fargate 0.5 vCPU / 1 GB	~$0.025 per hour per task	set desired tasks to 0 between demos
Application Load Balancer	~$0.0225 per hour + usage	bills even with all services at 0; delete only after the last demo
S3, Lambda, ECR, CloudWatch	cents at this scale	set log retention (14 days)
NAT gateway (not used)	~$0.045 per hour + data	the design avoids it

Prices change and differ by region; check the AWS pricing pages and the Billing console. Free-tier and bootcamp credits may cover part of it.

9.3 Clean-up order

1. ECS: set every service to 0 tasks, delete the five services, wait until no task is "stopping", then delete the cluster (it refuses while tasks are still stopping); deregister task definitions (optional).
2. Delete the ALB, then the target group. Delete the Cloud Map namespace (after the eureka service is gone).
3. ECR: delete the five repositories (with their images).
4. Lambda (only if it was deployed): delete the function and the S3 event notification. S3: empty and delete the buckets (`ic-frontend-nlnu19`, `ic-frontend-m2-nlnu19`, and `ic-photos-...` if created).
5. EC2: terminate the six instances (five app servers + ic-builder); **release the Elastic IP**; delete the key pair if no longer needed.
6. RDS: take a final snapshot, then delete `ic-mysql` (and the subnet group).
7. Delete the security groups, the IAM roles created here (and the inline policy on ecsTaskExecutionRole), any SNS topic, alarms or dashboard, and the log groups.
8. Delete ic-vpc last (this removes subnets, route tables, the Internet Gateway and the S3 endpoint).

PART 10

Interview questions and answers

Questions about AWS, the design and the case study, with short answers to give in your own words.

Architecture and design

Q. Walk me through your Milestone 1 architecture.

React on an S3 static website; five Spring Boot apps on five EC2s in a VPC - Eureka, the gateway with an Elastic IP, and three services, each run by systemd; one private RDS MySQL with a schema and a user per service; the CloudWatch agent for logs and memory. The S3-triggered Lambda that writes photo URLs to RDS is designed and coded; I can walk through it.

Q. What changes between Milestone 1 and 2?

Packaging and runtime: the same JARs become Docker images in ECR, run as ECS services on Fargate. An ALB replaces the Elastic IP, ECS Service Connect gives Eureka the name eureka.ic.local, logs go to CloudWatch through the awslogs driver, and ECS restarts and replaces tasks. The database is the same.

Q. Why did the code need changes for AWS?

Every service registered itself in Eureka as "localhost" and used a local database. The `aws` profile reads the Eureka URL and RDS settings from environment variables and registers the private IP. The gateway reads its CORS origin from configuration. One JAR then runs everywhere.

Q. Why one RDS instance for three services?

Cost and simplicity. Isolation is kept with three schemas and three MySQL users, each limited to its own schema.

Q. Why Eureka and not Cloud Map for everything?

No code change, and the same model in both milestones. Only Eureka needs a stable address: ECS Service Connect provides eureka.ic.local on an API-only Cloud Map namespace (a DNS namespace was denied by the sandbox). Replacing Eureka with Service Connect for all services is a valid AWS-native alternative.

Q. Why Fargate instead of the EC2 launch type?

No servers to patch or scale; pay per task; the case study asks for it. The EC2 launch type is cheaper at large, steady scale.

Networking and security

Q. What makes a subnet public?

Its route table has `0.0.0.0/0` pointing to an Internet Gateway. Resources also need a public IP.

Q. Security group vs network ACL?

A security group is stateful, allow-only, on a network interface, and can reference other groups. A NACL is stateless, allows and denies, numbered rules, on a subnet.

Q. How did you apply least privilege?

Security groups that reference each other (RDS only from the app, Lambda and ECS groups); only the gateway/ALB public; roles instead of keys; a Lambda role that reads one prefix of one bucket; per-service DB users; a column-level grant for the Lambda.

Q. How does the Lambda reach S3 without internet?

Through an S3 gateway VPC endpoint - a route in the private route tables. It is free, unlike a NAT gateway.

Q. How would you add HTTPS?

A domain in Route 53, a certificate from ACM, an HTTPS listener on the ALB, and CloudFront in front of the S3 bucket (with Origin Access Control, so the bucket can be private).

Q. Is your API secure?

Internal services are not reachable from the internet, and DB connections are encrypted. Per-request authentication (JWT or Cognito) is the planned next step - the API does not check tokens yet.

Services

Q. What is an IAM role and why use it on EC2?

An identity with temporary credentials that a service assumes. The CloudWatch agent and Docker push work without storing access keys on the server.

Q. What happens when S3 invokes your Lambda twice for the same upload?

Nothing harmful. The function is idempotent: it sets the same URL again (FOUND_ROWS makes it report a match).

Q. What is a cold start?

The first invocation creates the execution environment and loads the code; later calls reuse it. We create the S3 client outside the handler for that reason.

Q. Task definition vs task vs service?

Blueprint vs running copy vs the controller that keeps N copies running, replaces failed ones and connects them to the load balancer.

Q. Task role vs task execution role?

The execution role is used by ECS to pull images and write logs; the task role is for the application's own AWS calls (none needed here).

Q. Multi-AZ vs read replica?

Multi-AZ is a synchronous standby for failover (availability). A read replica is an asynchronous copy for read scaling.

Q. How are backups handled?

RDS automated backups with 7-day point-in-time restore, plus manual snapshots before changes; S3 versioning; images versioned in ECR; code in Git.

Q. Scalability vs elasticity in your system?

Scalability: we can add tasks (user-service to 2) and both register in Eureka. Elasticity would be ECS service auto scaling with target tracking on CPU - explicitly denied in the sandbox, so we scale by hand.

Q. How do you monitor it?

CloudWatch Logs per service (agent on EC2, awslogs on ECS); automatic metrics for EC2/RDS/ECS/ALB; Container Insights on the cluster; Logs Insights queries for errors. Dashboards and alarms to an SNS e-mail topic are designed (6.15).

Q. What happens when an ECS task crashes?

The essential container exits, the task stops, and the service starts a replacement because running < desired. The new task has a new IP; it registers in Eureka and, for the gateway, in the target group - nothing to update by hand.

Q. How does the ALB know the gateway is healthy?

Every 30 s it calls /actuator/health on the target; consecutive 200s make it healthy, failures unhealthy. A 120 s grace period stops ECS from killing the task while Spring Boot starts.

Q. What is CORS and where did you handle it?

Browsers only let a page read responses from another origin if that server allows it. The S3 site and the API are different origins, so the gateway's CorsWebFilter allows exactly the site address from FRONTEND_ORIGIN.

Problems we solved (tell them as short stories)

Q. Fargate and Eureka

After the first rollout the gateway returned 500. The task logs showed every service registered as 169.254.172.2 - Fargate's link-local interface, which Spring picked first. Setting SPRING_CLOUD_INETUTILS_PREFERREDNETWORKS=10.0 in the task definitions made them register their VPC IPs; no new image was needed.

Q. The missing log-group permission

The first service rolled back by the circuit breaker. The stopped reason said the execution role was not authorised for logs:CreateLogGroup. A one-statement inline policy limited to /ecs/* fixed it - least privilege in practice.

Q. Service discovery inside a sandbox

A Cloud Map DNS namespace needs a Route 53 hosted zone, which the account denied. I switched to an API-only namespace and ECS Service Connect, which gives the tasks the name eureka.ic.local through an Envoy sidecar.

Q. RDS creation refused

The Dev/Test template was an implicit deny; the bootcamp policy allows only small instances. The Sandbox template with db.t3.micro worked.

Q. mysql hanging

Connections were silently dropped: the RDS security group referenced the wrong source. With the application security group as the source it connected at once - a hang means a firewall, a refusal means nothing listening.

Q. A fix that did not apply

A new task-definition revision changes nothing until the service is updated to it. The Deployments tab shows which revision is running.

Q. Port 8080 in the office

The Milestone 1 site worked at home but not in the office: the office network blocks outgoing 8080. Milestone 2 uses the ALB on port 80, which avoids that problem.

APPENDIX A

Remaining files, copy-paste ready

The files referred to in Part 5 that are not printed there in full.

A.1 - project-service/src/main/resources/application-aws.properties

```
# ================================================================================================
# AWS profile of project-service - loaded in addition to application.properties when SPRING_PROFILES_ACTIVE=aws.
# Every value that differs between the laptop and AWS comes from an environment variable, so the SAME
# JAR runs locally (default profile), on EC2 (Milestone 1) and on ECS Fargate (Milestone 2).
# ================================================================================================

# Eureka registry address, e.g. http://10.0.3.25:8761/eureka/ (EC2) or http://eureka.ic.local:8761/eureka/ (ECS)
eureka.client.service-url.defaultZone=${EUREKA_URL}
# Register this instance by its private IP instead of "localhost", so other machines can call it
eureka.instance.prefer-ip-address=true
# Unique per running copy - two tasks of the same service must not share an instance id
eureka.instance.instance-id=${spring.application.name}:${spring.cloud.client.ip-address}:${server.port}

# Amazon RDS (MySQL). DB_HOST = RDS endpoint. The schema is created by 01-schemas-and-users.sql.
# sslMode=REQUIRED: the connection to MySQL is encrypted.
spring.datasource.url=jdbc:mysql://${DB_HOST}:3306/project_db?sslMode=REQUIRED
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

A.2 - issue-service/src/main/resources/application-aws.properties

```
# ================================================================================================
# AWS profile of issue-service - loaded in addition to application.properties when SPRING_PROFILES_ACTIVE=aws.
# Every value that differs between the laptop and AWS comes from an environment variable, so the SAME
# JAR runs locally (default profile), on EC2 (Milestone 1) and on ECS Fargate (Milestone 2).
# ================================================================================================

# Eureka registry address, e.g. http://10.0.3.25:8761/eureka/ (EC2) or http://eureka.ic.local:8761/eureka/ (ECS)
eureka.client.service-url.defaultZone=${EUREKA_URL}
# Register this instance by its private IP instead of "localhost", so other machines can call it
eureka.instance.prefer-ip-address=true
# Unique per running copy - two tasks of the same service must not share an instance id
eureka.instance.instance-id=${spring.application.name}:${spring.cloud.client.ip-address}:${server.port}

# Amazon RDS (MySQL). DB_HOST = RDS endpoint. The schema is created by 01-schemas-and-users.sql.
# sslMode=REQUIRED: the connection to MySQL is encrypted.
spring.datasource.url=jdbc:mysql://${DB_HOST}:3306/issue_db?sslMode=REQUIRED
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

A.3 - eureka-server/Dockerfile

```
# Container image of eureka-server (Milestone 2).
# Build context = the eureka-server folder, after "mvn clean package" created target/eureka-server-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/eureka-server-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8761

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

A.4 - api-gateway/Dockerfile

```
# Container image of api-gateway (Milestone 2).
# Build context = the api-gateway folder, after "mvn clean package" created target/api-gateway-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/api-gateway-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8080

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

A.5 - project-service/Dockerfile

```
# Container image of project-service (Milestone 2).
# Build context = the project-service folder, after "mvn clean package" created
» target/project-service-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/project-service-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8082

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

A.6 - issue-service/Dockerfile

```
# Container image of issue-service (Milestone 2).
# Build context = the issue-service folder, after "mvn clean package" created target/issue-service-1.0.0.jar.
FROM amazoncorretto:21-alpine

# Run as an unprivileged user, never as root
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY target/issue-service-1.0.0.jar app.jar
USER app

# Documentation only: the port the application listens on
EXPOSE 8083

# Let the JVM size its heap from the container memory limit; "exec" makes Java PID 1 so it receives SIGTERM
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

A.7 - aws/lambda/profile-photo-processor/requirements.txt

```
pymysql>=1.1,<2
```

APPENDIX B

Command cheat sheet

Task	Command
SSH	`ssh -i ic-key.pem ec2-user@<public-ip>`
SSH via the gateway to a private address	`ssh -i ic-key.pem -J ec2-user@<gateway-eip> ec2-user@<private-ip>`
Tunnel a private port to the laptop	`ssh -i ic-key.pem -L 8761:<eureka-private-ip>:8761 ec2-user@<gateway-eip>`
Copy a file up / down	`scp -i ic-key.pem file ec2-user@<ip>:~` / `scp -i ic-key.pem ec2-user@<ip>:~/file .`
Service control	`sudo systemctl start|stop|restart|status issue-tracker`
Follow the app log	`tail -f /var/log/issue-tracker/app.log`
First-boot output	`sudo tail -n 50 /var/log/cloud-init-output.log`
CloudWatch agent status	`sudo systemctl status amazon-cloudwatch-agent`
MySQL client to RDS	`mysql -h <rds-endpoint> -u admin -p --ssl`
Who am I in AWS	`aws sts get-caller-identity`
Docker basics	`docker images`, `docker ps -a`, `docker logs -f <name>`, `docker rm -f <name>`
Build an image	`docker build -t issue-tracker/<app>:1.0.0 ./<app>`
ECR login	`aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <account>.dkr.ecr.us-east-1.amazonaws.com`
Tag + push	`docker tag issue-tracker/<app>:1.0.0 <registry>/issue-tracker/<app>:1.0.0` then `docker push ...`
List images in ECR	`aws ecr list-images --repository-name issue-tracker/<app> --region us-east-1`
React build for AWS	`.\aws\scripts\build-frontend.ps1 -ApiUrl http://<EIP:8080 or ALB>/api`
Masked app.env (screen share)	`sudo sed 's/PASSWORD=.*/PASSWORD=********/' /etc/issue-tracker/app.env`
Scale an ECS service (CloudShell / configured CLI)	`aws ecs update-service --cluster ic-cluster --service ic-user-service-svc --desired-count 1`
Copy Dockerfiles + JARs to the builder	`.\aws\scripts\copy-to-builder.ps1 -Server <builder-ip>`
Build + push all images (builder)	`bash build-and-push.sh [version]`